CSE 4345 Big Data Analytics

Final exam prep book · Weeks 5 to 10 · exam on 13 October 2026

Every end-of-chapter question from the slides (126 in all), copied as written, with the answer directly under it. Part 1 questions come from the Student Assessment Pool; Part 2 questions come from the Discussion Questions and Live Exercise. Chapters with calculations end with a maths section.

Contents

1. **Chapter 1 · Week 5** Handling Big Data Files: YARN, File Formats & Optimization
    - Part 1 · Student Assessment Pool: MCQ (4)
    - Part 1 · Student Assessment Pool: True/False (4)
    - Part 1 · Student Assessment Pool: Descriptive (3)
    - Part 1 · Student Assessment Pool: Analytical (2)
    - Part 2 · Discussion Questions (5)
    - Part 2 · Live Exercise (1)
    - Maths section
2. **Chapter 2 · Week 6** Apache Spark: RDDs, DataFrames & In-Memory Computing
    - Part 1 · Student Assessment Pool: MCQ (4)
    - Part 1 · Student Assessment Pool: True/False (4)
    - Part 1 · Student Assessment Pool: Descriptive (3)
    - Part 1 · Student Assessment Pool: Analytical (2)
    - Part 2 · Discussion Questions (5)
    - Part 2 · Live Exercise (4)
    - Maths section
3. **Chapter 3 · Week 7** Querying Big Data: Hive, HiveQL & the SQL-on-Hadoop Ecosystem
    - Part 1 · Student Assessment Pool: MCQ (4)
    - Part 1 · Student Assessment Pool: True/False (4)
    - Part 1 · Student Assessment Pool: Descriptive (3)
    - Part 1 · Student Assessment Pool: Analytical (2)

- Part 2 · Discussion Questions (3)
- Part 2 · Live Exercise (5)
- Maths section

4. **Chapter 4 · Week 8** Data Integration at Scale: Pipelines, Data Lakes & Modern Architectures
    - Part 1 · Student Assessment Pool: MCQ (4)
    - Part 1 · Student Assessment Pool: True/False (4)
    - Part 1 · Student Assessment Pool: Descriptive (3)
    - Part 1 · Student Assessment Pool: Analytical (2)
    - Part 2 · Discussion Questions (5)
    - Part 2 · Live Exercise (5)
    - Maths section
5. **Chapter 5 · Week 9** Data Visualization: Principles, Techniques & Big Data Tools
    - Part 1 · Student Assessment Pool: MCQ (4)
    - Part 1 · Student Assessment Pool: True/False (4)
    - Part 1 · Student Assessment Pool: Descriptive (3)
    - Part 1 · Student Assessment Pool: Analytical (2)
    - Part 2 · Discussion Questions (5)
    - Part 2 · Live Exercise (1)
    - Maths section
6. **Chapter 6 · Week 10** Big Data in Real Applications: Social Media, Time Series & Health Data
    - Part 1 · Student Assessment Pool: MCQ (4)
    - Part 1 · Student Assessment Pool: True/False (4)
    - Part 1 · Student Assessment Pool: Descriptive (3)
    - Part 1 · Student Assessment Pool: Analytical (2)
    - Part 2 · Discussion Questions (3)
    - Part 2 · Live Exercise (6)
    - Maths section

Chapter 1 · Week 5: Handling Big Data Files: YARN, File Formats & Optimization

The chapter in plain words

Week 5 asks two questions that come up once data is stored in HDFS and processed with MapReduce. First, **scheduling**: how does one big cluster (for example 1,000 machines) get shared fairly by 50 jobs from different teams at the same time? Second, **format**: how should data be laid out on disk so a query that needs 3 of 100 columns reads only about 3% of the data? (P1 slide 6)

YARN (Yet Another Resource Negotiator) is the answer to the scheduling question. In Hadoop 1.x one process, the **JobTracker**, did everything: it handed out cluster resources and also ran every MapReduce job. That made it a bottleneck (about 4,000 nodes maximum) and it could only run MapReduce. Hadoop 2.0 (2013) split the job in two: a global **ResourceManager** hands out resources, and each job gets its own **ApplicationMaster** that manages just that job. (P1 slides 7, 9)

Think of YARN as a hotel. The **ResourceManager** is the front desk that decides who gets which rooms. The **NodeManager** is the floor manager on each floor who opens rooms and checks they are used properly. The **ApplicationMaster** is a tour group leader who asks the front desk for rooms and then tells group members which room to go to. A **container** is one room: a fixed amount of CPU cores and RAM on one machine. (P1 slides 10-11)

YARN can share the cluster with three **schedulers**: **FIFO** (first in, first out; one long job can block everyone), **Capacity** (the cluster is split into queues with guaranteed percentages, e.g. Engineering 60%, Finance 40%, and idle capacity can be borrowed), and **Fair** (all running jobs get equal shares so short jobs finish fast). **DRF** (Dominant Resource Fairness) makes fairness work when jobs need different mixes of CPU and RAM. (P1 slides 13-14)

For the format question, the key idea is **row storage vs columnar storage**. A row store (CSV, JSON, Avro) writes one whole record after another, so reading one column still means reading every column. A columnar store (Parquet, ORC) writes all values of one column together, so a query can read only the columns it needs. On a 100-column, 10 TB table, a 3-column query reads 10 TB from CSV but about 300 GB from Parquet: a 33× speedup from format alone. (P1 slides 6, 16)

Parquet splits a file into **row groups** (about 128 MB each), each row group into **column chunks**, and each chunk into **pages**. The file footer stores the min and max of each column chunk. A query like `WHERE score > 90` reads the footer first and skips any row group whose max score is below 90. This skipping is called **predicate pushdown**. **Avro** is a row format whose strength is **schema evolution**: new fields can be added without breaking old readers. **ORC** is columnar and tuned for Hive with the highest compression. (P1 slides 17-18, 22-23)

Compression codecs trade speed for size: Snappy is fast with medium compression, Gzip is slow with high compression, Bzip2 is slowest but smallest, LZO is fast, Zstd is fast with high compression. The property that matters most is **splittability**: can the file be cut into pieces that different Mappers read in parallel? A 10 GB gzip CSV is **not** splittable, so exactly 1 Mapper reads all of it. Parquet + Snappy is splittable at row-group boundaries. Rule of thumb: Parquet + Snappy for analytics, Avro + Snappy for streaming ingest, never raw gzip on big CSV. (P1 slides 20-21)

Part 2 covers the YARN research paper (Vavilapalli et al., ACM SoCC 2013) and LinkedIn's YARN cluster (2013-2016). LinkedIn merged three separate clusters (MapReduce, Spark, Samza) into one 5,000-node YARN cluster using the Capacity Scheduler with 6 queues. Average utilisation went from 45% to 78%, so the same hardware served 73% more work, and the PYMK (People You May Know) job dropped from 18 hours on MapReduce v1 to 4 hours. The big lesson: YARN is the operating system of the cluster and frameworks are its processes. (P2 slides 17, 19, 21)

Part 1 · Student Assessment Pool: MCQ

Q1. Which component in YARN is responsible for global resource allocation across the entire cluster?

(a) NodeManager
(b) ApplicationMaster
(c) ResourceManager
(d) NameNode

ANSWER

Answer: (c) ResourceManager

The ResourceManager (RM) is the *global* cluster resource scheduler. There is one RM per cluster, running on a dedicated master node. Its Scheduler part decides which application gets which containers (P1 slide 10). The instructor note says it plainly: one RM per cluster, one AM per running application (P1 slide 28).

- (a) NodeManager: a *per-node* agent. It only launches and watches containers on its own machine.
- (b) ApplicationMaster: a *per-job* manager. It asks the RM for resources for one job only. This is the most common wrong choice.
- (d) NameNode: an HDFS component that stores file metadata. It does not allocate CPU or RAM.

Q2. Which Hadoop file format stores data in columnar format and is best suited for analytical, read-heavy workloads?

(a) Avro
(b) Text/CSV
(c) Parquet
(d) JSON

ANSWER

Answer: (c) Parquet

Parquet is columnar: all values of one column are stored together, so a query reads only the columns it needs. Its row groups carry min/max statistics that allow predicate pushdown (skipping whole row groups). It is the standard for Spark, Presto and ML pipelines (P1 slides 17-18). ORC is also columnar, but it is not an option here; the instructor note says ORC would be partly correct for Hive, while Parquet is the broader analytics standard (P1 slide 28).

- (a) Avro: row-oriented. Its strength is schema evolution for streaming ingest, not analytics.
- (b) Text/CSV: row-oriented, no built-in schema. A query must read every column.
- (d) JSON: row-oriented and not splittable in raw form. Used for APIs and logs.

Q3. What does “splittable” mean for a file format in Hadoop?

(a) The file can be deleted in parts by HDFS
(b) Multiple Mappers can process different parts of the file simultaneously
(c) The file format supports splitting schema across multiple NameNodes
(d) The file can be distributed across multiple data centres

ANSWER

Answer: (b) Multiple Mappers can process different parts of the file simultaneously

A file is splittable if reading can start from any point in the file without decompressing from the beginning (P1 slide 21). Then each piece (for example each Parquet row group) can go to its own Mapper and they run in parallel. The instructor note adds that splittability enables data-local task scheduling: each Mapper runs on the node that holds its piece (P1 slide 29). Example: a 10 GB gzip CSV is not splittable, so exactly 1 Mapper reads it all (P1 slide 20).

- (a) Splittability is about parallel reading, not deleting.
- (c) Schemas are not split across NameNodes; this is made up.
- (d) Spreading data across data centres is replication/geography, not splittability.

Q4. YARN was introduced in which major Hadoop version, and what did it replace?

(a) Hadoop 1.0; replaced the NameNode
(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster
(c) Hadoop 3.0; replaced DataNodes with Containers
(d) Hadoop 2.0; replaced the Secondary NameNode with a Standby NameNode

ANSWER

Answer: (b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster

In Hadoop 1.x the JobTracker did both resource management and job scheduling, only ran MapReduce, and topped out at about 4,000 nodes. Hadoop 2.0 (2013) separated the two jobs: a global ResourceManager plus one ApplicationMaster per job (P1 slides 7, 9).

- (a) Wrong version, and YARN did not touch the NameNode (that is HDFS).
- (c) Wrong version; containers are resource slices, they do not replace DataNodes (which store HDFS blocks).
- (d) Right version but wrong change. The Standby NameNode (HDFS high availability) is also a Hadoop 2.0 feature, but it has nothing to do with YARN. The instructor note calls this the common distractor (P1 slide 29).

Part 1 · Student Assessment Pool: True/False

1. Parquet is a row-oriented file format optimised for write-heavy workloads.

ANSWER

False.

Parquet is a *columnar* format built for *read-heavy* analytics: it stores each column together so a query reads only the columns it needs, and uses min/max stats for predicate pushdown (P1 slides 16-18).

Corrected: Parquet is a column-oriented file format optimised for read-heavy analytical workloads. (The row-oriented, append-friendly format is Avro.)

2. YARN allows multiple computation frameworks such as Spark and Tez to share resources on the same Hadoop cluster.

ANSWER

True.

Because each application brings its own ApplicationMaster that holds the framework-specific logic, YARN is framework-agnostic: MapReduce, Spark, Tez and Samza AMs all run on the same cluster (P1 slide 9, P2 slide 9).

3. Gzip-compressed files are natively splittable in Hadoop, allowing multiple Mappers to process them in parallel.

ANSWER

False.

Gzip is not splittable: a gzip stream can only be decompressed from its start, so a 10 GB gzip CSV is processed by exactly 1 Mapper with no parallelism (P1 slide 20).

Corrected: Gzip-compressed files are not splittable in Hadoop, so one Mapper must process the whole file; use bzip2 (natively splittable) or Parquet + Snappy (splittable at row groups) instead.

4. ORC format is specifically optimised for Hive workloads and achieves among the highest compression ratios of any Hadoop format.

ANSWER

True.

The slides list ORC as columnar with Zlib/Snappy compression, "best for Hive, highest compression", with Hive ACID support and vectorized reads (P1 slides 17, 25).

Part 1 · Student Assessment Pool: Descriptive

D1. Explain YARN's three-component architecture — ResourceManager, NodeManager, and ApplicationMaster. Draw a diagram showing how they interact when a Spark job is submitted to the cluster.

ANSWER

Definition. YARN (Yet Another Resource Negotiator) is Hadoop 2's resource manager. It separates *cluster resource management* from *per-job execution logic* using three components (P1 slides 10-11).

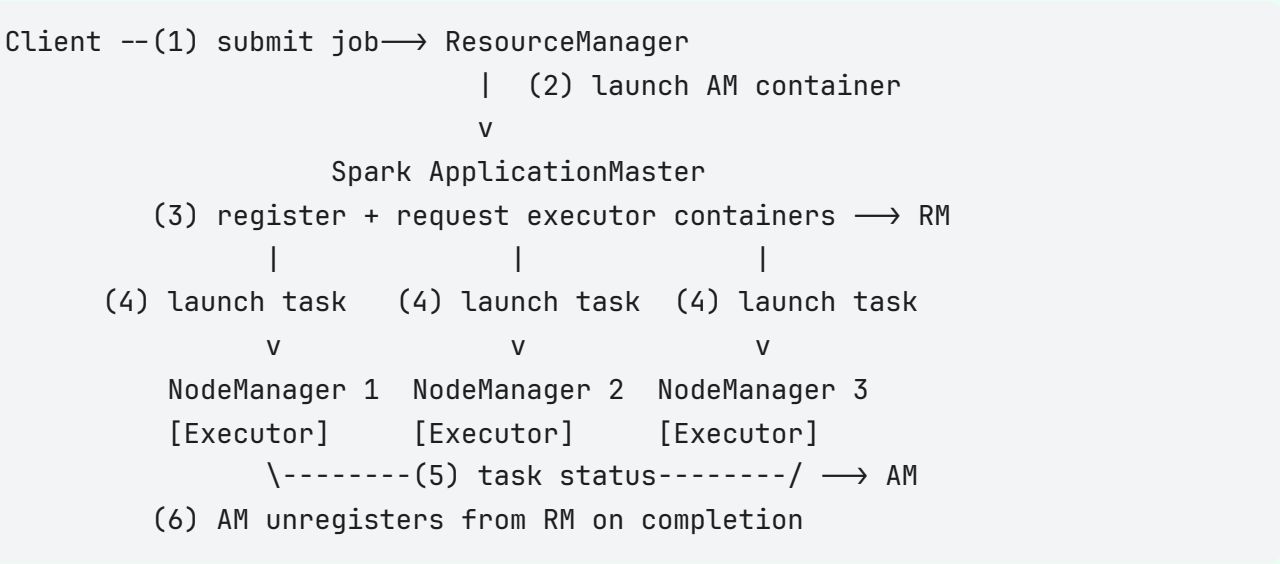
1. **ResourceManager (RM)**: one per cluster, on a master node. Its *Scheduler* hands out containers (pluggable: FIFO, Capacity, Fair). Its *ApplicationsManager* accepts job submissions and starts the first container for each ApplicationMaster.
2. **NodeManager (NM)**: one per worker node. It launches and monitors containers, reports node health and container status to the RM by heartbeat, and enforces CPU/RAM limits per

container.

3. **ApplicationMaster (AM)**: one per running job. It requests containers from the RM, tells NMs to launch tasks in them, watches progress, re-requests containers for failed tasks, and releases containers when done.

A **container** is a slice of one node: a set number of CPU cores plus a set amount of RAM.

Diagram (Spark job submission, P1 slide 12):



Example (P1 slide 11): the Spark AM asks for 100 containers of 4 cores and 8 GB each. The cluster is busy, so the RM grants 80. The AM runs Spark executors in those 80 containers and assigns tasks to them.

D2. What is columnar storage? Why does it improve performance for analytics queries that select only a subset of columns? Use a concrete example with numbers to illustrate the I/O reduction.

ANSWER

Definition. Columnar storage writes all the values of one column next to each other on disk, instead of writing one full row after another. Parquet and ORC are columnar; CSV, JSON and Avro are row stores (P1 slides 16-17).

Why it is faster for analytics:

- **Less I/O.** A query reads only the columns it names and physically skips the rest. In a row store, `SELECT score FROM users` still reads id, name and age, because they sit in the same row on disk (P1 slide 16).
- **Skipping row groups.** Parquet keeps min/max stats per column chunk in its footer, so a filter like `WHERE score > 90` skips whole row groups without reading them (predicate pushdown, P1 slide 23).
- **Better compression.** Values in one column are of the same type and often similar, so they compress well.

Worked example (P1 slide 6): a 100-column table holds 10 TB. A query selects 3 columns.

- CSV (row store) reads all 10 TB.
- Parquet reads 3/100 of the data = 0.03 × 10 TB = 300 GB.
- I/O reduction = 1 − 3/100 = 97%. Speedup ≈ 100/3 ≈ 33× from format alone.

Analogy: a library that shelves each chapter type together lets you grab only "Chapter 3" of every book, instead of carrying every whole book to your desk.

D3. Compare Avro and Parquet. Explain what “schema evolution” means and why Avro handles it better. In which scenarios would you use each?

ANSWER

Schema evolution means the structure of the records (the schema: field names and types) changes over time, for example a new field is added, while old files and old programs keep working.

	Avro	**Parquet**
Orientation	Row	Columnar
Schema	Self-describing (JSON schema)	Self-describing
Splittable	Yes	Yes (at row groups)
Compression	Snappy/Deflate	Snappy/Gzip
Strength	Schema evolution, append-friendly writes	Fast analytic reads, predicate pushdown
Best for	Kafka → HDFS streaming ingest	Spark, Presto, ML pipelines

(P1 slides 17-18)

Why Avro handles schema evolution better: Avro stores the *writer schema* with the data and lets the reader supply its own *reader schema*. Avro then matches fields by name: a field the reader expects but the file lacks gets its **default value**; a field in the file that the reader does not know is **ignored**. This gives backward and forward compatibility. Example: when LinkedIn adds a new field to a Kafka event schema, old consumers reading the same HDFS files do not break (P1 slide 18).

When to use each (P1 slides 25-26):

- **Avro**: streaming ingest (Kafka → HDFS), data exchange / APIs, anywhere records are appended and the schema keeps changing. bKash ingests Kafka events as Avro for API versioning.
- **Parquet**: Spark analytics pipelines and ML feature tables, where queries read a few columns of many rows. bKash converts nightly to Parquet + Snappy for analytics.

Part 1 · Student Assessment Pool: Analytical

A1. Data Lake Performance Migration: A Dhaka-based telecom company stores 10 TB of daily call-detail records (CDR) on HDFS as gzip-compressed CSV files. Their data engineers report that a Hive query selecting 4 of 80 columns takes 6 hours. – Identify two specific reasons why this configuration is slow – Propose a migration strategy: new format, compression codec, and partitioning scheme – Estimate the expected I/O reduction (show your calculation) – What YARN scheduler would you use and why, given that multiple teams share the cluster?

ANSWER

Step 1: Two reasons it is slow

1. **Gzip is not splittable.** Each gzip file must be read from its start by exactly one Mapper. A big gzip CSV gets no parallelism even though HDFS stores it in many blocks (P1 slide 20). Most of the cluster sits idle.
2. **CSV is a row store.** To read 4 columns, Hive must read and parse all 80 columns of every row, so it reads the full 10 TB (P1 slides 6, 16). There are also no min/max stats, so no row group can be skipped.

Step 2: Migration strategy

- **Format: ORC or Parquet** (columnar). For a Hive warehouse the slides recommend ORC (best compression, Hive ACID, vectorized reads); Parquet is the choice if Spark also reads it (P1 slide 25).
- **Codec: Snappy** for hot data (fast, and the file stays splittable at row-group/stripe boundaries). Use Zlib for ORC in Hive, or Zstd for cold archive data (P1 slides 21, 25).
- **Partitioning:** partition by date, like bKash's `dt=YYYY-MM-DD/region=` (P1 slide 26), e.g. `dt=2026-10-13/region=Dhaka`. A query for one day or region then reads only that folder.
- Run a one-time conversion job (Hive `INSERT OVERWRITE ... STORED AS ORC` or a Spark job) and convert new data nightly.

Step 3: I/O reduction estimate

- Fraction of columns read = 4 / 80 = 0.05 = 5%.
- Data read per day = 10 TB × 0.05 = 0.5 TB = 500 GB.
- I/O reduction = 1 − 0.05 = 0.95 = **95%**, i.e. 80/4 = **20×** less data.
- If the query is I/O-bound, time drops roughly from 6 h = 360 min to 360 × 0.05 = **about 18 minutes**. This is a rough estimate; in practice it can be faster, because the files are now splittable (many Mappers in parallel), and predicate pushdown and date partitions skip even more data.

Step 4: Scheduler choice

Use the **Capacity Scheduler**. It splits the cluster into queues with guaranteed shares, e.g. a production ETL queue, an analyst ad-hoc queue and a data-science queue, and lets a queue borrow idle capacity from others (P1 slide 13). One team's huge job cannot starve the others, as in the a2i example where queue A keeps its 60% guarantee (P1 slide 14). FIFO is unsuitable because one long job blocks everyone. Fair Scheduler is a reasonable alternative if the main goal is quick turnaround for short ad-hoc queries.

A2. YARN Failure Trace: A NodeManager (NM2) hosting 5 running containers for a MapReduce job fails mid-execution. Trace the complete failure recovery sequence: which components detect the failure, in what order, what state changes occur in the ResourceManager, and how are the 5 containers' tasks restarted.

ANSWER

Background: every NodeManager sends a regular **heartbeat** (an "I am alive" message with node health and container status) to the ResourceManager (P1 slide 10).

1. **Failure happens.** NM2 crashes. Its 5 containers die with it, and NM2 stops sending heartbeats.
2. **RM detects it.** The ResourceManager notices the missing heartbeats. After the timeout (more than 10 minutes, P2 slide 11) it declares NM2 lost.
3. **RM state changes.** The RM marks NM2 as lost/unhealthy, removes its CPU and RAM from the pool of schedulable capacity, and marks all 5 containers on NM2 as **failed** (P1 slide 12).
4. **AM is notified.** The RM tells the job's MapReduce ApplicationMaster that its 5 containers failed (P2 slide 11). The AM may also have noticed earlier, because it stopped getting task status from those containers.
5. **AM re-requests resources.** The AM asks the RM for 5 replacement containers of the same size (P1 slide 11: "re-requests containers for failed tasks").
6. **RM allocates elsewhere.** The Scheduler grants containers on healthy nodes, e.g. NM1 and NM3, ideally on nodes holding HDFS replicas of the input blocks (data locality).
7. **Tasks restart.** The AM tells NM1/NM3 to launch the 5 tasks in the new containers. Map tasks re-run from the start of their input split (input is safe in HDFS, which keeps replicas). Completed map tasks whose output lived on NM2's local disk may also need to be re-run, because reducers can no longer fetch that output.
8. **Job continues.** New containers send status to the AM; the job finishes normally, only later.

Edge cases: if the AM itself had been on NM2, the RM restarts a new AM (up to `yarn.resourcemanager.am.max-attempts`, default 2); MapReduce's AM can recover job state from HDFS checkpoints (P1 slide 12, P2 slide 11). If the RM failed, a Standby RM takes over via ZooKeeper in under 60 seconds (YARN 2.4+, P2 slide 11).

Part 2 · Discussion Questions

L1. Resource consolidation increases utilisation: merging 3 siloed clusters into one raised average utilisation from 45% to 78% — the same capacity served 73% more workload

ANSWER

Note: P2 slide 21 lists these as lessons, not separate questions; each is treated here as a discussion prompt.

What it means. A **silo** is a cluster kept for one workload only. Before YARN, LinkedIn ran three: Cluster A for MapReduce/Hive ETL, Cluster B for Spark, Cluster C for Samza streams (P2 slide 16). Each sat idle at different times, but idle machines in one silo could not help a busy one. One shared YARN cluster lets any workload use any idle machine.

The numbers. Utilisation went from 45% to 78% (P2 slide 19). 78 / 45 = 1.73, so the same hardware did 73% more work. At 45% the silos wasted 55% of their capacity; at 78% only 22% is unused.

For consolidation:

- Peaks in one workload fill gaps in another, so less hardware sits idle.
- No cross-cluster data copying, which had added latency; all frameworks read the same HDFS (P2 slides 16, 18).
- One configuration to manage instead of three, which had tripled ops cost (P2 slide 16).

Against / risks:

- Workloads now compete; without queues and guarantees, one team can slow another (handled by the Capacity Scheduler, lesson L2).
- Borrowing capacity means taking it back later, which needs pre-emption that can kill jobs (lesson L3).
- One shared cluster is a bigger single point of failure, so ResourceManager HA (ZooKeeper, P2 slide 11) becomes essential.

Conclusion. The utilisation gain is large and measured, so consolidation wins, provided scheduling and isolation are set up carefully.

L2. Capacity Scheduler enables SLAs: production queues guaranteed minimum resources; analyst adhoc queries cannot starve revenue-generating pipelines

ANSWER

What it means. An **SLA** (service level agreement) is a promise such as "the nightly ETL finishes by 6 a.m." **Starving** a job means it never gets enough resources to run. The Capacity Scheduler splits the cluster into queues with guaranteed shares. LinkedIn used 6 queues: production 40%, spark 25%, samza 15%, adhoc 10%, tez 7%, reserve 3% (P2 slide 17).

For:

- Production (Hive ETL, Pig) always has at least 40%, so a flood of analyst queries cannot delay revenue pipelines such as ad targeting.
- Unused capacity is not wasted: other queues borrow it elastically (P1 slide 13).
- Same idea in Bangladesh: a2i keeps DGHS batch jobs at 60% so streaming cannot take over (P1 slide 14).

Against / limits:

- Fixed percentages need regular tuning; if needs change, a queue may be too small while another is oversized.
- Analysts in the 10% adhoc queue may wait long at busy times. The Fair Scheduler gives short jobs faster turnaround (P1 slide 13).
- Getting borrowed capacity back quickly needs pre-emption (L3).

Conclusion. For a shared cluster with revenue-critical jobs, guaranteed queues are the safer choice; Fair suits clusters with many short, equal-priority jobs.

L3. Pre-emption needs tuning: aggressive pre-emption killed long-running Spark jobs; LinkedIn set 30-second grace periods and added Spark checkpoint-on-preempt hooks

ANSWER

What it means. Pre-emption is the scheduler taking containers back from a queue that borrowed them, so the owning queue gets its guarantee. A **grace period** is a warning time before the container is killed. A **checkpoint** saves a job's progress so it can resume instead of restarting.

For pre-emption:

- Without it, borrowed capacity might never return, and guarantees (L2) would be empty promises.
- It makes borrowing safe, which is what drives utilisation up (L1).

Against / costs:

- Killing a container wastes all the work done in it. Long Spark jobs, such as PYMK's multi-hour graph run (4 h on YARN, P2 slide 19), lost the most.
- Too-aggressive settings mean jobs keep restarting and the cluster does work twice.

LinkedIn's fix (P2 slides 17, 21): a 30-second warning before reclaiming capacity, plus Spark hooks that checkpoint state when pre-empted, so the job resumes rather than starts over.

Conclusion. Pre-emption is needed, but its timing must match the workload: fast for short tasks, gentle (grace + checkpoints) for long-running ones.

L4. YARN is the OS; frameworks are processes: the paradigm shift — the cluster is the computer, YARN is its kernel

ANSWER

What it means. On a laptop, the operating system **kernel** gives CPU and memory to programs (processes) and isolates them from each other. YARN plays the same role for a whole cluster: the ResourceManager hands out containers (CPU + RAM) and NodeManagers enforce limits, while MapReduce, Spark, Tez and Samza are just applications, each with its own ApplicationMaster (P2 slides 7, 9).

For the analogy:

- YARN is framework-agnostic: any framework that writes an AM can run (P2 slide 9).
- Isolation like an OS: cgroups for CPU and memory limits; a container over its memory limit is killed to protect others (P2 slide 9).
- The container model became the ancestor of Kubernetes Pods, and the three-tier design (global scheduler / node agent / per-app master) is the same in Kubernetes (P2 slides 13, 27).

Against / limits:

- YARN only manages CPU and memory; it is not a full OS (no file system of its own, it relies on HDFS).
- Kubernetes has largely replaced YARN for new deployments (P2 slide 13), so YARN is one kernel among several, not the only one.

Conclusion. The analogy holds for resource management and isolation, and explains why separating the scheduler from per-app logic made Hadoop a general cluster platform.

L5. Samza on YARN: stream processing is just another set of long-running containers; YARN's fault tolerance restarts failed Samza tasks

ANSWER

What it means. Samza is LinkedIn's stream processing framework, built natively for YARN (P2 slide 13). A stream job never "finishes": it keeps reading Kafka events. On YARN it is simply a set of containers that stay alive indefinitely, in the samza queue (15%, P2 slide 17).

For:

- No separate streaming cluster (the old Cluster C, P2 slide 16); streams share hardware with batch jobs.
- Free fault tolerance: if a node fails, the RM marks its containers failed and the AM gets replacements elsewhere (P2 slide 11).
- It worked at scale: Samza Kafka lag stayed under 500 ms at p99 (P2 slide 19).

Against / risks:

- Long-running containers hold capacity permanently, so the queue must be sized for peak stream load.
- Pre-emption or a node failure interrupts the stream; YARN's NodeManager failure detection (heartbeat timeout over 10 minutes, P2 slide 11) is slow for real-time data unless the framework recovers faster itself.
- Restarted tasks must resume from the right Kafka position to avoid lost or duplicate events; that is the framework's job, not YARN's.

Conclusion. Treating streams as long-lived containers lets one cluster serve batch and streaming, but streaming needs its own guaranteed queue and fast recovery logic.

Part 2 · Live Exercise

LE. Task (20 minutes, pairs) Model LinkedIn's 5,000-node YARN cluster in Python. Given: • 5,000 nodes; each: 64 GB RAM, 16 vCPUs • Capacity Scheduler: production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10% • Container sizes: Hive mapper: {4 GB, 2 vCPU}; Spark executor: {8 GB, 4 vCPU}; Samza task: {2 GB, 1 vCPU} Questions: 1 Max concurrent containers per queue? 2 Which resource is the bottleneck (CPU or RAM)? 3 If production needs 10,000 Hive mappers, how many nodes are required?

```
NODES    = 5_000
RAM_GB   = 64
VCPUS    = 16

queues = {
  "production": 0.40,
  "spark":      0.25,
  "samza":      0.15,
  "adhoc":      0.10,
  "reserve":    0.10,
}
containers = {
  "production": {"ram": 4, "cpu": 2},
  "spark":      {"ram": 8, "cpu": 4},
  "samza":      {"ram": 2, "cpu": 1},
  "adhoc":      {"ram": 4, "cpu": 2},
}

for q, frac in queues.items():
    nodes_q = NODES * frac
    ram_q   = nodes_q * RAM_GB
    cpu_q   = nodes_q * VCPUS
    if q in containers:
        c = containers[q]
        by_ram = ram_q // c["ram"]
        by_cpu = cpu_q // c["cpu"]
        max_c = int(min(by_ram, by_cpu))
        bottleneck = ("RAM" if by_ram < by_cpu
                            else "CPU")
        print(f"{q}: {max_c} containers "
              f"(bottleneck: {bottleneck})")
```

ANSWER

What to do, step by step

1. For each queue, find its share of nodes: `nodes_q = NODES × frac`.

2. Turn that into total RAM and vCPUs for the queue.
3. Divide by the container size twice: once by RAM, once by CPU. The smaller number is the real limit, because a container needs both.
4. Whichever resource gives the smaller number is the bottleneck.

Complete runnable code (the slide skeleton from P2 slides 24-25, joined, plus a Q3 block):

```
NODES    = 5_000
RAM_GB   = 64
VCPUS    = 16

queues = {
  "production": 0.40,
  "spark":      0.25,
  "samza":      0.15,
  "adhoc":      0.10,
  "reserve":    0.10,
}
containers = {
  "production": {"ram": 4, "cpu": 2},
  "spark":      {"ram": 8, "cpu": 4},
  "samza":      {"ram": 2, "cpu": 1},
  "adhoc":      {"ram": 4, "cpu": 2},
}

for q, frac in queues.items():
    nodes_q = NODES * frac
    ram_q    = nodes_q * RAM_GB
    cpu_q    = nodes_q * VCPUS
    if q in containers:
        c = containers[q]
        by_ram = ram_q // c["ram"]
        by_cpu = cpu_q // c["cpu"]
        max_c = int(min(by_ram, by_cpu))
        bottleneck = ("RAM" if by_ram < by_cpu
                            else "CPU")
        print(f"{q}: {max_c} containers "
              f"(bottleneck: {bottleneck})")

# Q3: nodes needed for 10,000 Hive mappers
import math
c = containers["production"]
per_node = min(RAM_GB // c["ram"], VCPUS // c["cpu"])
need = 10_000
nodes_needed = math.ceil(need / per_node)
```

```
print(f"Hive mappers per node: {per_node}")
print(f"Nodes needed for {need} mappers: {nodes_needed}")
print(f"Production queue has {int(NODES * queues['production'])} nodes")
print(f"Cluster size if 40% must equal {nodes_needed} nodes: "
      f"{nodes_needed / queues['production']:.0f}")
```

Expected output:

```
production: 16000 containers (bottleneck: CPU)
spark: 5000 containers (bottleneck: CPU)
samza: 12000 containers (bottleneck: CPU)
adhoc: 4000 containers (bottleneck: CPU)
Hive mappers per node: 8
Nodes needed for 10000 mappers: 1250
Production queue has 2000 nodes
Cluster size if 40% must equal 1250 nodes: 3125
```

"reserve" prints nothing, because it has no entry in `containers` (the `if q in containers` check skips it).

Working table

Queue	Nodes	RAM (GB)	vCPU	Container	By RAM	By CPU	Max
production	5,000 × 0.40 = 2,000	128,000	32,000	4 GB, 2 vCPU	32,000	16,000	**16,000**
spark	5,000 × 0.25 = 1,250	80,000	20,000	8 GB, 4 vCPU	10,000	5,000	**5,000**
samza	5,000 × 0.15 = 750	48,000	12,000	2 GB, 1 vCPU	24,000	12,000	**12,000**
adhoc	5,000 × 0.10 = 500	32,000	8,000	4 GB, 2 vCPU	8,000	4,000	**4,000**
reserve	5,000 × 0.10 = 500	32,000	8,000	none	kept free for bursts		

Q1. Max concurrent containers per queue? production 16,000; spark 5,000; samza 12,000; adhoc 4,000; reserve has no container type defined (it is overflow capacity).

Q2. Which resource is the bottleneck? CPU, for every queue. Each node has 64 GB / 16 vCPU = 4 GB of RAM per vCPU. Every container type asks for only 2 GB per vCPU (4/2, 8/4, 2/1). So the vCPUs run out when only half the RAM is used. Example: one node fits 64/4 = 16 Hive mappers by

RAM but only 16/2 = 8 by CPU; 8 mappers use 32 of 64 GB, leaving 50% of the RAM idle. Fix: give containers more RAM per vCPU (e.g. 4 GB per vCPU) or buy nodes with more cores.

Q3. Nodes for 10,000 Hive mappers? Per node = min(64/4, 16/2) = min(16, 8) = 8 mappers. Nodes = 10,000 / 8 = **1,250 nodes**. The production queue already has 2,000 nodes, so the job fits using 1,250/2,000 = 62.5% of the queue. If instead you read the question as "how big must the whole cluster be so that the 40% production share alone gives 1,250 nodes", the answer is 1,250 / 0.40 = 3,125 nodes.

Note: the exercise uses reserve 10% to make the shares add to 100%. The real LinkedIn layout on P2 slide 17 had six queues: production 40, spark 25, samza 15, adhoc 10, tez 7, reserve 3.

Maths for Week 5

Week 5's maths is small but shows up everywhere: what fraction of data a columnar query reads, how much predicate pushdown saves, how many containers fit in a YARN queue, how many nodes a job needs, how many Mappers a file gets, and simple speedup/utilisation ratios from the LinkedIn case. DRF is named on the slides without a formula, so its item uses the standard definition from the Ghodsi et al. (2011) paper the slides cite.

Columnar I/O reduction

```
Data read = D × k / N
I/O reduction = (1 - k / N) × 100%
Speedup ≈ N / k
```

WHAT EACH SYMBOL MEANS

- **D**: total table size
- **N**: total number of columns
- **k**: columns the query selects
- Assumes columns are about the same size and the row store reads everything.

WORKED EXAMPLE

Slide numbers (P1 slides 6, 16): D = 10 TB, N = 100, k = 3.

- Data read = 10 TB × 3/100 = 0.3 TB = 300 GB
- Reduction = (1 − 0.03) × 100% = 97%
- Speedup = 100/3 ≈ 33.3, so about 33×

PRACTICE

Telecom CDR (A1): D = 10 TB, N = 80, k = 4.

- Read = 10 × 4/80 = 0.5 TB = 500 GB
- Reduction = 1 − 0.05 = 95%
- Speedup = 80/4 = 20×; 6 h becomes about 360 × 0.05 = 18 min if I/O-bound

Variant: D = 20 TB, N = 200, k = 5: read = 20 × 5/200 = 0.5 TB; reduction = 97.5%; speedup = 40×.

Predicate pushdown saving

```
Reduction = (1 - R / D) × 100%
Speedup = D / R
```

WHAT EACH SYMBOL MEANS

- **D**: data on disk
- **R**: data actually read after skipping row groups (using footer min/max) and unused columns

WORKED EXAMPLE

P1 slide 23: 10 TB of Parquet, query physically reads 50 GB. D = 10,000 GB, R = 50 GB.

- R/D = 50/10,000 = 0.005
- Reduction = 99.5%
- Speedup = 10,000/50 = 200×

Row-group skip rule: Row Group 1 has score in [60, 85]; for `WHERE score > 90`, max 85 < 90, so skip. Row Group 2 has [85, 99]; max 99 > 90, so read it.

PRACTICE

A 2 TB table, query reads 40 GB. R/D = 40/2,000 = 0.02, reduction = 98%, speedup = 50×. Which row groups does `WHERE score < 70` read if RG1 = [60, 85], RG2 = [85, 99]? RG1 min 60 < 70, read it; RG2 min 85 ≥ 70, skip it.

Number of row groups / Mappers (splittability)

```
Splits = ⌈ F / B ⌉ if splittable; Splits = 1 if not (e.g. gzip)
```

WHAT EACH SYMBOL MEANS

- **F**: file size
- **B**: split size (HDFS block or Parquet row group, about 128 MB)
- ⌈ ⌉: round up

WORKED EXAMPLE

P1 slide 20: 10 GB gzip CSV, so 1 Mapper. The same 10 GB as Parquet with 128 MB row groups: 10 × 1024 / 128 = 80 splits, so up to 80 Mappers in parallel.

PRACTICE

10 GB with 256 MB row groups: 10 × 1024 / 256 = 40 Mappers. 10 TB (decimal) of Parquet with 128 MB row groups: $10 \times 10^{12} / 128 \times 10^{6}$ = 78,125 row groups (81,920 if you use binary TiB/MiB).

Capacity Scheduler: containers per queue and bottleneck

```
Nodes_q = N × f
by_RAM = ⌊ Nodes_q × RAM / c_ram ⌋
by_CPU = ⌊ Nodes_q × CPU / c_cpu ⌋
Max containers = min(by_RAM, by_CPU)
```

WHAT EACH SYMBOL MEANS

- **N**: nodes in cluster; **f**: queue share
- **RAM**, **CPU**: per-node GB and vCPUs
- **c_ram**, **c_cpu**: one container's RAM and vCPU
- The resource giving the smaller count is the bottleneck.

WORKED EXAMPLE

P2 slide 23, spark queue: N = 5,000, f = 0.25, 64 GB, 16 vCPU, executor {8 GB, 4 vCPU}.

- Nodes = 1,250; RAM = 80,000 GB; vCPU = 20,000
- by_RAM = 80,000/8 = 10,000; by_CPU = 20,000/4 = 5,000
- Max = 5,000 executors; bottleneck CPU

Also the a2i split (P1 slide 14) on a 1,000-node cluster: A 60% = 600 nodes, B 30% = 300, C 10% = 100.

PRACTICE

Cluster of 2,000 nodes, each 128 GB and 32 vCPU; spark queue 25%; executor {8 GB, 4 vCPU}. Nodes = 500; RAM = 64,000 GB; vCPU = 16,000. by_RAM = 8,000; by_CPU = 4,000. Max = 4,000; bottleneck CPU.

Nodes needed for a job

```
Per node = min(⌊RAM / c_ram⌋, ⌊CPU / c_cpu⌋)
Nodes = ⌈ T / per node ⌉
```

WHAT EACH SYMBOL MEANS

- **T**: containers (tasks) the job needs at once
- Other symbols as above

WORKED EXAMPLE

P2 slide 23 Q3: 10,000 Hive mappers {4 GB, 2 vCPU} on 64 GB / 16 vCPU nodes. Per node = min(16, 8) = 8. Nodes = 10,000 / 8 = 1,250. Production queue has 2,000 nodes, so it fits (62.5% of it). Whole cluster needed if only 40% is production: 1,250 / 0.4 = 3,125.

PRACTICE

3,000 Spark executors {8 GB, 4 vCPU}: per node = min(64/8, 16/4) = min(8, 4) = 4, nodes = 3,000/4 = 750. Samza tasks {2 GB, 1 vCPU}: per node = min(32, 16) = 16.

Speedup, utilisation and scaling ratios

```
Speedup = T_old / T_new
Extra work served = U_new / U_old - 1
Increase = new / old - 1
```

WHAT EACH SYMBOL MEANS

- **T**: run time
- **U**: average cluster utilisation

WORKED EXAMPLE

- LinkedIn utilisation 45% → 78% (P2 slide 21): 78/45 = 1.733, so 73% more workload on the same hardware.
- PYMK 18 h on MR1 → 4 h on YARN (P2 slide 19): 18/4 = 4.5× faster, a 77.8% cut in time.
- YARN tested at 6,000 nodes vs Hadoop 1.x's 4,000 limit (P2 slide 10): 6,000/4,000 − 1 = 50% more.
- Spark AM asks for 100 containers, gets 80 (P1 slide 11): 80% granted = 320 cores and 640 GB.

PRACTICE

A cluster goes from 50% to 80% utilisation: 80/50 − 1 = 60% more work. A job goes from 9 h to 2 h: 4.5× speedup.

Dominant Resource Fairness (DRF)

```
Share of resource r for user i = used_i,r / total_r
Dominant share_i = max over r of that share
DRF: keep giving the next task to the user with the lowest
dominant share, until something runs out (ends with equal
dominant shares).
```

WHAT EACH SYMBOL MEANS

- **used_i,r**: amount of resource r (CPU or RAM) user i holds
- **total_r**: cluster total of resource r
- **Dominant resource**: the resource a user needs the largest fraction of

The slides name DRF (Ghodsi et al., 2011) for fairness across CPU and memory (P1 slide 14); the formula is the paper's standard definition.

WORKED EXAMPLE

Cluster: 9 CPU, 18 GB. User A tasks need <1 CPU, 4 GB>; user B tasks need <3 CPU, 1 GB>.

- A per task: CPU 1/9, RAM 4/18 = 2/9. Dominant = RAM.
- B per task: CPU 3/9 = 1/3, RAM 1/18. Dominant = CPU.
- Give A 3 tasks and B 2 tasks: A uses 3 CPU, 12 GB (dominant share 12/18 = 2/3); B uses 6 CPU, 2 GB (dominant 6/9 = 2/3).
- Totals: 9 CPU used (all), 14 GB used. Dominant shares are equal, so allocation is fair.

PRACTICE

Cluster: 18 CPU, 36 GB. A needs <1 CPU, 6 GB> (dominant RAM, 6/36 = 1/6 per task); B needs <3 CPU, 2 GB> (dominant CPU, 3/18 = 1/6 per task). Equal per-task dominant shares, so give equal task counts. 4 each: CPU = 4 + 12 = 16 ≤ 18, RAM = 24 + 8 = 32 ≤ 36. A 5th each would need 20 CPU, too many. Final dominant shares: A 24/36 = 2/3, B 12/18 = 2/3.

Chapter 2 · Week 6: Apache Spark: RDDs, DataFrames & In-Memory Computing

The chapter in plain words

The problem Spark solves. MapReduce (Week 4) writes every intermediate result to HDFS (the cluster's shared disk) before the next step can start. That is fine for a job that reads the data once. It is very slow for *iterative* algorithms, which pass over the same data again and again, such as K-Means clustering, logistic regression or PageRank. Example from the slides: 100 K-Means iterations over 50 GB cost 10 TB of disk I/O in MapReduce, but only 50 GB in Spark (P1 slide 6).

The big idea: keep the working data in RAM. Spark reads the data from HDFS once, keeps it in the cluster's memory (RAM), and runs every later iteration from memory. Reading RAM is far faster than reading disk. Analogy: MapReduce is a cook who puts every ingredient back in the store room after each step; Spark keeps them on the kitchen counter. Spark came from Matei Zaharia's PhD work at UC Berkeley AMPLab (2009 to 2013) (P1 slide 9).

Architecture: three parts. The **Driver** runs your main program, creates the SparkContext/SparkSession and builds the plan (a DAG, a graph of steps). The **Cluster Manager** (YARN, Kubernetes, Mesos or Spark Standalone) hands out machines and memory. **Executors** are worker processes (JVMs) on worker nodes that run tasks and hold cached data in memory (P1 slide 11).

RDD = Resilient Distributed Dataset. The core data object in Spark. It is a read-only, partitioned collection of records. *Partitioned* means it is split into pieces spread over many Executors so they work in parallel. *Immutable* (read-only) means you never change it; each operation makes a new RDD. *Lineage* means each RDD remembers the recipe of steps that built it from its parents (P1 slide 13).

Resilient = it can rebuild itself. HDFS protects data by keeping 3 copies. Spark instead keeps the recipe (lineage). If an Executor crashes and a partition is lost, Spark re-reads that piece of the input and re-applies the same steps. "The computation is the backup" (P1 slide 14).

Transformations vs. actions, and lazy evaluation. A *transformation* (map, filter, flatMap, reduceByKey, join...) only records a step in the plan and returns a new RDD; nothing runs yet. An *action* (count, collect, first, take, reduce, saveAsTextFile) starts the real work and returns a result to the Driver or writes to storage. Waiting until an action is called is *lazy evaluation*. It lets Spark see the whole plan, run filter and map together in one pass, and stop early (for example `first()` stops at the first match) (P1 slides 17 to 19).

Narrow vs. wide dependencies. In a *narrow* dependency each output partition needs at most one input partition (map, filter): no data moves between machines, and steps are chained in one *stage*. In a *wide* dependency an output partition needs data from many input partitions (groupByKey, reduceByKey, join): data must be *shuffled* (sent across the network), which starts a new stage and is the main cost in Spark. Prefer reduceByKey over groupByKey because it combines locally before the shuffle (P1 slides 21 to 22).

DataFrames, the ecosystem, and the papers. A DataFrame is a distributed table with named columns and a schema. The Catalyst optimizer rewrites DataFrame/SQL queries before running them, so DataFrames are 10 to 100 times faster than raw RDDs for structured data (P1 slide 26). On top of Spark Core sit Spark SQL, MLlib (machine learning), Structured Streaming and GraphX (P1 slide 29). Part 2 covers the two Zaharia papers (2010 HotCloud, 2012 NSDI best paper) and the Databricks GraySort record: 100 TB sorted in 23 minutes on 206 nodes, against Hadoop's 72 minutes on 2,100 nodes (P2 slides 13 to 14).

Part 1 · Student Assessment Pool: MCQ

Q1. What makes Spark significantly faster than MapReduce for iterative algorithms?

(a) Spark uses more CPU cores per node
(b) Spark writes intermediate results to faster SSDs
(c) Spark caches intermediate data as RDDs in memory
(d) Spark skips the shuffle phase entirely

ANSWER

Answer: (c) Spark caches intermediate data as RDDs in memory.

Iterative algorithms read the same data many times. MapReduce writes and re-reads every intermediate result on HDFS (disk) each iteration. Spark reads the data once, keeps it in RAM as a cached RDD, and later iterations read from memory. For 100 K-Means iterations on 50 GB this is 10 TB of disk I/O for MapReduce vs. 50 GB for Spark, about 200 times less (P1 slide 6).

- (a) Wrong: Spark runs on the same hardware as MapReduce. The gain comes from memory, not cores (instructor note, P1 slide 34).
- (b) Wrong: writing to disk, even a fast one, is exactly what Spark avoids between iterations.
- (d) Wrong: Spark still shuffles for wide operations like reduceByKey and groupByKey (P1 slide 21).

Q2. How does Spark achieve fault tolerance in RDDs without full data replication?

(a) Checkpointing to HDFS every 60 seconds
(b) Tracking lineage and recomputing lost partitions from parent RDDs
(c) Copying each RDD partition twice across two Executors
(d) Using a write-ahead log on the Driver node

ANSWER

Answer: (b) Tracking lineage and recomputing lost partitions from parent RDDs.

Each RDD records its lineage: the list of transformations that built it from its parents. If a partition is lost, Spark re-reads the matching input partition and re-applies those steps. No copies are kept, so "the computation is the backup" (P1 slides 14 to 15).

- (a) Wrong: checkpointing exists, but it is an optional extra for long lineage chains, not on a timer, and not the main mechanism (P1 slide 14).
- (c) Wrong: that is replication, the HDFS-style approach Spark avoids (instructor note, P1 slide 34).
- (d) Wrong: RDDs do not use a write-ahead log on the Driver; recovery comes from lineage.

Q3. Which of the following is **NOT** a component of the Spark ecosystem?

(a) MLlib
(b) GraphX
(c) Structured Streaming
(d) HBase

ANSWER

Answer: (d) HBase.

HBase is a separate NoSQL database (Week 4) that runs on HDFS. Spark can read from it as a data source, but it is not part of Spark itself (instructor note, P1 slide 35). The Spark ecosystem diagram shows Spark SQL, MLlib, Structured Streaming and GraphX on top of Spark Core; HBase appears only in the Storage row (P1 slide 29).

- (a) MLlib is Spark's distributed machine-learning library, so it is a component.
- (b) GraphX is Spark's graph library (PageRank, connected components), so it is a component.
- (c) Structured Streaming is Spark's stream-processing engine, so it is a component.

Q4. In Spark, a “transformation” such as map() or filter() is:

(a) Immediately executed when called and materialised to memory
(b) Lazily evaluated — only executed when a subsequent action is called
(c) Automatically persisted to HDFS after execution
(d) Only available in the Scala API, not in PySpark

ANSWER

Answer: (b) Lazily evaluated, only executed when a subsequent action is called.

A transformation only adds a step to the plan (the DAG) and returns a new RDD. Nothing runs until an action such as `count()` or `collect()` is called (P1 slides 17 to 19). Example: `rdd.filter(f)` alone does no work; `rdd.filter(f).count()` runs the filter.

- (a) Wrong: this is the opposite of lazy evaluation; the instructor calls it a critical misconception (P1 slide 35).
- (c) Wrong: results are not written to HDFS automatically; you write them only with an action like `saveAsTextFile`.
- (d) Wrong: PySpark has map and filter; the slides' examples are written in PySpark (P1 slide 24).

Part 1 · Student Assessment Pool: True/False

1. Spark can only run on top of Hadoop YARN and has no other deployment option.

ANSWER

False.

The Cluster Manager can be YARN, Kubernetes, Mesos or Spark Standalone (P1 slide 11).

Corrected: Spark can run on YARN, Kubernetes, Mesos or its own Standalone cluster manager.

2. RDDs are mutable — individual elements can be modified in place after creation.

ANSWER

False.

RDDs are immutable (read-only): they are never modified in place, and every transformation creates a new RDD (P1 slide 13).

Corrected: RDDs are immutable; to change data you apply a transformation that produces a new RDD.

3. Spark's DataFrame API provides schema-aware, named-column operations similar to SQL tables.

ANSWER

True.

A DataFrame is a distributed collection organised into named columns with a schema, like a table in a relational database, and it supports SQL syntax (P1 slide 26).

4. MapReduce is generally better than Spark for machine learning workloads that require many iterations over the same dataset.

ANSWER

False.

MapReduce writes to disk on every iteration, so it is slow for iterative work. Spark caches the data in memory and is 10 to 100 times faster for iterative algorithms (P1 slides 6, 31). The slides say to use Spark for any workload with more than 2 iterations over the same data (P1 slide 32).

Corrected: Spark is generally better than MapReduce for iterative machine learning workloads.

Part 1 · Student Assessment Pool: Descriptive

D1. Explain the concept of a **Resilient Distributed Dataset (RDD)**. What does *"resilient"* mean in this context? How does Spark recover a lost RDD partition without replication?

ANSWER

Definition. An RDD is a read-only, partitioned collection of records. It can only be created in two ways: by loading data from stable storage (HDFS, S3) or by transforming an existing RDD (map, filter) (P1 slide 13).

Five properties (P1 slide 13):

- **Immutable:** never changed in place; each transformation makes a new RDD.
- **Partitioned:** split into pieces spread across Executors, so the pieces are processed in parallel.
- **Lazy:** transformations run only when an action is called.
- **Lineage:** each RDD knows exactly how it was derived from its parents.
- **Resilient:** lost partitions are recomputed from lineage, not replicated.

What "resilient" means. The RDD can survive a machine failure. HDFS does this by storing 3 copies. An RDD does it by remembering the steps (its lineage) and re-running them. "The computation is the backup" (P1 slide 14).

Recovery steps (P1 slides 14 to 15):

1. Spark detects that a partition (say Part 2 of mappedRDD) is lost because its Executor crashed.
2. It traces the lineage: mappedRDD came from filteredRDD, which came from inputRDD.
3. It re-reads only Part 2 of the input from HDFS.
4. It re-applies filter, then map, to that one partition.
5. Part 2 is restored without restarting the whole job.

Trade-off: this is cheap for narrow dependencies but expensive for wide ones, so long lineage chains should be checkpointed to HDFS (P1 slide 14).

Diagram (text form):
inputRDD (HDFS) → filter → filteredRDD → map → mappedRDD [Part 2 lost] ⇒ re-read input Part 2 → filter → map → Part 2 restored.

Analogy: instead of photocopying a finished cake three times, you keep the recipe. If the cake is dropped, you bake that one cake again.

D2. Differentiate between **Spark transformations** and **actions**. Give two concrete examples of each. Why does Spark not execute transformations immediately?

ANSWER

Transformation: an operation that builds the computation DAG and returns a new RDD, but does not run straight away (lazy) (P1 slide 17).

Action: an operation that triggers execution of the DAG and returns a value to the Driver or writes to storage (eager) (P1 slide 18).

Point	Transformation	Action
When it runs	Lazily, only recorded	Immediately starts the job
Returns	A new RDD	A value (list, number) or output written to HDFS/disk
Examples	`map(f)` applies f to each element; `filter(f)` keeps elements where f is True (also flatMap, reduceByKey, join)	`count()` returns the number of elements; `collect()` returns all elements as a list (also first, take(n), saveAsTextFile)

Small example (P1 slide 24, word count): `lines.flatMap(...)`, `.map(lambda w: (w, 1))` and `.reduceByKey(...)` do no work. Only `counts.saveAsTextFile(...)` (an action) runs the whole chain.

Why not execute immediately? Rule: no transformation executes until an action is called (P1 slide 18). Waiting lets Spark:

- see the whole DAG first and optimise it (DAG scheduler, or Catalyst for DataFrames);

- pipeline steps, for example run filter and map in one pass over each partition, without writing intermediate results to disk;
- execute only the partitions that are needed, for example `first()` stops after the first match (P1 slide 19).

D3. Describe Spark's **lazy evaluation model**. What is the advantage of deferring computation until an action is called? Use the example `rdd.filter(x > 1000).first()` to illustrate.

ANSWER

Definition. Lazy evaluation means Spark does not run a transformation when you write it. It only records it in a computation graph (the DAG). The work starts only when an action is called (P1 slide 19).

What happens when the action is called (P1 slide 19):

1. Spark analyses the full DAG.
2. It applies the Catalyst optimizer (for DataFrames) or Spark's DAG scheduler.
3. It pipelines compatible transformations, e.g. filter and map in one pass over each partition, never writing intermediate results to disk.
4. It executes only the partitions needed.

Advantages:

- Less work: Spark can stop early and skip partitions it does not need.
- Fewer passes and no intermediate disk writes, thanks to pipelining.
- Whole-plan optimisation, since Spark sees all steps before running any.
- Lineage stays as a plan, which is what fault recovery replays.

Example: `rdd.filter(x > 1000).first()`

1. `filter(x > 1000)` is a transformation: Spark only notes "keep values above 1000". No data is read.
2. `first()` is an action: now Spark starts reading the first partition and tests values one by one.
3. As soon as it finds one value above 1000, it returns it and stops.
4. Without lazy evaluation, Spark would first filter the *entire* dataset, then take one element, wasting almost all the work (P1 slide 19).

Analogy: you write a shopping list (transformations) but only go to the shop when you need dinner (action). If you only need one item, you grab it and leave instead of buying the whole list.

Part 1 · Student Assessment Pool: Analytical

A1. Iterative I/O Cost Analysis: You need to train a K-Means clustering model over **50 GB** of bKash transaction data with **100 iterations**. Assume HDFS read speed = 500 MB/s and RAM cache read speed = 5,000 MB/s.

- Calculate the total HDFS I/O for MapReduce (all 100 iterations)
- Calculate the total I/O for Spark (50 GB fits in cluster RAM)
- Estimate the wall-clock time for each approach
- Under what conditions would Spark NOT outperform MapReduce?

ANSWER

Assumptions. 1 GB = 1,000 MB, so 50 GB = 50,000 MB. HDFS write speed is not given, so assume it equals the read speed, 500 MB/s. Speeds are for the whole cluster. CPU time is ignored; we count I/O only.

Step 1: MapReduce HDFS I/O. Each iteration reads the 50 GB and writes 50 GB of results back to HDFS (P1 slide 6).

- Per iteration: 50 GB read + 50 GB write = 100 GB
- Total: 100 iterations × 100 GB = 10,000 GB = **10 TB**

Step 2: Spark I/O. Iteration 1 reads 50 GB from HDFS and caches it in RAM; iterations 2 to 100 read from memory (P1 slide 6).

- HDFS I/O: **50 GB** (initial read only)
- RAM reads: 99 iterations × 50 GB = 4,950 GB from cache
- HDFS I/O ratio: 10,000 GB / 50 GB = **200× less disk I/O** (matches "≈ 200×" on P1 slide 6)

Step 3: wall-clock time.

MapReduce:

- Read 50,000 MB at 500 MB/s = 100 s; write 50,000 MB at 500 MB/s = 100 s
- Per iteration = 200 s
- Total = 100 × 200 s = 20,000 s ≈ **5.56 hours**

Spark:

- Iteration 1 from HDFS: 50,000 / 500 = 100 s
- Each later iteration from RAM: 50,000 / 5,000 = 10 s; 99 × 10 s = 990 s
- Total = 100 + 990 = 1,090 s ≈ **18.2 minutes**

Speedup: 20,000 / 1,090 ≈ **18.3×** faster in time. It is lower than the 200× I/O ratio because RAM, at 5,000 MB/s, is only 10× faster than HDFS in this question, and Spark still reads 50 GB per iteration (from memory). With realistic gains from skipping the HDFS writes and pipelining, the slides quote 10 to 100× for iterative work (P1 slide 31).

Step 4: when Spark would NOT outperform MapReduce.

- **Single-pass jobs:** one read, one write. There is no reuse, so caching gives nothing. For single-pass ETL over data larger than RAM, gains are only 2 to 5× (P2 slide 16).
- **Data does not fit in cluster RAM:** partitions spill to disk or are evicted (LRU) and recomputed, so Spark loses its edge (P2 slide 10).
- **Memory overhead too high:** Spark needs about 8 GB per Executor minimum; on very large clusters with little RAM per node this can be prohibitive (P1 slide 32).
- **Stable existing MapReduce code** that is already fast enough (P1 slide 32).
- Workloads that hurt the JVM memory model, such as many small files or high-cardinality groupBy (P2 slide 16).

A2. RDD Failure Recovery Trace: A Spark job computes `input` → `filteredRDD` → `mappedRDD` → `reducedRDD` over 10 partitions. Mid-execution, the Executor holding partitions 3 and 7 of `reducedRDD` crashes. Trace the complete recovery sequence: what does the Driver detect, which partitions of which RDDs need to be recomputed, and what is the worst-case recovery cost if `reducedRDD` involves a **wide** dependency (`groupByKey`)?

ANSWER

Step 1: the stages. filter and map are narrow, so input → filteredRDD → mappedRDD is pipelined in **Stage 1** (10 tasks, one per partition). groupByKey is wide, so it needs a shuffle and starts **Stage 2**, which builds reducedRDD (10 tasks) (P1 slides 20 to 21). Stage 1 tasks write shuffle output files on the local disk of the Executor that ran them.

Step 2: what the Driver detects. The Driver coordinates execution (P1 slide 11). It sees that the Executor is lost (it stops responding) and that the tasks for reducedRDD partitions 3 and 7 failed. It marks those two partitions as missing. Partitions 0 to 2, 4 to 6, 8 and 9 are unaffected.

Step 3: trace the lineage. reducedRDD came from mappedRDD by groupByKey; mappedRDD came from filteredRDD by map; filteredRDD came from input by filter (P1 slide 15).

Step 4: what must be recomputed.

- **If it were narrow all the way (for comparison):** rebuild only partitions 3 and 7 at each level: re-read input partitions 3 and 7, re-apply filter and map, then the last step. That is 2 of 10 partitions = **20%** of the work (P1 slide 21, "recompute only that partition from its single parent partition").
- **With groupByKey (wide):** reducedRDD partition 3 holds every value for the keys that hash to 3, and these values came from *all 10* mappedRDD partitions. The same holds for partition 7. So each lost partition depends on all 10 parent partitions (P1 slide 21).

Step 5: best case vs. worst case.

- **Best case:** the Stage 1 shuffle files are still on surviving nodes. The Driver re-runs only 2 Stage 2 tasks (partitions 3 and 7), which fetch their data again from all 10 map outputs. No filter or map work is repeated.
- **Worst case:** the crashed Executor also stored Stage 1 shuffle output (it likely ran map tasks too), or none of mappedRDD is cached. Then the fetch fails and the Driver must re-run Stage 1: re-read **all 10** input partitions from HDFS, re-apply filter and map to all 10 (filteredRDD and mappedRDD partitions 0 to 9), redo the full shuffle, then rebuild reducedRDD partitions 3 and 7. That is **10 + 10 + 10 = 30** partition computations in Stage 1 plus 2 in Stage 2, i.e. close to rerunning the whole job, against 2 × 4 = 8 partition computations in the narrow case.

Step 6: how to limit the cost. Use reduceByKey instead of groupByKey so less data is shuffled (P1 slide 22); cache mappedRDD; checkpoint long lineage chains to HDFS (P1 slide 14). This is why wide dependencies make lineage recovery "expensive" (P1 slide 21).

Part 2 · Discussion Questions

1. **In-memory is a multiplier, not a panacea**: Spark is 10–100× faster on iterative workloads; for single-pass ETL over data larger than RAM, gains are smaller (2–5×)

ANSWER

Note: P2 slide 16 is titled "Lessons Learned & Discussion Questions", but the rendered slide shows only the five "Key Lessons from Spark's Rise". No separate question list is printed on it (it seems to have been cut off). Each lesson is copied below exactly and answered as a discussion point.

Meaning. A multiplier makes an existing advantage bigger; a panacea fixes everything. Memory helps most when the same data is read many times.

For (where Spark wins big): iterative jobs reuse the working set. RDD paper: logistic regression 127 s/iteration on Hadoop vs. 0.96 s on Spark, 132×; K-means 150 s vs. 5 s, 30× (P2 slide 10). Databricks: logistic regression on 1 TB took 2 min on Spark vs. 10+ hours on Mahout (P2 slide 15).

Against (where gains are small): single-pass ETL (extract, transform, load) reads each byte once, so there is nothing to cache; if the data is larger than RAM, Spark spills to disk like MapReduce. Gains drop to 2 to 5×.

Conclusion: choose Spark by workload shape: more than 2 iterations over the same data favours Spark (P1 slide 32).

2. **The lineage model scales elegantly**: re-computing a lost partition from lineage is cheaper than maintaining 3× replicated in-memory state

ANSWER

For: replicating in-memory data 3 times would triple RAM use and send every write over the network. Lineage stores only the small list of steps, and failures are rare, so you pay only when something breaks. Only the lost partitions are rebuilt (P1 slide 14).

Against: recovery is cheap only for narrow dependencies. With wide dependencies (groupByKey, join) a lost partition may need all parent partitions recomputed, and long chains get slow to replay. The fix is to checkpoint long lineage chains to HDFS (P1 slides 14, 21).

Conclusion: lineage is the better default for iterative in-memory work, with checkpoints as a safety net (P2 slide 20).

3. **Shuffle is the bottleneck**: both at GraySort and in production; minimising wide dependencies (shuffle) is the primary Spark tuning skill

ANSWER

Why shuffle costs so much: a wide dependency moves data across the network, writes it to disk, and creates a stage boundary (a barrier where all tasks must finish) (P2 slide 9).

Evidence from GraySort: Spark won mainly by making the shuffle cheaper: sort-based shuffle that writes sorted runs once (saving about 120 TB of extra I/O at 100 TB), all 8 SSDs used as shuffle directories, off-heap sort buffers, and a single wide transformation (sortByKey) (P2 slide 15).

Tuning moves: prefer reduceByKey over groupByKey (local combine first, P1 slide 22); co-partition two RDDs on the same key so a join needs no shuffle (P2 slide 10); avoid unneeded repartition/distinct.

Other side: some shuffles cannot be avoided (a global sort or join needs them), so the goal is to minimise them, not remove them.

4. **Benchmark marketing is real**: GraySort made Spark famous; it also hid cases where Spark's JVM memory overhead hurt performance (many small files, high-cardinality `groupBy`)

ANSWER

Positive side: the record (100 TB in 23 min on 206 nodes vs. Hadoop's 72 min on 2,100 nodes) drew huge press. Spark went from 200 GitHub stars in 2013 to more than 36,000 by 2020, and Databricks raised a $150M Series B within 6 months (P2 slides 14 to 15).

Negative side: a benchmark tests one workload on tuned hardware (206 large EC2 nodes with 244 GB RAM and 8 SSDs each). It hides weak spots: many small files (overhead per file and task) and high-cardinality groupBy (a huge number of distinct keys, which fills JVM memory and causes garbage-collection pauses).

Lesson: test a system on your own workload before trusting a headline number.

5. **Open-source velocity**: the Apache Spark community grew to 1,000+ contributors in 5 years; Databricks' business model (cloud + support) aligned commercial incentives with open-source development

ANSWER

For: Spark was donated to Apache in 2013 and Databricks was founded the same year (P1 slide 9). Because Databricks earns from cloud service and support, not from licences, improving the free engine also helps its business. The result was fast growth: 1,000+ contributors in 5 years, and libraries such as Spark SQL, MLlib, Structured Streaming, GraphX and Delta Lake (P2 slide 11).

Risk (other side): when one company leads a project, its priorities can shape the roadmap, and some features may appear first in the paid product. Users should watch for vendor lock-in.

Conclusion: an open engine plus a paid service gave Spark both community trust and steady funding, which helped it become the default engine at Netflix, LinkedIn, Alibaba and Uber (P1 slide 9).

Part 2 · Live Exercise

Task. **Task (25 minutes, pairs)**

Run the following PySpark script locally (Spark local mode).

Goal: build and inspect an RDD lineage; observe lazy evaluation; compare cached vs. uncached timing.

```
from pyspark import SparkContext
import time
sc = SparkContext("local[4]", "RDDDemo")
sc.setLogLevel("WARN")
# Build lineage (lazy - no compute yet)
lines = sc.textFile("README.md")
words = lines.flatMap(lambda l: l.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a+b)
# Inspect lineage BEFORE action
print(counts.toDebugString().decode())
# First action - triggers full DAG
t0 = time.time()
n = counts.count()
print(f"count={n}  time={time.time()-t0:.2f}s")
# Cache and re-run
counts.cache()
counts.count()          # warm up cache
t1 = time.time()
n = counts.count()      # served from cache
print(f"cached count={n} "
      f"time={time.time()-t1:.4f}s")
sc.stop()
```

ANSWER

What the skeleton does, line by line (P2 slide 18):

1. `SparkContext("local[4]", ...)`: starts Spark in local mode with 4 worker threads on your laptop (no cluster needed).
2. `textFile`, `flatMap`, `map`, `reduceByKey`: four transformations. They only build the lineage; nothing is read yet (lazy).
3. `toDebugString()`: prints the lineage graph. In PySpark it returns bytes, hence `.decode()`.
4. First `count()`: the first action. It runs the full DAG: read file, split, map, shuffle, reduce.
5. `cache()`: marks `counts` to be kept in memory. It is lazy too, so the next `count()` fills the cache.
6. Last `count()`: served from the cached partitions, so it should be much faster.

Steps to run:

1. Install PySpark (`pip install pyspark`; needs Java 8/11/17).
2. Put any text file named `README.md` in the folder (the Spark download includes one).
3. Save the script as `rdd_demo.py` and run `python rdd_demo.py`.

Complete version that answers all three questions (adds a second uncached run, prints the lineage after `cache()`, and keeps the Spark UI open):

```
from pyspark import SparkContext
import time

sc = SparkContext("local[4]", "RDDDemo")
sc.setLogLevel("WARN")

# Build lineage (lazy - no compute yet)
lines  = sc.textFile("README.md")
words  = lines.flatMap(lambda l: l.split())
pairs  = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
# Q3 variant: comment the line above and use this instead
# counts = pairs.groupByKey().mapValues(lambda vs: sum(vs))

# Q1: lineage BEFORE cache()
print("--- lineage before cache ---")
print(counts.toDebugString().decode())

# Q2: two uncached runs
for i in (1, 2):
    t0 = time.time()
    n = counts.count()
    print(f"uncached run {i}: count={n}  time={time.time()-t0:.2f}s")

# Mark for caching (still lazy) and show lineage again
counts.cache()
print("--- lineage after cache() ---")
print(counts.toDebugString().decode())

counts.count()          # warm up cache (fills it)
print("--- lineage after cache is filled ---")
print(counts.toDebugString().decode())

t1 = time.time()
n = counts.count()      # served from cache
print(f"cached count={n}  time={time.time()-t1:.4f}s")
```

```
input("Open http://localhost:4040 now, then press Enter to stop...")
sc.stop()
```

Expected output shape (numbers depend on your file and machine; not run here):

```
--- lineage before cache ---
(2) PythonRDD[..] at RDD at PythonRDD.scala:..
 |  MapPartitionsRDD[..] at mapPartitions at PythonRDD.scala:..
 |  ShuffledRDD[..] at partitionBy at ...
 +-(2) PairwiseRDD[..] at reduceByKey at rdd_demo.py:..
    |  PythonRDD[..] at reduceByKey at rdd_demo.py:..
    |  README.md MapPartitionsRDD[..] at textFile at ...
    |  README.md HadoopRDD[..] at textFile at ...
uncached run 1: count=<N>  time=~1-3s
uncached run 2: count=<N>  time=smaller
cached count=<N>  time=~0.0x s
```

The `(2)` is the number of partitions: for a small local file `textFile` normally makes 2 partitions. The `+-` indent marks the shuffle (stage boundary) caused by reduceByKey. `count` = the number of distinct lower-cased words.

1. Call `rdd.toDebugString()` before and after `cache()` — what changes?

ANSWER

Expected answer:

- **Before cache():** the lineage lists the chain of RDDs (HadoopRDD from textFile, then the Python RDDs for flatMap/map, the shuffle for reduceByKey) with no storage information.
- **Right after cache():** the lineage itself (the list of parents) does *not* change, because cache() is lazy. The top RDD line now shows a storage level tag, e.g. `[Memory Serialized 1x Replicated]` (PySpark stores cached data serialised; the level is MEMORY_ONLY, P2 slide 10).
- **After an action has filled the cache:** the line also shows `CachedPartitions: 2; MemorySize: ...; DiskSize: 0.0 B`. Later jobs read these partitions from memory and skip the parent steps.

Why: cache() is a transformation-like marker; Spark keeps the lineage anyway so it can rebuild a cached partition if it is evicted or lost (P1 slides 14, 17).

2. Time `rdd.count()` twice without `cache()`. Then add `.cache()` and time again. Report the speedup.

ANSWER

How: run the timed `count()` twice before `cache()`, then warm the cache and time once more (code above). Speedup = uncached time / cached time.

What to expect:

- **Uncached run 1** is slowest: it starts Python workers, reads the file, runs both stages and the shuffle.
- **Uncached run 2** is usually already faster: Spark reuses the shuffle files from run 1 and shows Stage 1 as "skipped" in the UI, and the workers are warm. It still recomputes the reduce stage.
- **Cached run** is fastest: partitions come straight from memory.

Example of reporting (illustrative numbers, not measured): uncached 1.50 s, cached 0.05 s, speedup = 1.50 / 0.05 = 30×. On a tiny README the absolute times are small and job start-up overhead dominates, so the speedup can be modest. On large data the slides' figures apply: 127 s vs. 0.96 s per iteration = 132× (P2 slide 10).

Lesson: cache only RDDs reused in more than one action; caching single-use RDDs wastes RAM and evicts other data (LRU) (P2 slide 10).

3. Change `reduceByKey` to `groupByKey`. Use the Spark UI (localhost:4040) to compare shuffle read/write bytes between the two.

ANSWER

How: replace the reduceByKey line with `counts = pairs.groupByKey().mapValues(lambda vs: sum(vs))` (groupByKey alone gives lists, so sum them to get the same counts). Run both versions and, while the script waits at the `input()` prompt, open `http://localhost:4040` → **Stages** tab. Read **Shuffle Write** on the map stage and **Shuffle Read** on the reduce stage.

Expected result: the word counts are identical, but groupByKey shows **larger** shuffle read/write bytes.

Why: reduceByKey applies a local combine before shuffling, like a Combiner in MapReduce (P1 slide 22). Each partition first adds up its own `(word, 1)` pairs, so it sends at most one record per distinct word. groupByKey sends every single `(word, 1)` pair across the network.

Small worked example: 4 partitions each with 1,000 pairs and 50 distinct words. groupByKey shuffles 4 × 1,000 = 4,000 records; reduceByKey shuffles at most 4 × 50 = 200 records, which is 95% less.

On a small README the byte difference is a few KB, but the ratio shows the point. Lesson: shuffle is the bottleneck, so prefer reduceByKey (P2 slide 16).

Maths for Week 6

This chapter has several calculations: disk I/O for iterative jobs, I/O time, speedup ratios, GraySort throughput and cluster totals, and hash partitioning. All use simple multiplication and division. Convention: 1 GB = 1,000 MB, 1 TB = 1,000 GB, 1 PB = 1,000 TB.

Iterative disk I/O: MapReduce vs. Spark

```
MR_IO = k × (D_read + D_write)
Spark_HDFS_IO = D (read once)
Reduction = MR_IO / Spark_HDFS_IO
```

WHAT EACH SYMBOL MEANS

- **k**: number of iterations
- **D**: dataset size
- **D_read, D_write**: data read and written to HDFS per iteration (both = D in the slide example)
- **Reduction**: how many times less disk I/O Spark does

WORKED EXAMPLE

K-Means, D = 50 GB, k = 100 (P1 slide 6).

- MR_IO = 100 × (50 + 50) = 10,000 GB = 10 TB
- Spark_HDFS_IO = 50 GB
- Reduction = 10,000 / 50 = 200×

PRACTICE

D = 20 GB, k = 30.

- MR_IO = 30 × (20 + 20) = 1,200 GB = 1.2 TB
- Spark = 20 GB
- Reduction = 1,200 / 20 = 60×

I/O time and wall-clock speedup

```
t = size / speed
T_MR = k × (D/R_hdfs + D/W_hdfs)
```

```
T_Spark = D/R_hdfs + (k - 1) × D/R_ram
Speedup = T_MR / T_Spark
```

WHAT EACH SYMBOL MEANS

- **R_hdfs**: HDFS read speed (MB/s); **W_hdfs**: HDFS write speed (assumed equal to R_hdfs when not given)
- **R_ram**: RAM cache read speed (MB/s)
- **k, D**: iterations and data size in MB

WORKED EXAMPLE

P1 slide 38 A1: D = 50,000 MB, k = 100, R_hdfs = W_hdfs = 500 MB/s, R_ram = 5,000 MB/s.

- One HDFS pass: 50,000 / 500 = 100 s
- T_MR = 100 × (100 + 100) = 20,000 s ≈ 5.56 h
- One RAM pass: 50,000 / 5,000 = 10 s
- T_Spark = 100 + 99 × 10 = 1,090 s ≈ 18.2 min
- Speedup = 20,000 / 1,090 ≈ 18.3×

Second slide example (P2 slide 4): 10 iterations over 100 GB at 100 MB/s. Reads alone: 10 × 100,000 MB / 100 MB/s = 10,000 s ≈ 2.8 h, which is the slide's "≈ 3 hours of I/O". Counting writes too doubles it to 20,000 s ≈ 5.6 h.

PRACTICE

D = 20 GB (20,000 MB), k = 30, HDFS 400 MB/s read and write, RAM 4,000 MB/s.

- HDFS pass = 20,000 / 400 = 50 s; T_MR = 30 × (50 + 50) = 3,000 s = 50 min
- RAM pass = 20,000 / 4,000 = 5 s; T_Spark = 50 + 29 × 5 = 195 s
- Speedup = 3,000 / 195 ≈ 15.4×

Benchmark speedup per iteration

```
Speedup = t_old / t_new
```

WHAT EACH SYMBOL MEANS

- **t_old**: time per iteration on Hadoop MapReduce
- **t_new**: time per iteration on Spark

WORKED EXAMPLE

RDD paper (P2 slide 10): logistic regression 127 s vs. 0.96 s: 127 / 0.96 ≈ 132×. K-means 150 s vs. 5 s: 150 / 5 = 30×. Over 10 iterations of logistic regression: Hadoop 1,270 s vs. Spark 9.6 s.

PRACTICE

Hadoop 120 s/iteration, Spark 1.5 s/iteration: speedup = 120 / 1.5 = 80×.

GraySort throughput, cluster totals and efficiency per node

```
Throughput = data / time
Total resource = nodes × per-node amount
Time ratio = T_old / T_new; Node ratio = N_old / N_new
Efficiency per node = (T_old × N_old) / (T_new × N_new)
```

WHAT EACH SYMBOL MEANS

- **T**: sort time in minutes; **N**: number of nodes
- **T × N**: node-minutes, the total machine time used
- **Efficiency per node**: how many times fewer node-minutes the new system needs

WORKED EXAMPLE

Databricks (P2 slides 13 to 14, 20): Spark 100 TB in 23 min on 206 nodes; Hadoop 72 min on 2,100 nodes.

- Throughput = 100 / 23 ≈ 4.35 TB/min (the slide's 4.27 TB/min matches the more exact 23.4 min: 100 / 23.4 ≈ 4.27)
- Total RAM = 206 × 244 GB = 50,264 GB ≈ 50.3 TB
- SSD per node = 8 × 800 GB = 6,400 GB = 6.4 TB; total = 206 × 6.4 = 1,318.4 TB ≈ 1.3 PB
- Time ratio = 72 / 23 ≈ 3.1×; node ratio = 2,100 / 206 ≈ 10.2×
- Efficiency per node = (72 × 2,100) / (23 × 206) = 151,200 / 4,738 ≈ 32×
- Records = 100 TB / 100 bytes = 10^{14} / 10^{2} = 10^{12} records; HDFS storage with 3× replication = 300 TB

Note: P2 slide 2 says "3.5× fewer machines", but 2,100 / 206 gives 10.2×, as slide 14 states.

PRACTICE

System A sorts 50 TB in 60 min on 1,000 nodes; system B in 20 min on 250 nodes.

- B throughput = 50 / 20 = 2.5 TB/min
- Time ratio = 60 / 20 = 3×; node ratio = 1,000 / 250 = 4×
- Efficiency per node = (60 × 1,000) / (20 × 250) = 60,000 / 5,000 = 12×

Hash partitioning and shuffle volume

```
partition = key.hashCode() % numPartitions
groupByKey records shuffled = total pairs
```

```
reduceByKey records shuffled ≤ partitions × distinct keys per partition
```

WHAT EACH SYMBOL MEANS

- **hashCode()**: a number computed from the key
- **%**: remainder after division
- **numPartitions**: number of output partitions

WORKED EXAMPLE

Default hash partitioner (P2 slide 10), 4 partitions: a key with hash 17 goes to 17 % 4 = 1; hash 42 goes to 42 % 4 = 2. Shuffle: 4 partitions × 1,000 pairs, 50 distinct words each. groupByKey shuffles 4,000 records; reduceByKey at most 4 × 50 = 200, i.e. 1 − 200/4,000 = 95% less (P1 slide 22).

PRACTICE

Hashes 10, 23, 31 with numPartitions = 4: 10 % 4 = 2, 23 % 4 = 3, 31 % 4 = 3. Recovery share: if 2 of 10 partitions are lost with only narrow dependencies, 2 / 10 = 20% of the work is redone.

Chapter 3 · Week 7: Querying Big Data: Hive, HiveQL & the SQL-on-Hadoop Ecosystem

The chapter in plain words

The problem Hive solves. Hadoop stores huge data on HDFS and processes it with MapReduce (MR), but every MR job had to be a Java program, often 100+ lines for a simple GROUP BY. Analysts know SQL, not Java, so a simple question took days to answer (P1 slide 5). Facebook needed thousands of non-engineers to query petabytes.

Hive = SQL on top of Hadoop. Apache Hive (Facebook, 2008) gives you HiveQL, a SQL-like language. You write SELECT; Hive translates it into MapReduce jobs, Tez DAGs or Spark jobs (the "execution engine"). Analogy: you order food in your own language and the waiter (Hive) translates it into kitchen instructions (MR jobs) (P1 slide 5).

The parts of Hive. HiveServer2 accepts queries from clients (Beeline, JDBC, Tableau). The Driver parses, checks, plans and optimises the query. The Metastore is a normal relational database (MySQL/PostgreSQL) that stores table schemas, partition lists and HDFS file locations. The Execution Engine runs the jobs. HDFS holds the actual data: a table is a directory, a partition is a sub-directory (P1 slides 8-9).

Schema-on-read. A traditional database (MySQL) checks data when you INSERT it (schema-on-write) and rejects bad rows. Hive just copies files to HDFS and applies the schema only when you query. Loading is fast and flexible, but bad values show up silently as NULL at query time (P1 slides 13-14).

Partitioning and bucketing. Partitioning splits a table into HDFS sub-directories by a low-cardinality column such as date or country, so a query with WHERE on that column reads only the matching folders (partition pruning). Bucketing splits data further into a fixed number N of files using hash(column) % N, so two tables bucketed the same way on a join key can be joined bucket-by-bucket without a big shuffle (P1 slides 19-24).

Hive is batch, not interactive. HiveQL looks like SQL, but a COUNT(*) on 1 TB can take 5-10 minutes on MapReduce because MR writes intermediate results to disk between stages. Presto (Meta, 2013) keeps everything in an in-memory pipeline and answers in seconds. Use Hive for big ETL and nightly jobs, Presto/Trino or Spark SQL for dashboards and ad-hoc questions (P1 slides 17, 26).

The research and the case study. Thusoo et al. (VLDB 2009, ICDE 2010) describe Hive at Facebook: 2 PB, 15 TB/day growth, 7,500+ queries/day, 200+ analysts. Their most lasting idea is the Metastore, now the shared catalogue for Spark, Presto/Trino, Impala and AWS Glue (P2 slides

5-12). Spotify used HiveQL plus Spark, scheduled with Luigi, over ~3 billion events/day and 2 billion playlists to build Discover Weekly (P2 slides 14-18).

Part 1 · Student Assessment Pool: MCQ

Q1. In Hive, where is table schema metadata (column names, types, partition info) stored?

(a) HDFS NameNode
(b) Hive Metastore (relational DB such as MySQL)
(c) HBase
(d) ZooKeeper

ANSWER

Answer: (b) Hive Metastore (relational DB such as MySQL).

The Metastore is a relational database (MySQL or PostgreSQL) that stores table and column definitions, data types, partition values with their HDFS paths, SerDe info, statistics and privileges (P1 slide 10). Without it Hive cannot find or parse any table (P1 slide 11).

- (a) Wrong: the NameNode stores HDFS block and file metadata (which blocks make up a file, where they live), not table schemas. This is the common distractor (P1 slide 28 instructor note).
- (c) Wrong: HBase is a NoSQL column store for data, not Hive's catalogue.
- (d) Wrong: ZooKeeper does coordination (locks, leader election), not schema storage.

Q2. What does "schema-on-read" mean in the context of Hive?

(a) Schema is validated when data is inserted into the table
(b) Schema is applied when data is queried, not when it is stored on HDFS
(c) Schema is stored separately from data in HDFS metadata
(d) Schema is inferred automatically from column names in the file header

ANSWER

Answer: (b) Schema is applied when data is queried, not when it is stored on HDFS.

Hive copies raw files to HDFS as they are. The column types are only applied at SELECT time. A bad value (text "abc" in a DOUBLE column) is stored and comes back as NULL (P1 slide 13).

- (a) Wrong: that is schema-on-write, how MySQL works.
- (c) Wrong: the schema does live separately, but in the Metastore (a relational DB), not in HDFS metadata, and this sentence does not say *when* the schema is applied, which is the point of the term.

- (d) Wrong: that is schema inference (like CSV auto-detect in Spark). In schema-on-read the user defines the schema explicitly with CREATE TABLE (P1 slide 28 instructor note).

Q3. Which Hive optimization technique physically organises data into separate HDFS sub-directories by a column's value to avoid full table scans?

(a) Bucketing
(b) Indexing
(c) Partitioning
(d) Compression

ANSWER

Answer: (c) Partitioning.

Partitioning stores each value of the partition column in its own HDFS sub-directory, e.g. `/data/sales/year=2024/month=01/`. A query with `WHERE year=2024 AND month=1` reads only that folder and skips the rest (P1 slides 19-20).

- (a) Wrong: bucketing splits data *within* a partition into files by hash. It cuts join cost but does not skip data at the directory level (P1 slide 29 instructor note).
- (b) Wrong: Hive has no traditional B-tree indexes; it relies on partitions and buckets (P1 slide 16).
- (d) Wrong: compression makes files smaller but does not split them into directories by value.

Q4. Presto's primary performance advantage over Hive+MapReduce for interactive queries is:

(a) Presto supports more SQL functions than HiveQL
(b) Presto has better native integration with HBase and Cassandra
(c) Presto executes queries entirely in memory without writing intermediate results to disk
(d) Presto has native Python UDF support

ANSWER

Answer: (c) Presto executes queries entirely in memory without writing intermediate results to disk.

Hive+MR writes intermediate data to HDFS after the map phase and after the reduce phase of every stage. Presto streams data through an in-memory pipeline (split, stage, output). That cuts typical times from 5-15 minutes to 5-60 seconds (P1 slide 26). The key words are "intermediate results to disk" (P1 slide 29 instructor note).

- (a) Wrong: the number of SQL functions does not decide speed.

- (b) Wrong: Presto does have connectors (Cassandra, MySQL, Kafka, S3), but that is about reach, not speed.
- (d) Wrong: UDF language support is not the source of its speed.

Part 1 · Student Assessment Pool: True/False

1. HiveQL supports full ACID transactions on all table types, just like MySQL.

ANSWER

False.

Hive has only limited ACID support: UPDATE/DELETE work only on ORC tables created as ACID (transactional) tables, not on all tables (P1 slide 16).

Corrected: HiveQL supports ACID transactions only on ACID-enabled transactional ORC tables; MySQL supports them on all tables.

2. Partitioning in Hive reduces query time by eliminating irrelevant data partitions from the HDFS scan.

ANSWER

True.

When the WHERE clause filters on the partition key, Hive reads only the matching sub-directories (partition pruning), so less data is read and the job is shorter (P1 slide 19).

3. Hive directly executes a HiveQL query on HDFS without any intermediate translation step.

ANSWER

False.

The Driver parses the HiveQL into a syntax tree, checks it against the Metastore, builds a logical plan, optimises it and generates a physical plan of MapReduce/Tez/Spark jobs, which the execution engine runs (P1 slide 8).

Corrected: Hive compiles a HiveQL query into MapReduce, Tez or Spark jobs, and those jobs read HDFS.

4. Bucketing in Hive is particularly useful for optimising JOIN operations between two large tables.

ANSWER

True.

If both tables are bucketed on the join key into the same number of buckets, Hive joins bucket i of one table with bucket i of the other (bucket map join), so no full sort-merge shuffle is needed (P1 slide 23).

Part 1 · Student Assessment Pool: Descriptive

D1. Explain the Hive architecture: what is the role of HiveServer2, the Driver, and the Metastore? How is a HiveQL query compiled into executable jobs? Draw the component diagram.

ANSWER

Definition. Hive is a data warehouse layer on Hadoop that turns SQL-like HiveQL queries into distributed jobs over files on HDFS (P1 slide 5).

Roles (P1 slide 8):

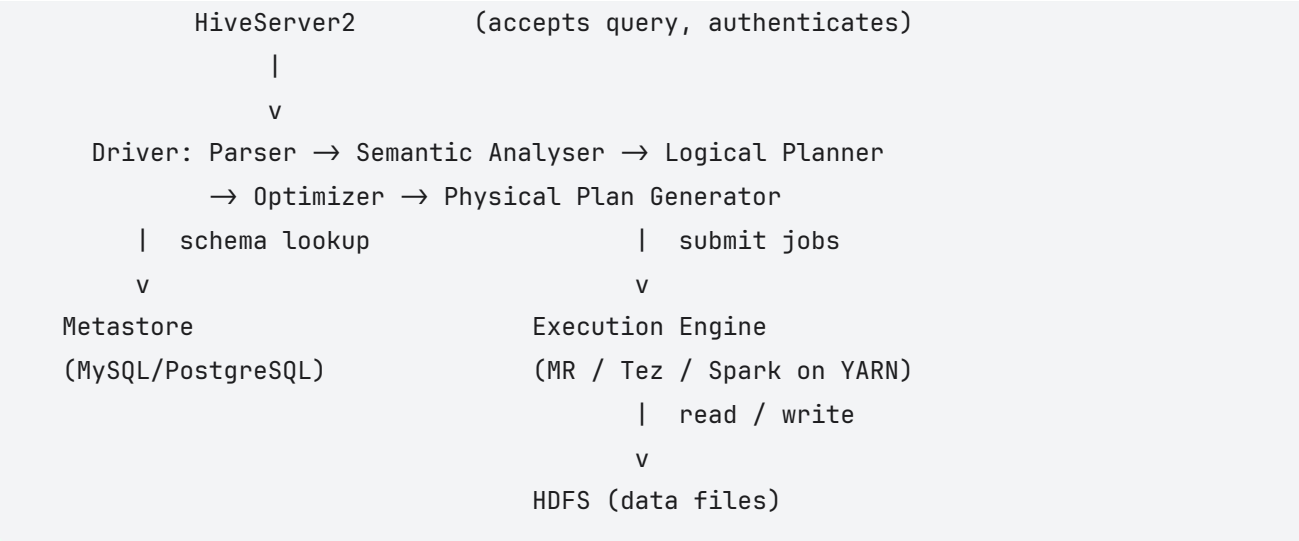
- **HiveServer2**: the front door. A JDBC/ODBC server that authenticates users and accepts HiveQL from Beeline, BI tools (Tableau, Power BI) and JDBC drivers.
- **Driver**: the brain. It runs the pipeline Parser, Semantic Analyser, Logical Planner, Query Optimizer, Physical Plan Generator, and manages the query's lifecycle.
- **Metastore**: the catalogue. A relational DB (MySQL/PostgreSQL) holding table schemas, column types, partition info and HDFS file locations.
- **Execution Engine**: MapReduce, Tez or Spark (`SET hive.execution.engine=spark`). **HDFS** holds the data: tables are directories, partitions are sub-directories.

Compilation steps (P2 slide 11):

1. Parse: HiveQL text into an abstract syntax tree (AST).
2. Semantic analysis: look up tables and columns in the Metastore, check types (e.g. `user_id` is BIGINT, `dt` is a partition column).
3. Logical plan: Filter, Join, GroupBy, OrderBy operators.
4. Optimise and build the physical plan: MR/Tez/Spark jobs, with partition pruning.
5. Submit jobs to YARN in dependency order; results go to HDFS and back to the client.

Diagram (P1 slide 9):

```
Beeline / JDBC / BI tool (Tableau)
              |
              v
```



Example: `SELECT country, COUNT(*) FROM logs GROUP BY country` becomes one MR job: mappers emit (country, 1), reducers sum per country.

D2. Differentiate between partitioning and bucketing in Hive. When would you choose each? Describe a scenario in Bangladesh e-commerce data where both are used together.

ANSWER

Definitions. Partitioning stores data in separate HDFS sub-directories by the value of a column (P1 slide 19). **Bucketing** splits each partition into a fixed number N of files, putting a row in file `hash(column) % N` (P1 slide 23).

Point	Partitioning	Bucketing
Physical unit	Directory per value	Fixed number of files
Decided by	Column value	hash(column) % N
Main benefit	Skips data (partition pruning) for WHERE filters	Faster JOINs (bucket map join) and sampling
Good column	Low cardinality: date, country, region, status	High cardinality join key: user_id
Risk	High-cardinality key creates millions of directories and overloads the NameNode	Both tables need the same bucket count on the join key to get the benefit
DDL	`PARTITIONED BY (year INT, month INT)`	`CLUSTERED BY (user_id) INTO 32 BUCKETS`

Choose partitioning when queries usually filter on a column with few values (dates). **Choose bucketing** when you often join large tables on a key with many values, which you must never

partition on (P1 slide 19: a 100 M user table would create 100 M directories).

Bangladesh scenario. A Daraz-style shop stores orders. Managers run daily sales reports by date and division (Dhaka, Chattogram...). Analysts join orders with a customer table on `customer_id`.

```
CREATE TABLE orders (
  order_id BIGINT, customer_id BIGINT, amount DOUBLE
)
PARTITIONED BY (order_date STRING, division STRING)
CLUSTERED BY (customer_id) INTO 32 BUCKETS
STORED AS ORC;
```

A report for `order_date='2026-10-01' AND division='Dhaka'` reads one folder. A join with a `customers` table also bucketed on `customer_id` into 32 buckets joins bucket 0 with bucket 0, and so on, with no full shuffle.

D3. What is "schema-on-read"? What are its advantages and disadvantages compared to "schema-on-write" in a traditional RDBMS such as MySQL? Give one example where schema-on-read causes a data quality problem.

ANSWER

Definition. Schema-on-read means the table structure (columns and types) is applied when data is queried, not when it is loaded. Hive copies raw files to HDFS and only interprets them at SELECT time (P1 slide 13). Schema-on-write (MySQL) checks every row at INSERT time and rejects bad rows.

Advantages of schema-on-read (P1 slides 13-14):

- Fast loading: a raw file copy to HDFS, no validation cost.
- Flexible: load web logs before you know the final schema; no ETL needed at ingest.
- Many schemas on the same data: define several Hive tables over the same files as needs change.
- Open formats: ORC, Parquet, CSV, JSON, Avro.

Disadvantages:

- Slower queries: data is parsed and checked on every query.
- Bad data is silent: invalid values become NULL instead of raising an error.
- Suited to OLAP (analysis), not OLTP like banking or e-commerce orders, where schema-on-write is better.

Data quality example (P1 slide 13). A bKash transaction file has `abc` in the `amount` column (declared DOUBLE). MySQL would reject the INSERT. Hive stores it, and the query returns NULL for

that row. `SUM(amount)` then ignores it, so the revenue report is too low and nobody is warned. Fix: add downstream checks such as Great Expectations or dbt tests (P1 slide 14).

Part 1 · Student Assessment Pool: Analytical

A1. Partitioning & Bucketing Design: You have a 5 TB e-commerce transaction table with columns: (txn_id, user_id, product_id, amount, country, txn_date). Design a complete partitioning and bucketing strategy to optimise: – Daily revenue reports filtered by country and txn_date – JOINs between this table and a 10 M-row user profile table (keyed on user_id). Write the complete CREATE TABLE DDL. Justify why you chose a specific number of buckets and which column(s) you partitioned on.

ANSWER

Step 1: pick partition columns. The reports filter on `txn_date` and `country`. Both have low cardinality (a few hundred or thousand days, a handful of countries), so they are safe partition keys (P1 slide 19). Never partition on `user_id` or `txn_id`: millions of directories would overload the NameNode.

Step 2: pick the bucket column. The join key is `user_id`, which has high cardinality. Bucket both tables on `user_id` with the same number of buckets so Hive can do a bucket map join (bucket i with bucket i, no full shuffle) (P1 slide 23). `SORTED BY (user_id)` also enables a sort-merge bucket join.

Step 3: size the buckets. Assumptions (state these in the exam): 3 years of data and 5 countries.

- Partitions = 3 × 365 days × 5 countries = 1,095 × 5 = 5,475.
- Data per partition = 5 TB / 5,475 ≈ 0.913 GB.
- With 8 buckets: 0.913 GB / 8 ≈ 114 MB per file, close to one 128 MB HDFS block. Good.
- With 32 buckets: about 28.5 MB per file: too many small files (5,475 × 32 = 175,200 files) for the NameNode.
- Total files with 8 buckets: 5,475 × 8 = 43,800.
- User table: 10 M rows × ~200 bytes ≈ 2 GB; 8 buckets ≈ 250 MB each.

So 8 buckets gives files near the block size on both sides, and the same count on both tables makes the bucket map join possible. If the data were bigger per day, raise the count (16, 32) keeping it equal or a multiple on both tables.

Step 4: DDL and queries.

```
-- Fact table: about 5 TB
CREATE TABLE transactions (
  txn_id      BIGINT,
  user_id     BIGINT,
  product_id  BIGINT,
```

```
  amount      DOUBLE
)
PARTITIONED BY (txn_date STRING, country STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;

-- Dimension table: 10 M users, same key, same bucket count
CREATE TABLE user_profile (
  user_id   BIGINT,
  name      STRING,
  city      STRING,
  signup_dt DATE
)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;

-- Load with dynamic partitions (partition columns last)
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;
INSERT INTO transactions PARTITION (txn_date, country)
SELECT txn_id, user_id, product_id, amount, txn_date, country
FROM raw_transactions;

-- Daily report: reads one directory
SELECT SUM(amount) AS revenue
FROM transactions
WHERE txn_date = '2024-10-01' AND country = 'BD';

-- Join: bucket i with bucket i
SET hive.optimize.bucketmapjoin = true;
SET hive.optimize.bucketmapjoin.sortedmerge = true;
SELECT u.city, SUM(t.amount)
FROM transactions t JOIN user_profile u ON t.user_id = u.user_id
WHERE t.txn_date = '2024-10-01'
GROUP BY u.city;
```

Effect. A daily report for one country reads one partition, about 1/5,475 of the table (about 0.018%) instead of 5 TB. The join reads matching bucket pairs instead of shuffling both tables. Note: partition columns are not repeated in the column list, and must come last in a dynamic partition INSERT (P1 slide 22).

A2. Query Compilation to MapReduce: Write HiveQL to find the top 5 products by total revenue in 2023. Then trace how Hive compiles this query into MapReduce stages: what is emitted by the Map phase, what is the shuffle key, and what does the Reduce phase compute? How would switching the execution engine from MapReduce to Tez change the execution plan?

ANSWER

Step 1: HiveQL (same style as P1 slide 18):

```
SELECT   product_id,
         SUM(amount) AS total_revenue
FROM     transactions
WHERE    YEAR(txn_date) = 2023
GROUP BY product_id
ORDER BY total_revenue DESC
LIMIT    5;
```

Step 2: compilation (P1 slide 8, P2 slide 11). Parse to AST; semantic analysis checks `transactions`, `product_id`, `amount`, `txn_date` in the Metastore; logical plan = Filter, GroupBy, OrderBy, Limit; physical plan = two MR jobs.

Job 1: aggregate.

- **Map:** read each row (or only the 2023 partitions if the table is partitioned by date), drop rows where YEAR(txn_date) is not 2023, emit `(product_id, amount)`. Hive does map-side aggregation (a combiner): each mapper emits one partial sum per product (P2 slide 10).
- **Shuffle key:** `product_id`. All partial sums for one product go to the same reducer.
- **Reduce:** add the partial sums: `(product_id, SUM(amount))`. Output is written to a temporary HDFS file.

Job 2: sort and limit.

- **Map:** read Job 1 output, emit `(total_revenue, product_id)`; each mapper can keep only its own top 5 (limit push-down).
- **Shuffle key:** `total_revenue` in descending order, all to **one** reducer because ORDER BY needs a global order.
- **Reduce:** output the first 5 rows.

Tiny example. 2023 rows: (P1,100), (P2,50), (P1,30), (P3,80), (P2,20), (P4,10). Job 1 gives P1=130, P2=70, P3=80, P4=10. Job 2 sorts: P1 130, P3 80, P2 70, P4 10 (all 4 shown since fewer than 5).

Step 3: MapReduce vs Tez. With `SET hive.execution.engine=tez` the two jobs become one DAG: Map 1 (scan, filter, partial sum) to Reducer 2 (final sum) to Reducer 3 (sort, limit 5). Differences:

- No HDFS write between Job 1 and Job 2: data passes directly between stages.

- One job submission instead of two, and containers can be reused, so less start-up cost.
- Same HiveQL, same result; only the plan and speed change (Tez is "DAG of tasks, faster", P1 slide 8).

Part 2 · Discussion Questions

1. Spotify's co-occurrence query runs GROUP BY song_a, song_b over 2 billion playlist rows. Estimate the output size if there are 30M songs. Propose a partitioning + bucketing scheme for the playlist_songs Hive table that would reduce the shuffle size for this query. [CLO4 – Apply]

ANSWER

What the query does. For every playlist, every pair of songs in it is one row; GROUP BY counts how many playlists contain each pair; `HAVING COUNT(*) > 5` keeps frequent pairs (P2 slide 16).

Estimate 1: upper bound from the song count. Possible distinct pairs = 30M × (30M − 1) / 2 ≈ 4.5 × 10^14. At 16 bytes per output row (two 4-byte song ids + 8-byte count) that is about 7.2 PB. This can never happen in practice, because pairs only come from real playlists.

Estimate 2: from the data. Assume the 2 billion rows are (playlist, song) rows and a playlist has about 20 songs on average.

- Playlists = 2 × 10^9 / 20 = 10^8 (100 million).
- Pairs per playlist = 20 × 19 / 2 = 190.
- Pair rows before GROUP BY = 10^8 × 190 = 1.9 × 10^10 (19 billion).
- Distinct pairs are at most 1.9 × 10^10, which is about 0.0042% of the 4.5 × 10^14 possible pairs; at 16 bytes that is at most about 304 GB.
- After `HAVING COUNT(*) > 5` most pairs vanish (the long tail), so the final output is far smaller, likely a few GB.

(If instead there are 2 billion playlists of 20 songs, the pair rows are 2 × 10^9 × 190 = 3.8 × 10^11, about 6.08 TB at 16 bytes. Show your assumption.)

Where the shuffle comes from. (1) The self-join on `playlist_id` to build pairs, which shuffles all 2B rows by playlist. (2) The GROUP BY on (song_a, song_b), which shuffles the pairs.

Proposed scheme:

```
CREATE TABLE playlist_songs (
  playlist_id BIGINT,
  song_id     INT,
  added_ts    TIMESTAMP
)
PARTITIONED BY (week STRING)
```

```
CLUSTERED BY (playlist_id) SORTED BY (playlist_id, song_id)
INTO 1024 BUCKETS
STORED AS ORC;
```

- **Partition by week:** the weekly job reads only new or changed weeks. The slides report that partitioning play_events by week cut the query from 8 hours to 2 hours, a 4× speed-up (P2 slide 18).
- **Bucket by playlist_id:** all songs of one playlist sit in one bucket file, so the self-join is done inside each bucket with no shuffle. Sorting by (playlist_id, song_id) gives a sort-merge bucket join and makes `a.song_id < b.song_id` pairs easy to generate.
- **Map-side aggregation** on (song_a, song_b) shrinks the second shuffle, the same trick that cut shuffle data about 250,000× for GROUP BY country (P2 slide 10).
- Do not partition by song_id or playlist_id: 30M or 100M+ directories would crash the NameNode (P1 slide 19).

Other side. Bucketing by playlist_id does not help the GROUP BY shuffle itself, because the group key is the song pair. An option is to precompute and store the weekly pair counts and add each new week incrementally (Luigi skips finished outputs, P2 slide 14).

2. The ALS collaborative filtering model is a matrix factorisation over a 75M-user × 30M-song matrix. At 4 bytes per float and rank k = 50, calculate the model size and explain whether it fits in a 5,000-node YARN cluster's aggregate RAM. [CLO4 – Analyse]

ANSWER

Idea. ALS (Alternating Least Squares) does not store the full user × song matrix. It learns two small matrices: a user matrix U (one row of k numbers per user) and a song matrix V (one row of k numbers per song). A user's predicted liking for a song = dot product of their two rows. Analogy: describe each person and each song by 50 "taste" scores instead of storing every person-song rating.

Calculation.

- User factors: 75 × 10^6 × 50 × 4 B = 1.5 × 10^10 B = 15 GB.
- Song factors: 30 × 10^6 × 50 × 4 B = 6 × 10^9 B = 6 GB.
- Model size = (75M + 30M) × 50 × 4 B = 2.1 × 10^10 B = **21 GB** (about 19.6 GiB).

Compare with the full matrix. 75M × 30M × 4 B = 9 × 10^15 B = 9 PB. Factorisation makes the model about 430,000 times smaller (9 × 10^15 / 2.1 × 10^10).

Does it fit? The slides do not give RAM per node. Assume 64 GB per node: aggregate RAM = 5,000 × 64 GB = 320 TB. 21 GB is about 0.0066% of that. Even split evenly it is about 4.2 MB per node. So yes, it fits easily; it would even fit in one large server's memory. The dense 9 PB matrix would not fit (about 28 times the 320 TB).

Caveats. Training needs more than the model: the rating/interaction data, shuffled factor blocks each iteration, and JVM overhead, often several times the model size. Spark ALS partitions U and V into blocks across executors so each node holds only a slice. Still well within the cluster.

3. Spotify uses Luigi while other companies use Apache Airflow. Compare the two on: dependency declaration style, backfill capability, and UI/observability. Which would you choose for a 100-step daily pipeline? [CLO3 – Evaluate]

ANSWER

Background. Both are Python workflow schedulers: they run many jobs (HiveQL, Spark) in the right order. Luigi was open-sourced by Spotify in 2012. A Luigi task declares its **output** (an HDFS path) and its **input dependencies** (other tasks), and Luigi skips tasks whose output already exists (P2 slide 14). Airflow (from Airbnb, now Apache) is general knowledge, not on the slides.

Aspect	Luigi	Airflow
Dependency style	Target-based and pull: each Task class has `requires()`, `output()`, `run()`. You ask for the final task; Luigi walks back through requirements and runs only what is missing.	DAG-based and push: a DAG file lists tasks (operators) and edges (`t1 >> t2`). The scheduler runs the DAG on a timetable.
Backfill	Re-run for past dates by passing date parameters (or range tasks); idempotent because finished outputs are skipped. No built-in scheduler: usually triggered by cron.	Built in: each run has a logical date; `catchup` and `airflow dags backfill` run missed dates; per-task retries and clearing.
UI / observability	Simple central-scheduler web page showing task status and the dependency graph.	Rich web UI: grid and graph views, per-task logs, run history, retries, SLAs and alerting.

Choice for a 100-step daily pipeline: Airflow. With 100 steps you need a clear view of which step failed, its logs, automatic retries, alerts, and an easy way to re-run past days. Airflow gives these out of the box.

Other side. Luigi is lighter and its "skip if output exists" model is a natural fit for file-based Hadoop pipelines like Spotify's, where it enabled incremental daily runs (P2 slide 14). A small team already using Luigi might keep it. But at 100 steps the monitoring and backfill tooling of Airflow outweighs that.

Part 2 · Live Exercise

Task. Task (25 minutes, pairs): Simulate a Hive session using PySpark's SQL interface (which reads the same Hive Metastore). Dataset: a 10,000-row synthetic play_events CSV (generate with the skeleton below).

```
from pyspark.sql import SparkSession
import random, datetime

spark = SparkSession.builder \
        .appName("HiveDemo") \
        .getOrCreate()
spark.setLogLevel("WARN")

# Generate synthetic data
rows = []
for i in range(10_000):
    rows.append({
      "user_id":    random.randint(1, 500),
      "song_id":    random.randint(1, 200),
      "playlist_id":random.randint(1, 1000),
      "dt":         "2024-10-01"
    })

df = spark.createDataFrame(rows)
df.createOrReplaceTempView("play_events")

# Co-occurrence query
result = spark.sql("""
    SELECT a.song_id AS song_a,
           b.song_id AS song_b,
           COUNT(*) AS co_count
    FROM play_events a
    JOIN play_events b
      ON a.playlist_id = b.playlist_id
     AND a.song_id < b.song_id
    GROUP BY a.song_id, b.song_id
    HAVING COUNT(*) > 2
    ORDER BY co_count DESC
""")
result.show(10)
```

ANSWER

What the skeleton does. It builds 10,000 random rows (user 1-500, song 1-200, playlist 1-1000, all on dt 2024-10-01), registers them as a temporary view `play_events`, then runs the Spotify-style co-

occurrence query: a self-join on `playlist_id` makes every pair of songs in the same playlist, `a.song_id < b.song_id` keeps each pair once and drops a song paired with itself, GROUP BY counts the pairs, HAVING keeps pairs seen more than 2 times (P2 slide 16 uses > 5 at Spotify scale).

Steps.

1. Install PySpark (`pip install pyspark`) or open `pyspark` on the lab Hadoop machine.
2. Save the complete script below as `live_w07.py` and run `spark-submit live_w07.py`.
3. Read the printed tables, timings and plans, and answer Goals 1-4.

Complete script (expected to run; not run here). Changes from the skeleton: fixed random seed, two dates so pruning shows up, timing, the songs table, and EXPLAIN. `datetime` from the skeleton is unused and dropped.

```
# live_w07.py  -- run with: spark-submit live_w07.py   (or paste into pyspark)
from pyspark.sql import SparkSession
import random, time

spark = SparkSession.builder \
        .appName("HiveDemo") \
        .getOrCreate()
spark.setLogLevel("WARN")

# ---- Generate synthetic data (two dates so pruning is visible) ----
random.seed(42)
rows = []
for i in range(10_000):
    rows.append({
      "user_id":     random.randint(1, 500),
      "song_id":     random.randint(1, 200),
      "playlist_id": random.randint(1, 1000),
      "dt":          random.choice(["2024-10-01", "2024-10-02"])
    })
df = spark.createDataFrame(rows)

# ---- Goal 1: partitioned table on dt ----
spark.sql("DROP TABLE IF EXISTS play_events_part")
(df.write.mode("overwrite")
   .partitionBy("dt")
   .format("parquet")
   .saveAsTable("play_events_part"))
spark.sql("SHOW PARTITIONS play_events_part").show()

# ---- Goal 2: co-occurrence query, timed ----
cooc_sql = """
```

```
    SELECT a.song_id AS song_a,
           b.song_id AS song_b,
           COUNT(*) AS co_count
    FROM play_events_part a
    JOIN play_events_part b
      ON a.playlist_id = b.playlist_id
     AND a.song_id < b.song_id
    WHERE a.dt = '2024-10-01' AND b.dt = '2024-10-01'
    GROUP BY a.song_id, b.song_id
    HAVING COUNT(*) > 2
    ORDER BY co_count DESC
"""
t0 = time.time()
result = spark.sql(cooc_sql)
result.show(10)
print("rows kept:", result.count())
print("co-occurrence time: %.2f s" % (time.time() - t0))

# ---- Goal 3: songs dimension table + broadcast hint ----
genres = ["pop", "rock", "baul", "rabindra", "hiphop"]
songs = spark.createDataFrame(
    [(s, "Song %d" % s, random.choice(genres)) for s in range(1, 201)],
    ["song_id", "title", "genre"])
songs.createOrReplaceTempView("songs")

plain_sql = """
    SELECT s.genre, COUNT(*) AS plays
    FROM play_events_part p JOIN songs s ON p.song_id = s.song_id
    WHERE p.dt = '2024-10-01'
    GROUP BY s.genre
"""
hint_sql = """
    SELECT /*+ BROADCAST(s) */ s.genre, COUNT(*) AS plays
    FROM play_events_part p JOIN songs s ON p.song_id = s.song_id
    WHERE p.dt = '2024-10-01'
    GROUP BY s.genre
"""
# turn off automatic broadcast so the comparison is fair
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
t0 = time.time(); spark.sql(plain_sql).collect(); t_plain = time.time() - t0
t0 = time.time(); spark.sql(hint_sql).collect();  t_hint  = time.time() - t0
print("no hint: %.2f s   broadcast hint: %.2f s" % (t_plain, t_hint))

# ---- Goal 4: EXPLAIN, find partition pruning ----
spark.sql("EXPLAIN FORMATTED " + hint_sql).show(truncate=False)
spark.sql(hint_sql).explain(mode="formatted")
```

```
spark.stop()
```

Expected output size (from a Python simulation of the same random data, 20 seeds, using the skeleton's single date). Each playlist gets about 10,000 / 1,000 = 10 rows, so the join produces about 50,000 pair rows; these fall into about 18,000 distinct song pairs out of 200 × 199 / 2 = 19,900 possible; about 9,000 pairs pass `HAVING COUNT(*) > 2`; the top `co_count` is about 11-13. With the full script's two dates, the day-1 filter keeps about half the rows (about 5 per playlist), so the counts are much smaller and fewer pairs pass. Exact numbers change with the random data.

1. Create a partitioned Spark table (partition on dt).

ANSWER

`df.write.partitionBy("dt").saveAsTable("play_events_part")` writes one sub-directory per date under the warehouse folder: `spark-warehouse/play_events_part/dt=2024-10-01/` and `dt=2024-10-02/`, the same layout as Hive's `year=2024/month=01/` (P1 slide 20). The table and its partitions are registered in the catalogue (the Metastore if Spark is built with Hive support, otherwise Spark's own catalogue). Expected `SHOW PARTITIONS` output: two rows, `dt=2024-10-01` and `dt=2024-10-02`.

Hive equivalent: `CREATE TABLE play_events (user_id INT, song_id INT, playlist_id INT) PARTITIONED BY (dt STRING) STORED AS ORC;` plus a dynamic partition INSERT with `dt` last in the SELECT (P1 slide 22).

2. Run the co-occurrence query and time it.

ANSWER

Wrap the query in `time.time()` calls. Call an action (`show`, `count`, `collect`): Spark is lazy (Week 6), so `spark.sql()` alone only builds a plan and takes almost no time.

Expected: a table of `song_a, song_b, co_count` sorted by co_count descending, and a time of a few seconds on a laptop; most of it is Spark start-up and the shuffle for the join and GROUP BY, since 10,000 rows is tiny. The first run is slower than later runs (JVM warm-up). With the skeleton's single date the top count is about 11-13 and about 9,000 pairs are kept (simulated); with the two-date script, fewer.

Why it is expensive at scale: the self-join grows with the square of playlist length (10 songs give 45 pairs), which is why Spotify partitioned by week and needed hours on 2 billion playlists (P2 slide 18).

3. Add a broadcast join hint for the songs dimension table and compare timings.

ANSWER

A **broadcast join** sends a full copy of the small table (songs: 200 rows) to every executor. Each executor joins its slice of the big table locally, so the big table is not shuffled. Hive calls this a map-side join. The hint is `/*+ BROADCAST(s) */` right after SELECT.

Without the hint (and with `spark.sql.autoBroadcastJoinThreshold = -1` so Spark does not broadcast on its own) Spark uses a SortMergeJoin: both sides are shuffled by song_id and sorted. With the hint the plan shows **BroadcastHashJoin** and a **BroadcastExchange** for songs.

Expected: the hinted query is a little faster, but on 10,000 rows the gap is small, often well under a second and sometimes lost in noise. At real scale (billions of play events, 30M songs) avoiding the shuffle of the big table is a large saving. Note: by default Spark already broadcasts tables under 10 MB automatically, which is why the script turns that off for the comparison.

4. Use EXPLAIN to print the physical plan and identify where partition pruning occurs.

ANSWER

Run `EXPLAIN FORMATTED <query>` or `df.explain(mode="formatted")`. Read the plan from the bottom (scan) up.

Where pruning shows: in the **FileScan parquet** node for `play_events_part`, the line

```
PartitionFilters: [isnotnull(dt#..), (dt#.. = 2024-10-01)]
```

That means the filter on `dt` is applied to directory names before any file is opened, so only the `dt=2024-10-01/` folder is read and `dt=2024-10-02/` is skipped. Filters on normal columns appear instead as `PushedFilters` (checked inside the files). Expected plan, roughly: FileScan (with PartitionFilters) then BroadcastHashJoin with BroadcastExchange(songs) then HashAggregate (partial) then Exchange then HashAggregate (final).

With the original skeleton (a temp view from an in-memory DataFrame, one date only) there are no partitions on disk, so the plan shows a scan of an in-memory/RDD source and no PartitionFilters. That is why Goal 1 writes a real partitioned table. Same idea as Hive's partition pruning in P2 slide 11 (read only dt=2024-10-01/country=BD, IN, US and skip the other 365 × 200 partitions).

Maths for Week 7

This chapter has a handful of simple calculations: how much map-side aggregation shrinks a shuffle, how much data partition pruning skips, which bucket a row lands in, how big partitions and bucket files are,

how big an ALS model is, and speed-up ratios. Each is one or two steps. All numbers below were checked in Python.

Map-side aggregation (combiner) shuffle reduction

```
Reduction factor = N_rows / G
```

WHAT EACH SYMBOL MEANS

- **N_rows**: rows a mapper would emit without aggregation (one per input row).
- **G**: number of distinct groups (GROUP BY values); with aggregation the mapper emits at most one row per group.

This is the ideal case of one mapper. With M mappers the shuffle is up to M × G rows, so the real factor is N_rows / (M × G).

WORKED EXAMPLE

P2 slide 10: `SELECT country, COUNT(*) FROM logs GROUP BY country`, 50 million rows, 200 countries.

50,000,000 / 200 = **250,000×** less shuffle data.

PRACTICE

120 million rows grouped by 400 regions. Reduction = 120,000,000 / 400 = **300,000×**.

Partition pruning: fraction of data read

```
Fraction read = P_read / P_total    (assuming equal-size partitions)
```

WHAT EACH SYMBOL MEANS

- **P_read**: partitions matching the WHERE filter.
- **P_total**: all partitions = product of the number of values of each partition column.

WORKED EXAMPLE

P2 slide 11: table partitioned by dt (365 days) and country (200). P_total = 365 × 200 = 73,000. Query reads dt=2024-10-01 for BD, IN, US: P_read = 3.

Fraction = 3 / 73,000 ≈ 0.0041%; skipped = 72,997 partitions.

P1 slide 20: 1 month of 36 = 1/36 ≈ 2.78% read.

PRACTICE

24 months × 50 countries; query one month for 2 countries. P_total = 1,200, P_read = 2. Fraction = 2 / 1,200 ≈ **0.167%**.

Bucket assignment

```
bucket = hash(column) mod N
```

WHAT EACH SYMBOL MEANS

- **hash(column)**: hash of the bucketing column; for an INT/BIGINT key in Hive this is essentially the value itself.
- **N**: number of buckets from `INTO N BUCKETS`.
- **mod**: remainder after division.

Rows with the same key always land in the same bucket number, in every table bucketed the same way, which is what makes bucket i × bucket i joins work (P1 slides 23-24).

WORKED EXAMPLE

P1 slide 24 uses 4 buckets on user_id. user_id = 98: 98 mod 4 = **2**, so the row goes to bucket 2 in both sales and users.

PRACTICE

32 buckets (P1 slide 24 DDL), user_id = 1,234,567. 1,234,567 mod 32 = **7**.

Partition and bucket file sizing

```
File size ≈ S / (P × N);   files = P × N
```

WHAT EACH SYMBOL MEANS

- **S**: total table size.
- **P**: number of partitions.
- **N**: buckets per partition.

Aim for files near the HDFS block size (128 MB); far smaller means too many small files for the NameNode.

WORKED EXAMPLE

A1: S = 5 TB, 3 years × 5 countries: P = 1,095 × 5 = 5,475. Per partition = 5 TB / 5,475 ≈ 0.913 GB. N = 8: 0.913 GB / 8 ≈ 114 MB per file, 43,800 files. N = 32 would give about 28.5 MB files.

PRACTICE

S = 2 TB, partitioned by day for 2 years (P = 730), N = 16. File size = 2 TB / (730 × 16) = 2 TB / 11,680 ≈ **171 MB**; 11,680 files.

ALS model size

```
Size = (U + I) × k × b
```

WHAT EACH SYMBOL MEANS

- **U**: number of users.
- **I**: number of items (songs).
- **k**: rank, the number of latent factors per user/item.
- **b**: bytes per number (4 for a float).

Compare with the dense matrix U × I × b.

WORKED EXAMPLE

P2 slide 18: (75M + 30M) × 50 × 4 = 2.1 × 10^10 B = **21 GB**. Dense matrix: 75M × 30M × 4 = 9 × 10^15 B = 9 PB. With 5,000 nodes × 64 GB (assumed) = 320 TB RAM, 21 GB fits easily.

PRACTICE

10M users, 1M songs, k = 100, 4 bytes. Size = 11M × 100 × 4 = 4.4 × 10^9 B = **4.4 GB**.

Speed-up and time saved

```
Speed-up = T_old / T_new;   time saved % = (T_old - T_new) / T_old × 100
```

WHAT EACH SYMBOL MEANS

- **T_old**: time before the change.
- **T_new**: time after.

Use the same units for both.

WORKED EXAMPLE

P2 slide 18: partitioning play_events by week took the query from 8 h to 2 h. Speed-up = 8 / 2 = **4×**; saved = (8 − 2) / 8 = **75%**.

P1 slide 26: Hive+MR 5-15 min vs Presto 5-60 s. Same-end comparisons: 300 s / 60 s = 5× and 900 s / 5 s = 180×; the slide title rounds the typical gain to 10×.

PRACTICE

A report drops from 2 hours to 30 minutes. Speed-up = 120 / 30 = **4×**; saved = 90 / 120 = **75%**.

Co-occurrence pair count

```
Pairs per playlist = L(L - 1) / 2;   total pairs = number of playlists × L(L - 1) / 2;   max distinct pairs = S(S - 1) / 2
```

WHAT EACH SYMBOL MEANS

- **L**: songs in a playlist.
- **S**: songs in the catalogue.

WORKED EXAMPLE

Discussion Q1: S = 30M gives at most 4.5 × 10^14 distinct pairs. With 2B (playlist, song) rows and L = 20: 10^8 playlists × 190 = 1.9 × 10^10 pairs (about 304 GB at 16 bytes each).

PRACTICE

Live exercise: 1,000 playlists with about 10 songs each. Pairs = 1,000 × 10 × 9 / 2 = **45,000**. (With random playlist sizes the average is a bit higher, about 50,000, which matches the simulation.) Possible distinct pairs from 200 songs = 200 × 199 / 2 = 19,900.

Chapter 4 · Week 8: Data Integration at Scale: Pipelines, Data Lakes & Modern Architectures

The chapter in plain words

Data integration means moving data from the systems where it is born (bank databases, web servers, phone apps) to the systems where it is analysed (Hadoop, dashboards, fraud checkers). The old way was point-to-point ETL (Extract, Transform, Load): one nightly job for every pair of systems. With N systems that grows like N^2, so 50 systems can need 2,450 separate jobs (P1 slide 5). Those jobs break on schema changes, cannot replay lost data, and are at least 24 hours late.

The modern answer is an **event-driven pipeline**: put one central message bus (usually Apache Kafka) in the middle. Sources publish events to the bus, consumers subscribe. Adding a new consumer needs zero changes to any source (P1 slide 5). Think of a post office: senders drop letters at one place instead of driving to every recipient.

Four tools to know (P1 slide 7): **Sqoop** = bulk copy between a relational database and HDFS (nightly batch). **Flume** = collects log files into HDFS using Source, Channel, Sink. **Kafka** = real-time event bus, any source to any consumer. **NiFi** = visual, drag-and-drop data routing. Spark Streaming then processes the streams.

Kafka stores data as an append-only log: new messages are only added at the end, never changed or removed on read. A **topic** is a named log (e.g. bkash-transactions). It is split into **partitions** for parallel work. Each message has an **offset**, its position number in the partition. **Brokers** are the Kafka servers; each partition is copied to R brokers (replication factor, default 3). Consumers track their own offset, so a late consumer can go back to offset 0 and replay everything still retained (P1 slides 14-16).

Kafka is fast because of four tricks: sequential disk writes (about 600 MB/s vs much less for random writes), zero-copy transfer with the sendfile() call, batching many messages per network trip, and partition parallelism (P1 slide 18). In the 2011 paper benchmark Kafka produced 50.5 MB/s vs 4.3 MB/s for ActiveMQ and 14.7 MB/s for RabbitMQ (P2 slide 11).

Lambda architecture (Nathan Marz, 2011) has three layers: a Batch layer (accurate, hours old), a Speed layer (approximate, seconds old) and a Serving layer that merges both for queries. Its big cost: the same business logic is written twice, once for batch and once for stream, and teams spend 30-40% of effort keeping them in sync (P1 slides 21-23).

Kappa architecture (Jay Kreps, 2014) drops the batch layer. All history sits in a long-retention log (Kafka, or Iceberg on S3). One stream job handles live data, and to reprocess history you start a

new job at offset 0 and let it catch up (P1 slide 25). It only works if the stream engine is fast enough to replay the history.

A **data lake** stores raw data of any format with no schema needed up front (ingest, storage, processing, catalogue layers). Without governance it turns into a **data swamp**: files with no owner, no metadata, stale copies and mixed personal data. Prevent it with a catalogue at ingest, owners, access control, quality checks and table formats like Delta Lake or Iceberg (P1 slides 29-30). Part 2 covers the original Kafka paper (Kreps, Narkhede & Rao 2011) and Netflix's Kafka pipeline (100 billion events per day).

Part 1 · Student Assessment Pool: MCQ

Q1. Which tool is specifically designed to transfer data in bulk between an RDBMS (e.g., MySQL) and HDFS?

(a) Apache Flume
(b) Apache Kafka
(c) Apache Sqoop
(d) Apache NiFi

ANSWER

(c) Apache Sqoop

Sqoop means "SQL-to-Hadoop". It is a command-line tool that copies data in bulk between a relational database (MySQL, Oracle, PostgreSQL, SQL Server) and HDFS, Hive or HBase. It runs a MapReduce job where each map task reads one range of rows in parallel (P1 slide 9).

- (a) Flume is for collecting *log* data into HDFS, not database tables (P1 slide 11).
- (b) Kafka is a real-time event bus for streams, not a bulk table copier.
- (d) NiFi is a visual, no-code pipeline tool for routing data (P1 slide 7).

Q2. Apache Kafka's data model is based on which fundamental storage abstraction?

(a) B-tree indexes for fast point lookups
(b) Column-family storage (like HBase)
(c) Append-only immutable distributed logs (partitioned topics)
(d) Document storage with secondary indexes

ANSWER

(c) Append-only immutable distributed logs (partitioned topics)

Kafka's founding idea: model the data infrastructure as a distributed, replicated, append-only log. That gives durability, replay and decoupling for free (P1 slide 14). A topic is a log, split into partitions; messages are only added at the end and never changed (P2 slide 5: "a distributed, partitioned, replicated commit log service").

- (a) B-trees are database indexes for finding single rows; Kafka reads sequentially by offset.
- (b) Column-family storage is HBase's model, not Kafka's.
- (d) Document storage is MongoDB-style; Kafka has no secondary indexes, only an offset index.

Q3. In the Lambda architecture, which layer handles **real-time**, **low-latency** query views?

(a) Batch Layer
(b) Serving Layer
(c) Speed Layer
(d) Ingestion Layer

ANSWER

(c) Speed Layer

The Speed layer processes data that arrived since the last batch run using a stream processor (Kafka + Spark Streaming / Flink). Its views are approximate but under a few seconds old (P1 slide 21).

- (a) The Batch layer is accurate but 1-24 hours stale (high latency).
- (b) The Serving layer does not process data; it merges batch and real-time views and answers queries (instructor note, P1 slide 33). This is the common trap.
- (d) There is no "Ingestion Layer" in Lambda. Ingestion is a data-lake layer (P1 slide 29).

Q4. The Kappa architecture differs from the Lambda architecture primarily by:

(a) Adding a third processing layer for machine learning
(b) Using only relational databases for both batch and stream storage
(c) Eliminating the batch layer and using Kafka log replay for all reprocessing
(d) Replacing Hadoop with Spark in the batch layer while keeping the speed layer

ANSWER

(c) Eliminating the batch layer and using Kafka log replay for all reprocessing

Kreps argued the batch layer is unnecessary if the stream engine can reprocess history as fast as live data. Kappa has only log storage + one stream job; to reprocess, start a new job at offset 0 (P1 slide 25).

- (a) Kappa removes a layer; it does not add one.
- (b) Kappa uses a log (Kafka/Iceberg), not relational databases.
- (d) Common misconception (P1 slide 33): Kappa does not just swap Hadoop for Spark. It removes the whole batch layer and keeps one code path.

Part 1 · Student Assessment Pool: True/False

1. Apache Sqoop is designed for real-time streaming ingestion from transactional systems.

ANSWER

False. Sqoop is a *batch* tool: it runs bulk imports/exports (for example nightly at 01:00 for bKash's 10 M rows) as MapReduce jobs (P1 slides 7, 9).

Corrected: Apache Sqoop is designed for bulk batch transfer between an RDBMS and HDFS; real-time streaming ingestion is done with Kafka.

2. Kafka retains messages for a configurable period, allowing consumers to replay events from any past offset.

ANSWER

True. Kafka keeps messages for a set time (e.g. 7 days) or up to a storage quota, even after they are read, so a consumer can reset its offset (e.g. to 0) and replay the history (P1 slide 19).

3. The Lambda architecture requires maintaining two separate code paths implementing the same business logic.

ANSWER

True. The same logic is written once for the batch layer (Spark/Hive) and once for the speed layer (Spark Streaming), and both must be changed together; teams report 30-40% of engineering effort goes into keeping them consistent (P1 slide 23).

4. Apache Flume is the preferred tool for collecting and aggregating server log data into HDFS.

ANSWER

True. Flume is built to move semi-structured logs (web server, application, IoT) into HDFS through Source, Channel and Sink, with fan-in from many servers (P1 slides 11-12). Note: in modern stacks Kafka producers often replace it (P1 slide 12), but the statement as written is the slide's answer.

Part 1 · Student Assessment Pool: Descriptive

D1. Explain the **Lambda architecture** with a labelled diagram. What are its three layers? What is its primary engineering disadvantage compared to the Kappa architecture?

ANSWER

Definition. Lambda architecture (Nathan Marz, 2011) serves both accurate historical views and low-latency real-time views from the same data, with fault tolerance and the ability to recompute from scratch (P1 slide 21).

Diagram (P1 slide 22):

```
        Raw Data (immutable event log)
           |                    |
     Batch Layer           Speed Layer
  HDFS + Spark/Hive      Kafka + Flink
           |                    |
     Batch Views          Real-time Views
 (hours old, accurate)  (seconds old, approx.)
           \                  /
        Serving Layer (merge + query)
                    |
        Application / Dashboard
```

Three layers:

1. **Batch layer:** keeps the complete, immutable master dataset on HDFS and recomputes batch views every hour or night. Accurate but 1-24 hours stale.
2. **Speed layer:** stream processor (Kafka + Spark Streaming/Flink) covers only data since the last batch run. Approximate but under seconds.
3. **Serving layer:** merges both. Query result = batch view (up to N hours ago) UNION real-time view (last N hours).

Main disadvantage vs Kappa: dual code paths. The same business logic is written twice (batch job and stream job) and must be kept in sync; 30-40% of engineering effort goes to this. It also needs two clusters and a runtime merge (P1 slide 23). Kappa has one stream code path.

Example: a bKash dashboard shows yesterday's totals from the nightly Hive job plus today's transactions from Spark Streaming. If the fee rule changes, both jobs must be edited.

D2. Describe how Apache Kafka achieves **high throughput and fault tolerance**. Explain the roles of topics, partitions, offsets, brokers, and replication factor in the Kafka data model.

ANSWER

Definition. Kafka is a distributed, partitioned, replicated commit log: every message is appended to an ordered, immutable log (P2 slide 5).

Data model (P1 slide 16):

- **Topic:** a named, durable, append-only log for one kind of event (e.g. bkash-transactions).
- **Partition:** a topic is split into P ordered partitions so work runs in parallel. Messages with the same key go to the same partition, so order per key is kept.
- **Offset:** the integer position of a message inside a partition. Each consumer tracks its own offset; replay = move the offset back.
- **Broker:** a Kafka server storing partitions on disk. A cluster has B brokers.
- **Replication factor (R):** each partition has 1 leader and R − 1 follower copies on other brokers (default R = 3).

High throughput (P1 slide 18):

1. Sequential, append-only disk writes: about 600 MB/s vs under 1 MB/s random on HDDs.
2. Zero-copy: sendfile() sends data disk to network without a user-space copy (4 copies down to 2, P2 slide 9).
3. Batching: many messages per network trip (batch.size, linger.ms).
4. Partition parallelism: P partitions allow up to P consumers in a group at once; throughput grows with partition count.

Fault tolerance: replicas on other brokers take over if a leader's broker fails (leader election by ZooKeeper or KRaft); messages are retained on disk for days so a crashed consumer resumes from its saved offset.

Example: topic bkash-txns, 3 partitions, RF = 2: each partition has a leader on one broker and a replica on the other, so losing one broker loses no data (P1 slide 17).

D3. What is a **data lake**? How does it differ from a data warehouse in terms of schema enforcement, storage format, and typical use cases? What is a “data swamp” and how is it prevented?

ANSWER

Definition. A data lake is a central store for raw data at any scale and in any format (structured, semi-structured, unstructured) with no schema needed up front. Data goes in cheaply; transformation happens when needed (P1 slide 29).

Four layers: Ingestion (Sqoop, Flume, Kafka, NiFi), Storage (HDFS or S3/ADLS in raw, processed, curated zones), Processing (Spark, Hive, Presto), Catalogue/Governance (Atlas, Glue, Hive

Metastore).

Lake vs warehouse (warehouse side is general knowledge):

Aspect	Data lake	Data warehouse
Schema	Schema-on-read: structure applied when you query	Schema-on-write: data must fit the table design before loading
Storage format	Raw files of any type (CSV, JSON, logs, images, Parquet) on cheap object storage	Cleaned, structured relational tables
Use cases	Data science, ML training, exploring new data, keeping full raw history	Fixed business reports, BI dashboards, SQL by business users

Data swamp: a lake with no governance. Symptoms (P1 slide 30): analysts cannot find datasets (no catalogue); nobody knows which files are current; many copies with different transforms and no versioning; personal data (PII) mixed with other data, breaking privacy rules.

Prevention: catalogue every dataset at ingest; give each dataset an owner; access control; automated quality checks; use a table format (Delta Lake, Iceberg) to add ACID guarantees.

Example: a hospital dumps every lab CSV into S3 with no names or owners. A year later nobody knows which file is the latest, and patient IDs leak to analysts. That is a swamp.

Part 1 · Student Assessment Pool: Analytical

A1. **Pipeline Design for a Bank:** Design a data integration pipeline for a Bangladeshi commercial bank that must:

- (a) Synchronise 10 million customer records nightly from Oracle to Hadoop for analytics
- (b) Detect fraudulent card-swipe events in real time with a decision within 200 ms
- (c) Power a risk management dashboard with data no older than 5 minutes

Specify which tool (Sqoop, Flume, Kafka, Spark Streaming) you would use at each stage. Justify each choice with reference to latency, throughput, and fault-tolerance requirements. Which architecture (Lambda or Kappa) fits best, and why?

ANSWER

Step 1: classify each need by latency. (a) nightly batch, hours are fine. (b) under 200 ms, true real time. (c) under 5 minutes, near real time.

Step 2: (a) Nightly sync: Apache Sqoop.

- Latency: nightly is fine, so batch is enough.

- Throughput: Sqoop moves GB-TB/hour (P1 slide 7); it splits the table by primary-key range across parallel map tasks. With the usual 4 mappers, 10,000,000 / 4 = 2,500,000 rows each.
- Use incremental import (--incremental with a last-value on a timestamp column) so only changed rows move.
- Fault tolerance: it runs as a MapReduce job, so a failed map task is retried; output lands in replicated HDFS as Parquet partitioned by date. Same pattern as bKash's 10 M rows at 01:00 (P1 slide 9).
- Flume is not used: it is for logs, not database tables.

Step 3: (b) Fraud detection: Kafka + Spark Streaming.

- Card terminals publish each swipe as an event to a Kafka topic (e.g. card-swipes), keyed by card number so one card's swipes stay in order in one partition.
- Spark Streaming reads the topic, scores each event, and writes an approve/block decision. The slide gives bKash's fraud detector at under 200 ms with this exact stack (P1 slide 19).
- Throughput: Kafka handles GB-TB/hour; add partitions to scale consumers.
- Fault tolerance: RF = 3 replicas; events are retained, so if the fraud job crashes it restarts from its last offset and loses nothing.

Step 4: (c) Risk dashboard: Kafka → Spark Streaming aggregates → serving store. A second consumer group on the same topic computes rolling risk totals every minute and writes them to a fast store the dashboard reads. 5 minutes is easy for a stream job. Joining in the nightly Sqoop data (customer profiles in HDFS) gives context.

Step 5: architecture: Lambda fits best.

- The bank already has a true batch need (nightly Oracle sync of 10 M records for analytics) and regulatory batch reports. The slide lists "regulatory batch reports" and "regulatory requirement mandates a separate batch reconciliation" as reasons to choose Lambda and avoid Kappa (P1 slide 27).
- Batch layer = Sqoop + HDFS + Hive/Spark; Speed layer = Kafka + Spark Streaming (fraud, dashboard); Serving layer = dashboard merging both.
- Cost: two code paths. Reduce it, like Netflix, by making Kafka the single source both layers read from (P2 slide 18).
- Kappa could work only if the stream engine can replay years of history fast enough and regulators accept no separate batch reconciliation.

A2. Lambda to Kappa Migration: A team using Lambda architecture complains that maintaining two code paths (batch + stream) doubles development cost. Their CTO suggests migrating to Kappa. Analyse: (a) what must be true about the stream processing engine for Kappa to be viable; (b) what are the risks of migrating; (c) under what conditions is Kappa **not suitable** (give at least two concrete scenarios from the Bangladesh financial or healthcare sector)?

ANSWER

Background. Lambda's cost is real: 30-40% of effort keeps batch and stream logic in sync (P1 slide 23). Kappa removes the batch layer and keeps one stream code path with log replay (P1 slide 25).

(a) Conditions for Kappa to work

1. The stream engine (Flink, Spark Structured Streaming) can reprocess historical data about as fast as it handles live data (Kreps' condition, P1 slide 25).
2. The full history is kept in a replayable log: Kafka with long retention or Iceberg on S3 (default Kafka retention is only 7 days).
3. The business logic can be expressed as a stream job (windows, state, joins).
4. The engine supports exactly-once processing and stateful replay so replayed results match (Kafka 0.11+ exactly-once, P2 slide 12).

(b) Risks of migrating

- Replay speed: reprocessing TB+ of history through the stream engine may take days (P1 slide 27).
- Batch throughput drops: Kappa is limited by stream-engine throughput vs very high Spark/Hive batch throughput.
- Storage cost: keeping years of history in Kafka/S3 for replay.
- Rewrite effort and bugs: all batch SQL/Hive jobs must be rewritten as stream jobs; results may differ during transition, so both systems run in parallel for a while.
- Team skills: analysts used to batch SQL must learn stream SQL.

(c) When Kappa is NOT suitable (slide's three cases, P1 slide 27, with local scenarios):

1. **Regulatory batch reconciliation:** Bangladesh Bank requires a commercial bank to produce a daily end-of-day reconciliation and audit from a separate batch system. bKash already writes an audit log to HDFS for daily reconciliation (P1 slide 19). Kappa cannot drop that batch layer.
2. **TB+ history too slow to replay:** a national health-records system (e.g. DGHS data on millions of patients over 10 years) needs to recompute disease statistics after a coding change. Replaying all that through a stream engine is far slower than one Spark batch job.
3. **Batch SQL analytics preferred / complex logic:** a bank's yearly credit-risk ML model training and Basel capital reports are written in Hive/Spark SQL over full history; stream SQL fits poorly.

Conclusion: move streaming-friendly workloads (fraud, dashboards) to a single Kappa-style path, but keep a batch path where regulation or huge history demands it. Netflix did this gradually (P2 slide 18).

Part 2 · Discussion Questions

L1. **Kafka as the single source of truth**: all events written to Kafka first; the speed and batch layers are both consumers of the same topic — eliminating the data inconsistency problem of classic Lambda (two separate ingestion paths)

ANSWER

Meaning. In classic Lambda, data can enter the batch and speed layers by different routes, so the two layers may see different data. Netflix writes every event to Kafka once; Flink (speed, under 1 minute) and the S3 sink + Spark/Iceberg (batch, about 1 day) both read the same topic (P2 slide 15).

For: one copy of the truth, no mismatch between layers, easy to add consumers. **Against:** Kafka becomes a critical dependency; if it is down, everything is down, so it needs replication (2,000+ brokers at Netflix, P2 slide 17). The two code paths still exist; only the ingestion is unified.

L2. **Retention enables reprocessing**: 7-day retention lets Netflix replay any event window when a Flink job has a bug or a Spark job fails

ANSWER

Meaning. Kafka does not delete a message when it is read. With 7-day retention, a fixed job can reset its offset and recompute the last week (P2 slide 18).

For: bugs can be fixed after the fact without asking devices to resend. **Against:** only 7 days. A bug found on day 8 cannot be fixed from Kafka; the S3/Iceberg copy is needed. Longer retention costs storage (at 100 billion events/day, 7 days is about 700 billion events). Tiered storage (Kafka 3.6) moves old segments to S3 to cut this cost (P2 slide 12).

L3. **Kappa simplification**: Netflix increasingly moved batch jobs to Flink stream processing (processing historical S3 data as a bounded stream), reducing pipeline complexity

ANSWER

Meaning. A "bounded stream" is a stream with a start and an end, for example last month's files in S3. Flink can read it with the same code it uses for live data, so one code path covers both (Kappa idea, P1 slide 25).

For: less duplicate code, fewer systems. **Against:** Netflix still keeps Spark + Iceberg for ML training and billing (P2 slide 15), because some batch work is easier or cheaper in batch. The move is

gradual, not all at once.

L4. **Schema Registry is non-negotiable at scale**: 4,000 topics × evolving schemas ⇒ consumer breakage without schema governance

ANSWER

Meaning. A schema is the structure of a message (field names and types). If a producer renames a field, every consumer reading it may crash. Schema Registry stores the Avro/Protobuf schema per topic and lets a consumer read messages from any past producer version (P2 slide 12).

For: with 4,000+ topics and 150+ Flink jobs, manual coordination is impossible. **Against:** one more service to run, and teams must follow compatibility rules. This is the streaming version of the data-swamp lesson: without governance, data becomes unusable (P1 slide 30).

L5. **Backpressure is the real production problem**: during Super Bowl commercials, event rates spike 5×; Kafka's durable log absorbs the burst; Flink consumers catch up within minutes

ANSWER

Meaning. Backpressure is when data arrives faster than consumers can process it. A 5× spike on the 1.3 million events/second peak would be 6.5 million/second. Kafka just appends the extra events to disk; Flink reads them a little later and catches up (P2 slides 17-18).

For: no data lost, no need to size consumers for the rare peak. **Against:** during the spike results are delayed by minutes (the 5-minute A/B result latency may slip); brokers need disk space for the burst. Analogy: a water tank between a burst pipe and a small tap.

Part 2 · Live Exercise

Task. Simulate a Kafka producer/consumer pipeline using the **kafka-python** library with a local single-broker cluster (Docker Compose provided below).
Quick-start: `pip install kafka-python`

```
import json, random, time
from kafka import KafkaProducer, KafkaConsumer

TOPIC   = "play-events"
BROKER  = "localhost:9092"
CONTENT = [f"show_{i}" for i in range(20)]

# --- Producer ---
producer = KafkaProducer(
  bootstrap_servers=BROKER,
  value_serializer=lambda v:
      json.dumps(v).encode())

for _ in range(1_000):
    event = {
      "user_id":    random.randint(1, 500),
      "content_id": random.choice(CONTENT),
      "action":     random.choice(
                      ["play","pause","seek"]),
      "ts":         int(time.time())
    }
    producer.send(TOPIC, event)
producer.flush()
print("Produced 1,000 events")
```

ANSWER

What the skeleton does. It connects a producer to the broker at localhost:9092, turns each Python dict into JSON bytes (value_serializer), and sends 1,000 random events. Each event has a user (1-500), one of 20 shows (show_0 to show_19), an action (play, pause or seek) and a Unix timestamp. flush() waits until all buffered messages are actually sent, because send() only queues them (batching, P1 slide 18).

Step 1: start a broker. The Docker Compose file is cut off on the slide. A minimal single-broker setup that works the same: `docker run -d --name kafka -p 9092:9092 apache/kafka:3.7.0` (KRaft mode, no ZooKeeper). The topic play-events is auto-created on first send with the broker's default partition count (1 unless configured).

Step 2: `pip install kafka-python`, save the skeleton as producer.py, run `python3 producer.py`. Expected output: `Produced 1,000 events`.

Step 3: the consumer (the slide gives no consumer code; this completes it):

```
from collections import Counter
from kafka import KafkaConsumer, TopicPartition
import json

TOPIC  = "play-events"
BROKER = "localhost:9092"

def make_consumer(group):
    return KafkaConsumer(
        bootstrap_servers=BROKER,
        group_id=group,
        auto_offset_reset="earliest",   # new group starts at offset 0
        enable_auto_commit=True,
        consumer_timeout_ms=5000,       # stop after 5 s with no new messages
        value_deserializer=lambda b: json.loads(b.decode()))

def count(consumer):
    total, all_events, plays = 0, Counter(), Counter()
    for msg in consumer:
        e = msg.value
        total += 1
        all_events[e["content_id"]] += 1
        if e["action"] == "play":
            plays[e["content_id"]] += 1
    return total, all_events, plays

# Goal 2: read everything, count per content_id
c = make_consumer("play-counter")
c.subscribe([TOPIC])
total1, ev1, plays1 = count(c)
print("read", total1, "events")
print("plays per content_id:", sorted(plays1.items()))
c.close()

# Goal 3: reset the SAME group to the beginning and re-consume
c = make_consumer("play-counter")
parts = [TopicPartition(TOPIC, p) for p in c.partitions_for_topic(TOPIC)]
c.assign(parts)
c.seek_to_beginning(*parts)
total2, ev2, plays2 = count(c)
print("re-read", total2, "events; same counts?", plays1 == plays2)
c.close()
```

```
# Goal 4: a NEW group id has no saved offset, so it starts at offset 0
c = make_consumer("play-counter-v2")
c.subscribe([TOPIC])
total3, ev3, plays3 = count(c)
print("new group read", total3, "events")
c.close()
```

Save as consumer.py and run after the producer. Expected outputs are explained in Goals 2-4 below.

1. Run the producer — send 1,000 synthetic Netflix play events to topic `play-events`.

ANSWER

Run `python3 producer.py`. Expected: it prints `Produced 1,000 events`. The topic now holds 1,000 messages at offsets 0 to 999 (if one partition). Running the producer a second time appends 1,000 more (offsets 1,000-1,999); nothing is overwritten because the log is append-only (P1 slide 14).

2. Run the consumer — read all messages, count plays per `content_id`.

ANSWER

Run consumer.py. Expected: `read 1000 events`, then a count per show.

- Each event picks one of 20 shows at random, so each show gets about 1,000 / 20 = 50 events in total (typical spread about ±7).
- Only about 1 in 3 events has action "play", so about 1,000 / 3 ≈ 333 plays in total, about 16.7 plays per show (typical spread about ±4).
- Exact numbers change every run because the producer uses random.

"Count plays per content_id" can mean all events per show or only action = play; the code prints the play-only count and also keeps the all-events count.

3. Reset the consumer offset to the *beginning* and re-consume — verify idempotent results.

ANSWER

The code assigns the partitions and calls `seek_to_beginning`, so the same group goes back to offset 0. Expected: `re-read 1000 events; same counts? True`.

Why: reading from Kafka does not delete messages (unlike a traditional queue, P1 slide 15). The log still holds the same 1,000 events, so recounting gives exactly the same result. "Idempotent" means doing it again gives the same answer. This is replay, the basis of Kappa reprocessing.

Trap: if the producer was run again in between, the log has 2,000 events and counts will differ. That is new data, not a broken replay.

4. Change the consumer group ID and observe that a new group starts from offset 0.

ANSWER

Group "play-counter-v2" has no committed offset stored in Kafka (offsets live in the __consumer_offsets topic per group, P2 slide 7). With `auto_offset_reset="earliest"` it starts at offset 0 and reads all 1,000 events again. Expected: `new group read 1000 events`. The old group's position is untouched: each group tracks its own offset independently.

Trap: kafka-python's default is `auto_offset_reset="latest"`. With the default, a new group starts at the end of the log and reads 0 events. So you must set "earliest" to see the offset-0 behaviour.

Why it matters: this is how bKash feeds fraud detection, balance update and audit from one topic (P1 slide 19): each is its own group, each reads every event.

Maths for Week 8

This chapter has no heavy formulas, but the slides use several numbers that can be turned into one-step calculations: point-to-point connection counts, disk speed ratios, zero-copy savings, message throughput, consumer-group sharing, replica counts, event rates and storage, and Sqoop parallel splits. Each is shown with the slide's numbers.

Point-to-point ETL connections vs a message bus

```
Connections (point-to-point) = N × (N - 1)   |   Connections (bus) = N
```

WHAT EACH SYMBOL MEANS

- **N** = number of systems that must exchange data.
- N × (N − 1) counts every ordered pair (A to B and B to A are separate ETL jobs). If you count each pair once, divide by 2.
- With a bus each system connects only once, to the bus.

WORKED EXAMPLE

Slide: 50 systems (P1 slide 5).

50 × (50 − 1) = 50 × 49 = **2,450** ETL jobs, matching the slide. Counted once per pair: 2,450 / 2 = 1,225.

With Kafka as a bus: 50 connections.

PRACTICE

A company has 10 systems. Point-to-point: 10 × 9 = **90** jobs (45 if one per pair). With a bus: **10**. For 20 systems: 20 × 19 = 380.

Sequential vs random disk speed

```
Advantage = sequential speed ÷ random speed
```

WHAT EACH SYMBOL MEANS

- **Sequential speed** = MB/s when writing one block after another (append-only).
- **Random speed** = MB/s when jumping around the disk.

WORKED EXAMPLE

P2 slide 11: sequential ≈ 600 MB/s, random ≈ 100 MB/s. 600 / 100 = **6×** advantage.

P1 slide 18 uses a harsher random figure (< 1 MB/s): 600 / 1 = more than **600×**. Quote whichever slide the question uses.

PRACTICE

A disk writes 200 MB/s sequentially and 2 MB/s randomly. Advantage = 200 / 2 = **100×**.

Zero-copy: data copies per transfer

```
Speedup ≈ copies (traditional) ÷ copies (zero-copy)
```

WHAT EACH SYMBOL MEANS

- **Traditional path:** disk → kernel buffer → user space → socket buffer → NIC = 4 copies.
- **sendfile() path:** disk → kernel buffer → NIC = 2 copies.
- **NIC** = network interface card.

WORKED EXAMPLE

P2 slide 9: 4 / 2 = **2×** throughput, with CPU near zero. Copies saved per message = 4 − 2 = 2.

PRACTICE

A broker sends 1,000,000 messages. Copies saved = 1,000,000 × 2 = **2,000,000** memory copies avoided.

Throughput: MB/s to messages per second

```
Messages/sec = throughput (bytes/s) ÷ message size (bytes)   |
   Speedup = Kafka ÷ other
```

WHAT EACH SYMBOL MEANS

- **Throughput** in MB/s, with 1 MB = 1,000,000 bytes (the slide's table uses this).
- **Message size** = 100 bytes in the paper's benchmark.

WORKED EXAMPLE

P2 slide 11, producer: 50.5 MB/s ÷ 100 B = 50,500,000 / 100 = **505,000 msgs/s** (matches the table). ActiveMQ: 4.3 MB/s → 43,000. Kafka vs ActiveMQ = 50.5 / 4.3 ≈ **11.7×**. Kafka vs RabbitMQ = 50.5 / 14.7 ≈ **3.4×**.

PRACTICE

Consumer side: Kafka 22.0 MB/s ÷ 100 B = **220,000 msgs/s**. Kafka vs ActiveMQ consumer = 22.0 / 4.0 = **5.5×**; vs RabbitMQ = 22.0 / 8.2 ≈ **2.7×**. If messages were 1,000 B at 50.5 MB/s: 50,500 msgs/s.

Partitions and consumer groups

```
Active consumers = min(P, C)   |   Idle consumers = C - P (if
C > P)   |   Partitions per consumer ≈ P ÷ C
```

WHAT EACH SYMBOL MEANS

- **P** = partitions in the topic.
- **C** = consumers in one consumer group.
- Rule: each partition goes to exactly one consumer in the group (P1 slide 16), so extra consumers sit idle.

WORKED EXAMPLE

Pathao driver-location: P = 10 (P1 slide 19). Group of 4: 10 / 4 = 2.5, so two consumers get 3 partitions and two get 2 (3+3+2+2 = 10). Group of 12: min(10, 12) = 10 active, 12 − 10 = **2 idle**.

PRACTICE

page-views has 3 partitions (P2 slide 8). Group of 3: one each. Group of 5: 3 active, 2 idle. Topic with 6 partitions, 4 consumers: 6 / 4 = 1.5, so two get 2, two get 1.

Replication: copies and failures survived

```
Total partition copies = P × R   |   Followers per partition = R - 1   |   Broker failures survived = R - 1
```

WHAT EACH SYMBOL MEANS

- **P** = partitions, **R** = replication factor (default 3, P1 slide 16).
- One copy is the leader; the other R − 1 are followers.

WORKED EXAMPLE

bkash-txns: P = 3, RF = 2 (P1 slide 17). Copies = 3 × 2 = **6**. Followers per partition = 1. Survives **1** broker failure.

PRACTICE

Pathao driver-location with P = 10 and default R = 3: copies = 10 × 3 = **30**, followers = 10 × 2 = 20, survives 2 broker failures.

Event rate and retention storage

```
Events/sec = events per day ÷ 86,400   |   Retained events = events per day × retention days   |   Storage = retained events × message size × R
```

WHAT EACH SYMBOL MEANS

- 86,400 = seconds in a day.
- **Retention days** = how long Kafka keeps messages (7 days at Netflix).
- **R** = replication factor (each copy uses disk).

WORKED EXAMPLE

Netflix: 100 billion events/day (P2 slide 17). Average = 100,000,000,000 / 86,400 ≈ **1,157,407 events/s**. The peak of 1.3 million/s is about 1.12× the average. A Super Bowl 5× spike on the peak = 1.3 M × 5 = **6.5 million/s**. With 7-day retention Kafka holds 100 B × 7 = **700 billion events**.

PRACTICE

Pathao: assume 10,000 active drivers (the slide gives no driver count); one GPS ping every 3 s (P1 slide 19). Rate = 10,000 / 3 ≈ **3,333 msgs/s**. Per day = 3,333.3 × 86,400 = **288,000,000**. Over 7 days = 2,016,000,000. At 200 bytes each and R = 3: 2,016,000,000 × 200 × 3 = 1,209,600,000,000 bytes ≈ **1.21 TB**.

Sqoop parallel import split

```
Rows per mapper = total rows ÷ m
```

WHAT EACH SYMBOL MEANS

- **m** = number of parallel map tasks (Sqoop's -m option; 4 by default, general Sqoop knowledge).
- Sqoop splits rows by primary-key range so each mapper reads its own slice (P1 slide 9).

WORKED EXAMPLE

bKash: 10 M rows nightly (P1 slide 9). With m = 4: 10,000,000 / 4 = **2,500,000 rows** per mapper.

PRACTICE

Same table with m = 8: 10,000,000 / 8 = **1,250,000 rows** per mapper.

Live exercise: expected counts per show

```
Expected events per show = N ÷ S   |   Expected plays per show = N ÷ (S × A)
```

WHAT EACH SYMBOL MEANS

- **N** = events produced (1,000).
- **S** = number of shows (20).
- **A** = number of actions (3: play, pause, seek), each equally likely.

WORKED EXAMPLE

1,000 / 20 = **50** events per show. Plays: 1,000 / (20 × 3) ≈ **16.7** per show; total plays ≈ 1,000 / 3 ≈ 333. Random, so real counts vary (about ±7 and ±4).

PRACTICE

5,000 events, 50 shows, 3 actions: 5,000 / 50 = **100** events per show; 5,000 / 150 ≈ **33.3** plays per show.

Lambda maintenance overhead

```
Engineers on sync work = team size × overhead share
```

WHAT EACH SYMBOL MEANS

- **Overhead share** = 30% to 40% of effort spent keeping the two Lambda layers consistent (P1 slide 23).

WORKED EXAMPLE

A 10-engineer team: 10 × 0.30 = **3** to 10 × 0.40 = **4** engineers' worth of effort on keeping code paths in sync.

PRACTICE

A 25-engineer team: 25 × 0.30 = **7.5** to 25 × 0.40 = **10** engineers.

Chapter 5 · Week 9: Data Visualization: Principles, Techniques & Big Data Tools

The chapter in plain words

Data visualization means turning numbers into pictures (charts, maps, graphs) so a person can see patterns quickly. In big data it is the first quality check: before you trust any average, you plot the data. Anscombe's Quartet (1973) is the classic proof: four datasets share the same mean of x (9.0), mean of y (7.5), variance of x (11.0), correlation (r = 0.816) and fitted line (y = 3 + 0.5x), yet one is a straight line, one a curve, one has an outlier and one is a vertical cluster (P1 slide 5).

At big data scale, averages hide even more. A 10-billion-row table can mix groups with opposite trends, skewed data (income, web traffic) makes the mean meaningless, and a daily average can hide a sudden change. Example from the slides: DGHS monthly disease averages look stable, but a line chart shows a cholera spike every June to August and a dengue peak every October (P1 slide 6).

Edward Tufte (1983) gave the design rules. **Data-ink ratio** = ink that shows data ÷ total ink in the chart; make it as high as possible. Anything that does not carry data is **chartjunk**: 3-D effects, heavy gridlines, gradient fills, redundant legends, decorative borders. Rule of thumb: if erasing an element loses no information, erase it (P1 slide 8). He also gave **small multiples** (the same small chart repeated for each district or year, on the same scale), **sparklines** (tiny word-sized trend lines next to a number) and the **lie factor** (how much a graphic exaggerates the real change) (P2 slides 6-7).

Ben Shneiderman (1996) gave the interaction rule, called the **mantra**: "Overview first, zoom and filter, then details on demand." Think of Google Maps: you see the whole country, zoom into a city, then click a shop for its details. He also listed seven data types (1-D, 2-D, 3-D, temporal, multi-dimensional, tree, network) and the tasks a tool must support: overview, zoom, filter, details-on-demand, relate, history, extract (P1 slides 11-12, P2 slide 9). He invented the **treemap** (1991) and **dynamic queries** (sliders that update a chart in under 100 ms) (P2 slide 10).

Choosing a chart depends on the question: line chart for change over time, horizontal bar chart for which category is largest, choropleth map for how a rate spreads across regions, histogram or box plot for the spread of one variable, scatterplot for whether X predicts Y, network graph for connections. The pie chart is the most abused chart: use it only for 4 or fewer categories when the question is "what fraction of the whole?" (P1 slides 14-15).

Big data adds three special problems (P1 slides 17-20). **Overplotting**: millions of dots pile into a solid blob; fix with transparency, hexbin counts, or density contours (KDE). **Latency**: a filter on a 100-billion-row table takes minutes; fix with a pre-computed **data cube** (summary tables) or approximate answers from samples (AQP). **Streaming**: charts must update live; the pipeline is

Kafka → Spark Structured Streaming/Flink window aggregation → time-series database → Grafana/Kibana (polling every few seconds), or WebSocket push to a D3.js chart for sub-second updates.

Tools (P1 slides 22-23): Tableau and Power BI for business users; D3.js for custom interactive web charts (it binds data to SVG elements in the browser); Grafana for real-time operations monitoring; Apache Superset for open-source BI on Hadoop/Spark data lakes; Kibana for log search over Elasticsearch.

The Part 2 case study is the **New York Times Graphics Desk**, where Mike Bostock built D3.js. Their COVID-19 tracker (2020-2022) used Tufte's small multiples (a 50-state grid) and Shneiderman's mantra (national total, then state selector, then hover tooltip). The main lesson: the hardest work was cleaning messy data from 50 state health departments, not drawing the charts (P2 slides 13-15, 19).

Part 1 · Student Assessment Pool: MCQ

Q1. Edward Tufte's “data-ink ratio” principle states that a visualization should:

(a) Use more colours to highlight different categories of data
(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements
(c) Use ink sparingly to reduce printing costs
(d) Always include data labels using a different ink colour from the chart body

ANSWER

Answer: (b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements.

Why: Tufte defines data-ink ratio = ink encoding data ÷ total ink in the chart, and the goal is to make it as high as possible by erasing chartjunk (3-D effects, gradient fills, redundant legends, heavy gridlines) (P1 slide 8). It is about *what* the ink shows, not how much ink there is.

- (a) Wrong: more colours usually add non-data ink; the principle says nothing about adding colour.
- (c) Wrong: this is the classic trap. The ratio is about what the ink encodes, not saving ink or printing cost. One thick bar showing one important value has a high data-ink ratio (P1 slide 26 instructor notes).
- (d) Wrong: a different ink colour for labels is a styling choice and does not raise the share of ink that carries data.

Q2. Which visualization type is most appropriate for showing the distribution and spread of a continuous variable?

(a) Pie chart
(b) Line chart
(c) Histogram
(d) Treemap

ANSWER

Answer: (c) Histogram.

Why: a histogram cuts the number line into bins and draws a bar for how many values fall in each bin, so you see the shape and spread (where values bunch up, how wide, any skew). The slide's chart guide lists histogram for "What is the spread and shape of X?" (P1 slide 14).

- (a) Pie chart: shows part-to-whole, not spread.
- (b) Line chart: shows change over time; the slide lists it under "Avoid" for distributions.
- (d) Treemap: shows hierarchical proportions (nested rectangles) (P1 slide 26 instructor notes).

Q3. Shneiderman's visualization mantra begins with which step?

(a) Details first, then overview
(b) Overview first, zoom and filter, then details on demand
(c) Filter first, then provide an overview
(d) Details on demand first, then zoom out

ANSWER

Answer: (b) Overview first, zoom and filter, then details on demand.

Why: the mantra (Shneiderman, 1996) says the user first sees the whole picture, then narrows to a region or subset, then pulls up the raw record for one item (P1 slide 11). Analogy: Google Maps shows the country, you zoom into your city, then click one shop.

- (a) Wrong: puts details before the overview, the reverse of the mantra.
- (c) Wrong: filtering only makes sense after the overview shows you what to filter (P1 slide 27 instructor notes).
- (d) Wrong: also reversed; details-on-demand is the last step.

Q4. Which visualization tool is best suited for a real-time production operations dashboard monitoring server metrics and application error rates with 5-second refresh intervals?

(a) Tableau
(b) D3.js
(c) Power BI
(d) Grafana

ANSWER

Answer: (d) Grafana.

Why: Grafana is built for real-time metrics and ops dashboards. It reads directly from time-series databases (InfluxDB, Prometheus, Elasticsearch) and auto-refreshes every 5 s (P1 slides 20, 22). The decision rule on P1 slide 23: "real-time ops monitoring → Grafana".

- (a) Tableau: business dashboards; only limited scheduled refresh, not second-level streaming.
- (b) D3.js: can do real time with WebSockets, but you must build everything yourself (custom development).
- (c) Power BI: Microsoft executive reports; real-time support is limited (P1 slide 27 instructor notes).

Part 1 · Student Assessment Pool: True/False

1. A pie chart is a recommended choice for comparing more than 5 categories.

ANSWER

False.

Why: people judge angles and arc lengths much less accurately than bar lengths, and it gets worse as slices increase. The slide says use a pie only for 4 or fewer categories and only for "what fraction of the whole?" (P1 slide 15).

Corrected: For more than 5 categories, a horizontal bar chart ordered by value is the recommended choice (P1 slides 14, 28).

2. Hexbin plots are an effective technique for reducing overplotting when visualising millions of data points.

ANSWER

True.

Why: a hexbin divides the plot into a grid of hexagons, counts the points in each, and colours each hexagon by its count, so 10 million points become about 10,000 readable cells that show where density is high (P1 slide 17).

3. D3.js generates static PNG or PDF image files as its primary output format.

ANSWER

False.

Why: D3 (Data-Driven Documents) binds data to SVG elements in the web page (DOM) inside a browser, so the output is interactive, scalable vector graphics that update dynamically (P1 slides 23, 28).

Corrected: D3.js produces interactive SVG (or Canvas) graphics in the browser, not static image files.

4. Choropleth maps are used to visualise the geographic distribution of a quantitative variable.

ANSWER

True.

Why: a choropleth shades each region (district, division, country) by colour intensity according to a number, for example dengue incidence per 100,000 people per district (P1 slides 14, 28). It works best for rates or proportions; for absolute counts the slide recommends a proportional bubble map.

Part 1 · Student Assessment Pool: Descriptive

D1. Explain Tufte's data-ink ratio principle. Provide a “before and after” example: describe a cluttered bar chart (what chart-junk elements are present) and then describe the clean version after applying the principle.

ANSWER

Definition. Data-ink is the ink in a chart that actually encodes data (bar lengths, data points, lines). Tufte (1983) defined:

```
Data-ink ratio = ink used to display data ÷ total ink used in the graphic
```

The goal is to make this ratio as high as possible. Any ink that carries no data is **chartjunk** and must justify its existence. Test: if erasing an element does not reduce the information, erase it (P1 slide 8; P2 slide 6).

Before (cluttered quarterly sales bar chart).

- Title in large capitals with exclamation marks ("CHART TITLE!!!").
- A legend for a single series (redundant: the axis already names the bars).
- 3-D bars with shadows and perspective, which distort the heights.
- Gradient fills and a decorative border.
- Heavy dark gridlines competing with the bars.

After (clean version).

- Flat 2-D bars in one colour, y-axis starting at 0.
- No legend; quarters (Q1-Q4) labelled directly on the x-axis.
- No gridlines, or one light reference line; light thin axis.
- Plain short title; value labels (1k-4k) only where needed.

The data shown is identical, but nearly all remaining ink is data, so the reader sees the trend at once (P1 slide 8 before/after figure). Tufte's own summary: "Above all else, show the data."

D2. What is overplotting? Describe three techniques to address it when visualising millions of data points in a scatterplot. For each technique, explain what information it preserves that the original overplotted chart lost.

ANSWER

Definition. A scatterplot draws one dot per record. With millions of records the dots cover each other completely, so the chart becomes a solid blob that says nothing except "data exists here". This is **overplotting** (P1 slide 17). Analogy: a photo of a packed stadium from far away; you cannot tell where the crowd is thickest.

Three techniques (in increasing sophistication):

1. **Alpha transparency.** Make each dot almost see-through, e.g. opacity α = 0.02. Where many dots stack, the area looks darker. *Preserves:* relative density (dark = many points, light = few). *Limit:* at very high densities everything turns fully dark again.
2. **Hexbin aggregation.** Divide the plot into a hexagonal grid, count points in each hexagon, colour each hexagon by its count. 10 million points become about 10,000 cells, which is fast and interactive. *Preserves:* the actual count per area, so you can read exact density, which the blob hid.
3. **Kernel Density Estimation (KDE) / 2-D density contour.** Estimate a smooth probability density over the point cloud and draw contour lines (like height lines on a map) at probability levels. *Preserves:* the shape of the distribution, its modes (peaks, clusters) and its tails.

Example. Plotting 10 million bKash transactions (amount vs time of day): the raw scatter is one blob; a hexbin shows the busiest amount/time cells; KDE contours show two peaks, for example a lunchtime and an evening cluster (P1 slides 17-18).

D3. What is a data cube? How does pre-aggregating a billion-row fact table into a data cube enable interactive visualization at sub-second latency? What is the trade-off of this approach?

ANSWER

Definition. A **fact table** holds one row per event (each sale, each transaction). A **data cube** is a set of pre-computed summary tables (aggregates such as SUM, COUNT) for every combination of dimensions, for example district × date × product × channel (P1 slide 19). Analogy: instead of counting every receipt each time the boss asks, you keep a ready-made tally sheet per shop per day.

How it gives sub-second latency:

1. A scheduled Spark/Hive job (e.g. nightly) scans the huge fact table once and writes the cube.
2. The cube has millions of rows, not billions.
3. When the analyst moves a Tableau/Superset filter ("show Q1 2024 only"), the query reads the small cube, not the raw table.
4. Reading millions of pre-summed rows takes under 1 second, while a Hive query over 100 billion raw rows takes minutes.

Trade-offs:

- **Staleness:** the cube is only as fresh as the last job run, up to 24 hours old for a nightly job.
- **Fixed questions:** you can only slice by dimensions you pre-computed; a new dimension needs a rebuild.
- **Storage and compute:** materialising all combinations costs extra space and batch time.

Alternative: Approximate Query Processing (AQP, e.g. Apache DataSketches, BlinkDB) samples the table and returns a statistically sound estimate in milliseconds. It trades accuracy for speed: fine for dashboards, not for financial reconciliation (P1 slide 19).

Example: 100 billion rows reduced to a 5-million-row cube is 20,000 times fewer rows to read.

Part 1 · Student Assessment Pool: Analytical

A1. Public Health Dashboard Design: A DGHS public health dashboard needs to communicate: (a) COVID-19 confirmed case counts by district over 24 months (b) Real-time hospitalisation rate updated every 5 minutes (c) Demographic breakdown (age group, gender) of confirmed cases For each sub-requirement: specify the appropriate visualization type, justify using Shneiderman's data-type taxonomy, identify which big data visualization challenge applies (if any), and name the tool you would use. Apply Tufte's data-ink principle to state one specific chart-junk element to eliminate.

ANSWER

Step 1: classify each requirement by Shneiderman's data type (P1 slide 11).

Req.	Data type	Chart	Challenge	Tool
(a) Cases by district, 24 months	Temporal (plus 2-D map for the district view)	Small multiples of line charts, one per district, same y-scale; optional choropleth of case rate per 100,000 for the latest month	Latency: raw case records are large, so pre-aggregate to daily counts per district (data cube)	Tableau or Power BI (Superset if the data sits in Hive/Spark)
(b) Hospitalisation rate every 5 min	Temporal (live)	Single-number stat panel with a sparkline, plus a line chart of the last 24 h	Streaming: values arrive continuously	Grafana on a time-series DB (InfluxDB/Prometheus)
(c) Age group × gender	Categorical / multi-dimensional	Grouped horizontal bar chart (or back-to-back population pyramid)	None significant: the result is a small summary table	Tableau or Power BI

Step 2: justify.

- (a) Time series → line chart is the slide's recommendation for "How did X change over time?"; pie charts are listed under Avoid (P1 slide 14). Small multiples on the same scale let the eye compare districts directly, the way the NYT 50-state grid did (P2 slide 14). Size check: 24 months ≈ 730 days × 64 districts = 46,720 daily rows after aggregation, small enough for sub-second filtering.
- (b) 5-minute updates = 288 points per day. The slide's streaming pipeline is hospital reports → Kafka → Spark Structured Streaming window aggregation → TSDB → Grafana polling (P1 slide 20). A 5-minute window is slow by streaming standards, so Grafana polling is enough; WebSocket push is not needed.
- (c) Comparing categories → bar chart; horizontal bars when labels are long or there are more than 5 categories (P1 slide 14). A 3-D pie of age groups is the chart to avoid.

Step 3: Shneiderman's mantra on the whole dashboard. Overview = national totals at the top; zoom/filter = district selector and date-range slider (dynamic queries); details-on-demand = hover tooltip with exact date, count and 7-day change.

Step 4: Tufte chartjunk to eliminate. Remove 3-D effects from the demographic bars (depth and perspective distort comparison; "never use 3-D for 2-D data", P2 slide 6). Also drop heavy gridlines

on the line charts; keep at most one light reference line at the previous peak, as the NYT did.

A2. Streaming Sentiment Visualization: Design a complete visualization system for a streaming Twitter (X) sentiment analysis pipeline processing 10,000 tweets per second. Specify: (a) which Kafka aggregation window you would use and why; (b) which chart types best convey sentiment drift over time and topic clustering; (c) how you would handle the velocity challenge at the visualization layer without overloading the browser; (d) which tools form the pipeline from Kafka to browser render.

ANSWER

Step 1: size the stream. 10,000 tweets/s × 60 s = 600,000 tweets per minute; × 86,400 s = 864,000,000 tweets per day. No browser can draw that many points, so aggregation is required.

(a) Window. Use a **1-minute tumbling window** (fixed, non-overlapping blocks of time; each tweet counted once), the window size used in the slide's streaming architecture (P1 slide 20). The stream processor reading from Kafka (Spark Structured Streaming or Flink) computes, per topic per minute: tweet count, mean sentiment score, share of positive/neutral/negative. Why 1 minute: it turns 600,000 tweets into one point per topic, it smooths second-to-second noise, and it is still fast enough to see drift. Option: add a 15-minute sliding window (overlapping, recomputed each minute) as a smoother trend line; trade-off: smoother but reacts later.

(b) Charts.

- **Sentiment drift over time** (temporal data): line chart of mean sentiment per minute, or a 100% stacked area chart of positive/neutral/negative share; small multiples, one panel per topic on the same scale (Tufte).
- **Topic clustering** (high-dimensional data, e.g. tweet embeddings): t-SNE/UMAP scatterplot (P1 slide 14). Because there are millions of points, draw it as a **hexbin or KDE contour** to avoid overplotting (P1 slide 17), coloured by mean sentiment.
- A sparkline next to each topic's live count gives trend at a glance.

(c) Velocity at the visualization layer.

- Never send raw tweets to the browser; send only window aggregates. Over a 5-second refresh that is 50,000 raw tweets replaced by a handful of numbers.
- Keep a fixed rolling buffer in the chart (e.g. the last 60 one-minute points) and drop the oldest point on each update.
- Throttle redraws (e.g. at most once per second) and update the existing SVG elements with D3's data join instead of redrawing everything; use Canvas when many marks are needed.
- Pre-aggregate the cluster view server-side into hex cells, so the browser draws thousands of cells, not millions of dots.

(d) Pipeline. X/Twitter API producer → **Apache Kafka** topic (durable event log) → **Spark Structured Streaming / Flink** (sentiment scoring + 1-min tumbling window) → time-series store (**InfluxDB / Prometheus**, or **Elasticsearch**) → either **Grafana** polling every 5 s for the ops view (or Kibana on

Elasticsearch), or a **WebSocket** server pushing each new aggregate to a **D3.js** chart for sub-second custom views (P1 slide 20). This matches the Harvard CS171 lab that wires a Kafka consumer to a D3.js chart via WebSocket (P1 slide 24).

Tufte/Shneiderman check: no 3-D, no heavy gridlines; overview (all topics) → filter (topic dropdown, time range) → details (hover shows minute, count, score).

Part 2 · Discussion Questions

Lesson 1. Data quality is a visualisation problem: the NYT's biggest engineering effort was cleaning data, not drawing charts; retroactive state corrections forced D3 charts to support negative bars (days where cases were revised down)

ANSWER

Note: P2 slide 15 is titled "Lessons Learned & Discussion Questions" but only the five Key Lessons are printed on it. Each lesson is listed here as a discussion prompt.

What it means. A chart can only be as right as its data. The NYT pulled from Johns Hopkins CSSE, CDC and 50 state health departments, each with different column layouts (schemas), missing values and later corrections (P2 slide 14). Their pipeline was Python scrapers → pandas cleaning → GitHub CSV → D3 charts updated hourly.

For: most of the work was schema normalisation across 50 states, not SVG rendering (P2 slide 19). When a state revised cases down, the daily change was negative, so the chart design had to allow negative bars. Data problems showed up as visual design problems.

Against / nuance: cleaning is a data-engineering job, and good design can still not fix bad data. But hiding corrections would mislead readers, so the visual must show them honestly.

Exam point: in big data, visualization is the first quality check (P1 slide 6); bad data shows up first in charts.

Lesson 2. Accessibility is non-negotiable at 6M readers/day: every chart had a data table alternative for screen readers; colour scales were tested for colour-blindness (deuteranopia, protanopia)

ANSWER

What it means. With 6 million readers a day (P2 slide 13), even a small share who are blind or colour-blind is a large number of people. A screen reader cannot read an SVG chart, so a data table was offered. Deuteranopia and protanopia are forms of red-green colour-blindness, so red/green scales were avoided.

For: public-health information must reach everyone; the tracker was cited in CDC, WHO and UK SAGE briefs (P2 slide 14).

Against / cost: extra tables and testing take time on a daily deadline; colour-safe palettes limit design choices. The NYT judged the cost worth paying.

Lesson 3. Small multiples beat animation: the 50-state grid (Tufte small multiples) was more actionable than an animated map of spreading cases

ANSWER

What it means. Small multiples show every state at once on the same scale. Animation shows one frame at a time, so the viewer must remember the previous frame to compare (P2 slide 7).

For: readers could compare all states side by side and find their own state's trend without waiting.

Against: animation can grab attention and show spread across geography in a vivid way; a grid of 50 small charts can feel dense on a phone. For analysis, the grid wins; for a headline graphic, animation may still be used.

Lesson 4. Rolling averages are editorial decisions: choosing 7-day vs. 14-day average changes what story the chart tells; these are journalistic choices with ethical weight

ANSWER

What it means. A 7-day rolling average replaces each day with the mean of that day and the 6 before it. The NYT used it to remove day-of-week reporting artefacts, such as low weekend counts (P2 slide 14).

For 7-day: removes the weekly cycle and reacts within days to real change.

For 14-day: smoother, fewer false alarms, but hides a turning point for longer.

Why it is ethical: the same data can look calm or alarming depending on the window, and readers made health decisions from it. The window choice should be stated on the chart.

Lesson 5. GitHub as data infrastructure: publishing the raw CSV on GitHub enabled 1,000+ derivative dashboards worldwide within days

ANSWER

What it means. The NYT published its cleaned COVID data as CSV files on GitHub. Others built more than 1,000 dashboards from it within days (P2 slide 15).

For: openness lets researchers check the numbers and reuse the expensive cleaning work; GitHub gives version history, so corrections are traceable.

> **Against:** GitHub is not built as a database; others may misuse or misread the data, and the publisher must keep the files consistent as corrections arrive.

Part 2 · Live Exercise

Task. Live Exercise — D3.js Bar Chart with Tufte Styling Task (25 minutes, pairs) Build a minimal D3.js bar chart that follows Tufte's data-ink principles: 1. No gridlines (or only a single light reference line) 2. y-axis starts at 0 3. No fill gradient — use a flat colour 4. Tooltip on hover (Shneiderman: details-on-demand) 5. Add a sparkline for each bar showing the 7-day trend Dataset: weekly Kafka message counts for 5 topics (generate 7×5 random values). Extension: add a dropdown filter to select one topic and zoom into its 7-day line chart.

```
<!DOCTYPE html>
<meta charset="utf-8">
<style>
  .bar { fill: steelblue; }
  .bar:hover { fill: #b5651d; }
  .axis path, .axis line {
    stroke: #ccc; stroke-width: 0.5px; }
  .tooltip {
    position: absolute; background: #fff;
    border: 1px solid #ccc; padding: 6px;
    font-size: 11px; pointer-events: none; }
</style>
<body>
<script src=
  "https://d3js.org/d3.v7.min.js"></script>
<script>
const topics = ["orders","clicks",
                "errors","logins","search"];
const data = topics.map(t => ({
  topic: t,
  count: Math.floor(Math.random()*50000)
         + 10000
}));
const svg = d3.select("body").append("svg")
  .attr("width", 500).attr("height", 300);
const x = d3.scaleBand()
  .domain(data.map(d=>d.topic))
  .range([50,480]).padding(0.3);
const y = d3.scaleLinear()
  .domain([0, d3.max(data,d=>d.count)])
  .range([260, 20]);
svg.append("g").attr("transform","translate(0,260)")
  .call(d3.axisBottom(x));
svg.append("g").attr("transform","translate(50,0)")
  .call(d3.axisLeft(y).ticks(4));
svg.selectAll(".bar").data(data).join("rect")
  .attr("class","bar")
  .attr("x",d=>x(d.topic))
```

```
  .attr("y",d⇒y(d.count))
  .attr("width",x.bandwidth())
  .attr("height",d⇒260-y(d.count));
</script>
```

ANSWER

What you are building. A bar chart of weekly Kafka message counts for 5 topics (orders, clicks, errors, logins, search) in D3.js, styled by Tufte's rules and with Shneiderman's details-on-demand (P2 slide 17). D3 binds each data object to an SVG `rect`: the bar's height is computed from the data value.

Steps.

1. Save the code below as `index.html` and open it in a browser (needs internet for the D3 script from d3js.org).
2. Generate the 7×5 = 35 random values: 7 daily counts per topic, each `Math.floor(Math.random()*50000) + 10000`, so each day is between 10,000 and 59,999. The bar shows the weekly total (sum of 7 days).
3. Build scales: `scaleBand` spreads 5 bars over x = 50..480 px; `scaleLinear` maps 0..max to y = 260..20 px (SVG y grows downwards, so 0 is at the bottom).
4. Draw axes, bars, a reference line, the tooltip, the sparklines, then the dropdown extension.

Skeleton bug to fix: the skeleton's CSS styles `.axis path, .axis line` light grey, but the axis groups are never given `class="axis"`, so they render in default black. The completed code adds `.attr("class","axis")`.

Completed, runnable code:

```
<!DOCTYPE html>
<meta charset="utf-8">
<style>
  body { font-family: sans-serif; }
  .bar { fill: steelblue; }                  /* req 3: one flat colour */
  .bar:hover { fill: #b5651d; }
  .axis path, .axis line { stroke: #ccc; stroke-width: 0.5px; }
  .ref { stroke: #ddd; stroke-dasharray: 3 3; }   /* req 1: one light
reference line */
  .spark { fill: none; stroke: #555; stroke-width: 1px; }
  .tooltip { position: absolute; background: #fff; border: 1px solid #ccc;
             padding: 6px; font-size: 11px; pointer-events: none; opacity: 0;
}
</style>
<body>
<label>Topic: <select id="pick"><option value="">All topics</option></select>
```

```
</label><br>
<script src="https://d3js.org/d3.v7.min.js"></script>
<script>
const topics = ["orders","clicks","errors","logins","search"];
// 7 x 5 random values: 7 daily counts per topic; bar = weekly total
const data = topics.map(t => {
  const days = d3.range(7).map(() => Math.floor(Math.random()*50000) +
10000);
  return { topic: t, days: days, count: d3.sum(days) };
});

const W = 500, H = 340;                    // 40 px extra at the bottom for
sparklines
const svg = d3.select("body").append("svg").attr("width", W).attr("height",
H);

const x = d3.scaleBand().domain(data.map(d => d.topic)).range([50,
480]).padding(0.3);
const y = d3.scaleLinear().domain([0, d3.max(data, d =>
d.count)]).range([260, 20]); // req 2: starts at 0

svg.append("g").attr("class","axis").attr("transform","translate(0,260)")
   .call(d3.axisBottom(x));
svg.append("g").attr("class","axis").attr("transform","translate(50,0)")
   .call(d3.axisLeft(y).ticks(4));          // no gridlines drawn anywhere
(req 1)

// req 1: single light reference line at the mean weekly count
const mean = d3.mean(data, d => d.count);
svg.append("line").attr("class","ref")
   .attr("x1", 50).attr("x2", 480).attr("y1", y(mean)).attr("y2", y(mean));

// req 4: tooltip = details on demand
const tip = d3.select("body").append("div").attr("class","tooltip");

svg.selectAll(".bar").data(data).join("rect")
   .attr("class","bar")
   .attr("x", d => x(d.topic))
   .attr("y", d => y(d.count))
   .attr("width", x.bandwidth())
   .attr("height", d => 260 - y(d.count))
   .on("mouseover", (event, d) => {
      tip.style("opacity", 1)
         .html(`<b>${d.topic}</b><br>week: ${d3.format(",")(d.count)}<br>` +
               `last day: ${d3.format(",")(d.days[6])}`);
   })
```

```
   .on("mousemove", event => tip.style("left", (event.pageX + 10) + "px")
                                .style("top",  (event.pageY - 28) + "px"))
   .on("mouseout", () => tip.style("opacity", 0));

// req 5: one sparkline per bar, under the axis labels (y 290..325)
const sx = d3.scaleLinear().domain([0, 6]).range([0, x.bandwidth()]);
data.forEach(d => {
  const sy = d3.scaleLinear().domain(d3.extent(d.days)).range([325, 290]);
  const g = svg.append("g").attr("transform", `translate(${x(d.topic)},0)`);
  g.append("path").attr("class","spark")
   .attr("d", d3.line().x((v, i) => sx(i)).y(v => sy(v))(d.days));
  g.append("circle").attr("r", 1.5).attr("fill", "#b5651d")      // mark the
last day
   .attr("cx", sx(6)).attr("cy", sy(d.days[6]));
});

// Extension: dropdown filter → zoom into one topic's 7-day line chart
const pick = d3.select("#pick");
pick.selectAll("option.t").data(topics).join("option")
    .attr("class","t").attr("value", t => t).text(t => t);
const detail = d3.select("body").append("svg").attr("width",
W).attr("height", 220);

pick.on("change", function () {
  detail.selectAll("*").remove();
  const d = data.find(r => r.topic === this.value);
  svg.selectAll(".bar").attr("opacity", r => !d || r === d ? 1 : 0.25);
  if (!d) return;
  const lx = d3.scaleLinear().domain([0, 6]).range([50, 480]);
  const ly = d3.scaleLinear().domain([0, d3.max(d.days)]).range([190, 20]);
// still starts at 0

detail.append("g").attr("class","axis").attr("transform","translate(0,190)")
        .call(d3.axisBottom(lx).ticks(7).tickFormat(i => "day " + (i + 1)));
  detail.append("g").attr("class","axis").attr("transform","translate(50,0)")
        .call(d3.axisLeft(ly).ticks(4));
  detail.append("path").attr("fill","none").attr("stroke","steelblue")
        .attr("stroke-width", 1.5)
        .attr("d", d3.line().x((v, i) => lx(i)).y(v => ly(v))(d.days));
  detail.append("text").attr("x", 55).attr("y", 14).attr("font-size", 11)
        .text(d.topic + ": daily messages, last 7 days");
});
</script>
```

How each numbered requirement is met.

1. **No gridlines:** D3 axes draw no gridlines by default; only one dashed light-grey reference line at the mean is added.
2. **y-axis starts at 0:** the y domain is `[0, max]`. Starting at a higher value would exaggerate differences (a lie factor above 1, P2 slide 7).
3. **No fill gradient:** bars use one flat colour (`steelblue`); hover changes it to `#b5651d`.
4. **Tooltip on hover:** `mouseover` shows the topic, weekly total and last day's count next to the cursor; `mouseout` hides it. This is Shneiderman's details-on-demand.
5. **Sparkline per bar:** a tiny 7-point line (no axes) under each bar shows the 7-day trend, with a dot on the last day. Each sparkline uses its own min-max scale, because a sparkline shows shape and direction, not size.

Extension: the dropdown lists the 5 topics. Picking one fades the other bars and draws a full 7-day line chart of that topic below (zoom and filter, then details). Picking "All topics" restores the overview. Together this follows the full mantra.

Expected output (not run; values are random so they change on every reload): five steelblue bars of different heights. The tallest bar reaches the top of the plot area (y = 20, height 240 px) because the y domain ends at the maximum. A bar with half the maximum count is 120 px tall. Each band step is 430 ÷ 5.3 ≈ 81.1 px and each bar about 56.8 px wide. Weekly totals fall between 70,000 and 419,993. Hovering turns a bar brown and shows a tooltip.

Maths for Week 9

This chapter is mostly about design, but several slide facts are simple ratios: data-ink ratio, lie factor, how much hexbins and data cubes shrink the data, how alpha transparency saturates, how much a stream window compresses events, and how D3 scales turn values into pixels. All numbers below were recomputed in Python.

Data-ink ratio (Tufte)

```
Data-ink ratio = data-ink ÷ total ink
```

WHAT EACH SYMBOL MEANS

- **data-ink**: ink that encodes data (bars, points, lines).
- **total ink**: all ink in the chart, including borders, gridlines, legends, 3-D shading.
- Result is between 0 and 1; higher is better (P1 slide 8).

WORKED EXAMPLE

A chart uses 1,000 units of ink, of which 300 draw the bars. Ratio = 300 ÷ 1,000 = **0.3**. Remove 500 units of chartjunk (3-D, gridlines, legend): total = 500, data-ink still 300, ratio = 300 ÷ 500 = **0.6**. The ratio doubled without changing the data.

PRACTICE

A chart has 400 units of data-ink and 600 units of decoration. Ratio? Total = 1,000, ratio = 400 ÷ 1,000 = **0.4**. After deleting all decoration, ratio = 400 ÷ 400 = **1.0** (the maximum).

Lie factor (Tufte)

```
Lie factor = size of effect shown in graphic ÷ size of effect in data
```

WHAT EACH SYMBOL MEANS

- **effect in data**: the real % change between values.
- **effect shown**: the % change in the drawn size (e.g. bar height).
- 1 = truthful; above 1 = overstates; below 1 = understates (P2 slide 7).

WORKED EXAMPLE

Slide example: y-axis starts at 50 instead of 0. Data rises +10%, but the bar doubles in height (+100%). Lie factor = 100 ÷ 10 = **10**.

PRACTICE

Revenue goes from 100 to 110 (+10%). Because of a cut axis, the bars are drawn 40 px and 60 px tall: shown change = (60 − 40) ÷ 40 = +50%. Lie factor = 50 ÷ 10 = **5**, so the graphic overstates the change five times.

Hexbin reduction (overplotting)

```
reduction factor = number of points ÷ number of hex cells
```

WHAT EACH SYMBOL MEANS

- **points**: records in the scatter.
- **hex cells**: hexagons drawn after binning, each coloured by its count.

WORKED EXAMPLE

Slide: 10,000,000 points → about 10,000 hex cells (P1 slide 17). Factor = 10,000,000 ÷ 10,000 = **1,000** times fewer shapes to draw.

PRACTICE

2,000,000 GPS pings binned into 500 hexes. Factor = 2,000,000 ÷ 500 = **4,000**; the browser draws 1 − 500 ÷ 2,000,000 = **99.975%** fewer shapes.

Data cube size and row reduction (latency)

```
cube rows (finest level) ≈ d₁ × d₂ × … × dₖ; speed-up in rows read ≈ fact rows ÷ cube rows; number of group-by combinations = 2^k
```

WHAT EACH SYMBOL MEANS

- **d_i**: number of distinct values of dimension i (dates, districts, products, channels).
- **k**: number of dimensions.
- **fact rows**: rows in the raw table.

WORKED EXAMPLE

Slide: 100 billion fact rows; cube of millions of rows (P1 slide 19). Take 5 million: 100,000,000,000 ÷ 5,000,000 = **20,000** times fewer rows; the cube is 1 − 5,000,000 ÷ 100,000,000,000 = **99.995%** smaller.

PRACTICE

Dimensions: 365 dates × 64 districts × 50 products × 4 channels. Finest cube = 365 × 64 × 50 × 4 = **4,672,000** rows (at most; only combinations that occur are stored). Rows-read reduction vs 100 billion = 100,000,000,000 ÷ 4,672,000 ≈ **21,404**. With k = 4 dimensions there are 2^4 = **16** possible group-by combinations (including the grand total).

Alpha transparency saturation (why it fails at high density)

```
opacity after n stacked points = 1 - (1 - α)^n
```

WHAT EACH SYMBOL MEANS

- **α**: opacity of one point (slide uses 0.02).
- **n**: points stacked on the same pixel.
- This standard compositing formula is added here to explain the slide's remark that alpha "still fails at very high densities" (P1 slide 17).

WORKED EXAMPLE

α = 0.02: n = 10 → 1 − 0.98^{10} = **0.183**; n = 50 → **0.636**; n = 100 → **0.867**; n = 200 → **0.982**. Above roughly 200 points per pixel everything looks equally black, so dense regions can no longer be told apart.

PRACTICE

How many stacked points reach 50% opacity at α = 0.02? Solve $0.98^n = 0.5$: n = ln 0.5 ÷ ln 0.98 ≈ **34.3**, so about 35 points ($1 - 0.98^{35} = 0.507$). For 99%: n = ln 0.01 ÷ ln 0.98 ≈ **227.9**, about 228 points.

Streaming volume per window and refresh

```
events per window = event rate × window length; updates per day = 86,400 s ÷ refresh interval
```

WHAT EACH SYMBOL MEANS

- **event rate**: events per second.
- **window length**: tumbling window size in seconds.
- **refresh interval**: seconds between chart updates.

WORKED EXAMPLE

A2: 10,000 tweets/s, 1-minute tumbling window → 10,000 × 60 = **600,000** tweets summarised into one point per topic. Per 5-s Grafana refresh: 10,000 × 5 = **50,000** tweets. Per day: 10,000 × 86,400 = **864,000,000** tweets.

PRACTICE

A1(b): hospitalisation rate updated every 5 minutes. Updates per day = 24 × 60 ÷ 5 = **288**. A chart of the last 24 hours therefore needs only 288 points.

D3 linear and band scales (Live Exercise)

```
y(v) = 260 - 240 × v ÷ max; bar height = 260 - y(v) = 240 × v ÷ max; band step = (480 - 50) ÷ (n - p + 2p), bandwidth = step × (1 - p)
```

WHAT EACH SYMBOL MEANS

- **v**: a topic's count; **max**: largest count.
- 260 and 20: pixel range of the y scale (bottom, top), so 240 px of height.
- **n**: number of bars (5); **p**: padding (0.3); 50..480: x pixel range (430 px).

WORKED EXAMPLE

max = 60,000, v = 30,000: height = 240 × 30,000 ÷ 60,000 = **120 px**; the bar top is at y = 260 − 120 = **140**.

PRACTICE

Band layout for 5 bars, padding 0.3: step = 430 ÷ (5 − 0.3 + 0.6) = 430 ÷ 5.3 ≈ **81.13 px**; bandwidth = 81.13 × 0.7 ≈ **56.79 px**; the first bar starts at 50 + 0.3 × 81.13 ≈ **74.34 px**.

Chapter 6 · Week 10: Big Data in Real Applications: Social Media, Time Series & Health Data

The chapter in plain words

This week is about using the big data tools from Weeks 1 to 9 (HDFS, Kafka, Spark, Hive, Grafana) on three real domains: social media, time series and healthcare (P1 slide 5). The tools stay the same. What changes is the shape of the data and the risks.

Social media data is mainly a *graph*: people are nodes, and "follows" or "retweets" are edges. It is also timestamped, full of messy short text (slang, emojis, Bangla-English mixing) and heavy on images and video (P1 slide 7). Common analyses: **community detection** (finding tightly linked groups, using the Louvain algorithm), **influence maximisation** (picking the few users to seed so a message spreads furthest), **sentiment analysis** (is a post positive or negative, using VADER) and **trend detection** (spotting a phrase whose frequency suddenly jumps above its normal level).

Time series data is a list of values in time order, like weekly flu cases or daily sales. Order matters: shuffle the rows and the pattern is gone. Today's value is linked to yesterday's (autocorrelation). Most series have a **trend** (long-run drift up or down) and **seasonality** (repeating cycles, like every winter or every weekend) (P1 slide 14).

Three forecasting tools appear. **ARIMA** is the classic statistics model: it predicts from past values and past errors, but needs a stable (stationary) series and expert tuning. **Facebook Prophet** splits a series into parts you can read: y(t) = trend g(t) + seasonality s(t) + holidays h(t) + noise. It fits thousands of series automatically, and Spark can run one Prophet model per group in parallel with `applyInPandas` (P1 slides 16, 19). **LSTM** is a neural network for sequences. For **anomaly detection** the slides give STL (flag big leftover residuals) and Isolation Forest (odd points are easy to cut off from the rest) (P1 slide 18).

Healthcare big data comes in four forms: electronic health records (EHR), genomics (one genome is about 200 GB), wearables (per-second streams) and medical images (DICOM files of 200 MB to 2 GB) (P1 slide 21). ML on EHR data can predict ICU deaths, readmissions and sepsis earlier than doctors' rules of thumb (P1 slide 23). Because the data is about real patients, privacy law (HIPAA in the USA, GDPR in the EU, the upcoming PDPA in Bangladesh) shapes every design. Tools to protect it: de-identification, differential privacy (adding calibrated noise) and federated learning (training at each hospital without moving the raw data) (P1 slide 24).

AlphaFold (DeepMind) is the science example: trained on 170,000 known protein structures and searches across 250 million sequences, it predicts a protein's 3D shape from its amino-acid sequence. It released 200 million predicted structures in 2022 and won the 2024 Nobel Prize in Chemistry (P1 slides 25, 26, 32).

Part 2 adds two papers and a case study. **Kwak et al. (2010)** crawled 41.7 million Twitter users and 1.47 billion follow edges and found Twitter behaves like a *news medium*, not a friendship network: only 22.1% of follows are mutual and 85% of trending topics are headline news (P2 slides 5 to 7). **Taylor & Letham (2018)** is the Prophet paper (P2 slides 9, 10).

The case study is **flu surveillance**. Official CDC reports lag 1 to 2 weeks. Twitter posts, pharmacy sales, ER notes and searches can show an outbreak earlier (P1 slide 28). But **Google Flu Trends** over-predicted flu by 2× in 2012-2013 because search behaviour changed. The lesson: big data signals are a *supplement* to clinical reporting, never a replacement (P1 slide 29, P2 slide 14).

Part 1 · Student Assessment Pool: MCQ

Q1. Facebook's Prophet library is primarily designed for:

(a) Graph analytics and community detection in social networks
(b) Real-time stream processing of events
(c) Time series forecasting with decomposable trend, seasonality, and holiday components
(d) Text sentiment classification using deep learning

ANSWER

Answer: (c) Time series forecasting with decomposable trend, seasonality, and holiday components.

Why: Prophet models a series as $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$: trend, seasonality, holidays and noise (P1 slide 16). "Decomposable" means it splits the forecast into these separate, readable parts.

- (a) Wrong: graph analytics and community detection use GraphX, NetworkX or Louvain, not Prophet (P1 slide 31 notes).
- (b) Wrong: real-time stream processing is the job of Kafka and Spark Streaming. Prophet fits a model to historical data.
- (d) Wrong: Prophet has nothing to do with text or sentiment. Sentiment on social media uses tools like VADER (P1 slide 11).

Q2. Which algorithm is most commonly used for community detection in large-scale social network graphs?

(a) K-Means clustering
(b) PageRank
(c) Louvain algorithm
(d) ARIMA

ANSWER

Answer: (c) Louvain algorithm.

Why: a community is a group of nodes more densely linked to each other than to the rest of the graph. Louvain starts with every node in its own community, greedily merges nodes to raise *modularity* (edges inside communities minus what chance would give), collapses communities into super-nodes and repeats until modularity stops rising. It runs in O(n log n), so it scales to billion-node graphs in Spark GraphX (P1 slide 9).

- (a) Wrong: K-Means clusters points (vectors) by distance, not graph links.
- (b) Wrong: PageRank scores how important a node is, not which group it belongs to.
- (d) Wrong: ARIMA is a time series forecasting method (P1 slide 31 notes).

Q3. HIPAA in the United States primarily regulates the privacy and security of:

(a) Financial transaction data and bank records
(b) Healthcare data including patient records and medical information
(c) Social media user data and behavioural profiles
(d) Government data retention and public records

ANSWER

Answer: (b) Healthcare data including patient records and medical information.

Why: HIPAA is the Health Insurance Portability and Accountability Act. It requires removing 18 PHI (protected health information) identifiers before research use, and consent (P1 slides 24, 32).

- (a) Wrong: US financial data falls under GLBA (or Basel III for banks), not HIPAA (P1 slide 32 notes).
- (c) Wrong: social media profiles are not health records. In the EU, GDPR would cover them.
- (d) Wrong: government record keeping is handled by other laws, not a health privacy act.

Q4. DeepMind's AlphaFold used big data and deep learning to solve:

(a) Real-time financial fraud detection at bank-card transaction scale
(b) Social network influence maximisation over billion-node graphs
(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence
(d) Time series anomaly detection in IoT sensor streams

ANSWER

Answer: (c) The protein structure prediction problem, predicting 3D structure from amino acid sequence.

Why: a protein's 3D shape decides what it does, and lab methods take months per protein. AlphaFold 2 was trained on 170,000 structures from the Protein Data Bank, used multiple sequence

alignments built from 250 million UniProt sequences, reached a median GDT score above 92%, and released 200 million predicted structures in 2022 (P1 slides 25, 26). It won the 2024 Nobel Prize in Chemistry (P1 slide 32).

- (a) Wrong: fraud detection is an anomaly detection task (like the bKash Isolation Forest example, P1 slide 18), not AlphaFold.
- (b) Wrong: influence maximisation is Kempe et al.'s greedy algorithm (P1 slide 10).
- (d) Wrong: IoT anomaly detection uses STL or Isolation Forest (P1 slide 18).

Part 1 · Student Assessment Pool: True/False

1. Google Flu Trends successfully replaced traditional CDC flu surveillance and has been used as the primary source ever since.

ANSWER

False.

Google Flu Trends over-fitted to search behaviour and over-predicted flu incidence by 2× in 2012-2013, and it was discontinued (P1 slides 28, 29; P2 slide 14). Syndromic surveillance gives early signals, not ground truth.

Corrected: Google Flu Trends failed (2× over-prediction in 2012-2013) and was discontinued; CDC still relies on clinical reporting (ILINet, BioSense), with big data signals used only as a supplement.

2. Time series data is order-sensitive: randomly shuffling the records in a time series changes the analysis result.

ANSWER

True.

In a time series, the order is the signal: today's value is correlated with yesterday's (autocorrelation), and trend and seasonality only exist in time order, so shuffling rows destroys them (P1 slide 14).

Analogy: shuffle the pages of a diary and you can still count the words, but you can no longer tell what happened first.

3. Social network graphs are typically sparse, with most nodes connected to only a small fraction of all possible neighbours.

ANSWER

True.

Each user links to a tiny share of all other users. Twitter in 2010 had 41.7 million users but only 1.47 billion follow edges, and follower counts follow a power law where only the top 0.05% have over a million followers (P2 slides 5, 6).

Check: possible directed edges = 41.7M × (41.7M − 1) ≈ 1.74 × 10^15, so 1.47 billion edges is about 0.00008% of them. Average out-degree ≈ 1.47B / 41.7M ≈ 35.3.

4. Medical imaging data such as MRI and CT scans is classified as structured data because it is stored in files.

ANSWER

False.

Being stored in a file does not make data structured. CT, MRI, X-ray and pathology slides are **unstructured binary files** (DICOM, 200 MB to 2 GB each), kept in PACS and analysed by deep learning (CNN, ViT), not SQL (P1 slide 21).

Corrected: Medical imaging data such as MRI and CT scans is classified as unstructured data (binary DICOM files), even though it is stored in files.

Part 1 · Student Assessment Pool: Descriptive

D1. Describe three types of analysis commonly performed on social media data (e.g., community detection, sentiment analysis, influence maximisation). For each, state one specific ethical risk that arises when the analysis is deployed at scale.

ANSWER

Social media analytics treats a platform as a graph of users (nodes) and interactions (edges) with timestamped text attached (P1 slide 7). Three common analyses:

1. **Community detection.** Finds groups of users more densely linked to each other than to the rest of the graph. The Louvain algorithm merges nodes to maximise modularity and scales to billion-node graphs in Spark GraphX (P1 slide 9). Use: interest groups, political factions, bot networks.
   Ethical risk: profiling. Labelling users by group can expose political views, religion or sexuality they never shared, and can be used to target or watch a whole faction.
2. **Sentiment analysis.** Scores each post as positive, neutral or negative. VADER gives a compound score in [−1, +1]; above 0.05 is positive, below −0.05 negative (P1 slide 11).
   Ethical risk: suppressing dissent. A government can flag posts with negative sentiment about its policy and target their authors (P1 slide 12). Errors on Bangla slang or sarcasm can also wrongly flag people.

3. **Influence maximisation.** Chooses k seed users so a message spreads to the most people. It is NP-hard; Kempe et al.'s greedy method reaches at least (1 − 1/e) ≈ 63% of the best spread (P1 slide 10).
   Ethical risk: manipulation. The same method that spreads a health message can spread misinformation or micro-target ads at vulnerable users (P1 slide 12).

Mitigation in one line: transparency and purpose limitation: say what the analysis is for and use it only for that (P1 slide 12).

Example: the #QurbaniEid spike (1,200× in 6 hours) was checked with VADER plus a Bangla lexicon: 92% positive, so it was real celebration, not a coordinated campaign (P1 slide 12).

D2. What is the difference between ARIMA and Facebook Prophet for time series forecasting? Under what conditions would you prefer Prophet over ARIMA? When might ARIMA still be the better choice?

ANSWER

ARIMA(p, d, q) (AutoRegressive Integrated Moving Average) predicts the next value from the past p values (AR), after differencing the series d times to make it stationary (I), plus the past q forecast errors (MA) (P1 slide 14). "Stationary" means the mean and variance do not change over time.

Prophet fits an additive model y(t) = g(t) + s(t) + h(t) + ε(t): piecewise trend with automatic changepoints, Fourier seasonality (yearly, weekly, daily at once), a user list of holidays, and noise (P1 slide 16).

Point	ARIMA	Prophet
Input	Needs stationary data	Handles trend and changepoints directly
Seasonality	Struggles with several seasonal cycles	Several seasonalities at once
Holidays	Not built in	h(t) term (Eid, Puja, elections)
Tuning	Expert picks p, d, q	Automatic; analysts tune readable knobs
Scale	Cannot hand-fit 10,000 series	One model per series, parallel in Spark

Prefer Prophet when: the series has strong or multiple seasonality, holiday effects, trend shifts, missing data or outliers, and when you must forecast thousands of series automatically (P1 slides 15, 16; P2 slide 10). Example: 5,000 SKU demand forecasts, 8 hours down to about 12 minutes with applyInPandas (P1 slide 19).

ARIMA can still be better when: the series is short, stationary, has no strong seasonality and the dataset is small (P1 slide 14). It is a well-understood statistical model, and an expert can tune it closely for one important series.

D3. Identify three data integration and privacy challenges specific to healthcare big data analytics. For each, describe one technical mitigation strategy (e.g., differential privacy, federated learning, de-identification).

ANSWER

Healthcare big data mixes EHR records, genomics, wearable streams and DICOM images (P1 slide 21), all about identifiable patients under laws like HIPAA, GDPR and Bangladesh's PDPA (P1 slide 24).

1. **Challenge: identifiable patient records used for research.** EHRs hold names, dates of birth, NIDs and phone numbers. HIPAA requires removing 18 PHI identifiers first.
   Mitigation: de-identification. Safe harbour removes the 18 identifiers (name, DOB, zip, NID, phone, etc.); expert determination uses a statistical disclosure check by a qualified expert (P1 slide 24).
2. **Challenge: data cannot leave each hospital.** Reliable models need 1 million+ patient records (P1 slide 23), but records sit in separate hospitals and moving them breaks consent and law (PDPA needs explicit consent for NID-linked records).
   Mitigation: federated learning. Train the model inside each hospital and share only model updates, never raw data (P1 slide 24).
3. **Challenge: published statistics can leak individuals.** Even counts like "patients with condition X in zip Y" can reveal one person when the group is small, especially after joining EHR, genomic and wearable data.
   Mitigation: differential privacy. Add calibrated random noise to aggregate results so no single record can be inferred (P1 slide 24).

Also an integration problem: formats differ (FHIR resources, Parquet genomics, Kafka wearable streams, DICOM binaries). A common health data lake on HDFS/S3 with a Hive Metastore and the HL7 FHIR standard brings them together (P1 slides 21, 22).

Part 1 · Student Assessment Pool: Analytical

A1. Flu Outbreak Early-Warning System for Bangladesh: Design a big data system to detect flu outbreaks in Bangladesh using Twitter/Facebook posts and pharmacy purchase records from major pharmacy chains. (a) How would you collect and ingest these two data sources? (Specify tools: Kafka, Sqoop, Flume, REST API) (b) What features would you extract from each source for a prediction model? (Name specific NLP features and pharmacy signal metrics) (c) What model would you use, and how would you validate it against ground-truth CDC/DGHS surveillance data? (d) What are the ethical risks of this system? How would you mitigate them?

ANSWER

Goal: catch flu outbreaks before the 1 to 2 week lag of physician reports (P1 slide 28), using the CDC syndromic surveillance pattern: sources → Kafka → Spark Streaming → alert engine + Grafana (P1 slide 29).

(a) Collect and ingest

- **Social posts:** pull from the Twitter/X API v2 and Facebook public Pages through their **REST APIs** (keyword filters in English and Bangla: "jor", fever, flu, cough). A small producer writes each post as JSON to a **Kafka** topic `social_raw`.
- **Pharmacy sales:** chains keep sales in relational databases. Use **Sqoop** for nightly bulk import of history into HDFS, and for near-real-time, ship POS log files with **Flume** (or CDC events) into a Kafka topic `pharmacy_sales`.
- Land both raw streams in HDFS/S3 as Parquet partitioned by date and district, for replay and training.

(b) Features

- **NLP from posts:** daily count of flu-keyword posts per district (location from profile or geotag); share of posts with symptom terms (fever, cough, "can't taste"); first-person illness flag ("I have fever" vs news about flu); VADER/Bangla-lexicon sentiment; ratio of today's count to its 4-week baseline (the trend-spike idea, P1 slide 12).
- **Pharmacy signals:** daily units of OTC antipyretics (paracetamol, ibuprofen) and cough syrup per district; week-on-week % change; z-score versus the same week last year; share of stores in a district with a spike. These spike 3 to 5 days before clinical confirmation; social posts peak 7 to 10 days before ER visits (P1 slide 28).

(c) Model and validation

- Model: **Prophet** per district on DGHS weekly ILI counts, with yearly seasonality, Eid/Puja holidays, and the social and pharmacy features as extra regressors (covariates, as CDC did with Twitter, P2 slide 14). Run all districts in parallel with Spark `applyInPandas`. An Isolation Forest on the daily features gives a second, spike-based alarm. Combine them in a small ensemble, since ensembles beat single models (P2 slide 15).
- Validation: rolling-origin backtest. Train on years 1 to n, forecast the next weeks, compare with DGHS reported ILI. Report error (MAE), correlation, and for alerts: lead time in days, hit rate (outbreaks caught) and false alarm rate. Test on a later season the model never saw, because Google Flu Trends had $r^2 = 0.97$ in-sample and still failed (P2 slide 14).

(d) Ethical risks and mitigation

- Privacy: posts tie illness to named people. Keep only district-level counts, drop user IDs after aggregation, add differential-privacy noise to small counts.
- False alarms and panic: media coverage drives searches and posts even when cases are low (P2 slide 14). Treat signals as early warnings that a health officer checks, never as automatic public announcements.

- Bias: rural and older people post less, so they are under-counted. Weight with pharmacy data and DGHS data.
- Misuse: the same pipeline could be used to watch individuals. Purpose limitation, access logs and consent rules under PDPA (P1 slides 12, 24).

A2. Retail Demand Forecasting Pipeline: A large Bangladeshi supermarket chain needs 90-day demand forecasts for 5,000 store–product SKU combinations. Design the end-to-end big data pipeline: (a) data ingestion from POS systems; (b) storage and partitioning strategy in HDFS/S3; (c) forecasting model choice (ARIMA vs. Prophet vs. LSTM) and justification; (d) how you parallelise model fitting across 5,000 SKUs using Spark; (e) how forecasts are served to the inventory management dashboard. Identify the primary bottleneck in your pipeline.

ANSWER

Baseline from the slides: fitting 5,000 Prophet models one after another on one machine takes 8+ hours; with Spark applyInPandas on 20 executors it takes about 12 minutes, a 40× speedup (P1 slide 19). An SKU (stock keeping unit) here is one product in one store.

(a) Ingestion from POS

- Each till (point-of-sale system) sends sale events to **Kafka** (topic `pos_sales`) in real time. For stores with poor connectivity, upload end-of-day files; bulk-load historical sales from the central database with **Sqoop**.
- A Spark job aggregates events to one row per day per SKU: `(sku_id, store_id, date, units)`.

(b) Storage and partitioning

- Raw events in HDFS/S3 as Parquet, partitioned by `date` (easy daily appends and deletion of old data).
- Daily aggregate table in Hive, partitioned by date and bucketed or clustered by `sku_id`, so reading one SKU's full history does not scan everything.
- Forecast output table partitioned by `run_date`, so each nightly run is one new partition.

(c) Model choice

- **ARIMA:** needs stationary data and hand-picked (p, d, q); cannot be manually fitted for thousands of series (P1 slide 15). Rejected.
- **LSTM:** can learn complex patterns, but needs much data per series, GPU training, careful tuning, and is a black box for store managers.
- **Prophet (chosen):** weekly and yearly seasonality, a holiday list for Eid, Puja and Pohela Boishakh, automatic changepoints, tolerates missing days and outliers, and its parts are readable (P1 slide 16; P2 slide 10).

(d) Parallel fitting in Spark (P1 slide 19)

```
def fit(pdf):
    m = Prophet(yearly_seasonality=True, weekly_seasonality=True,
holidays=bd_holidays)
    m.fit(pdf[["ds", "y"]])
    fc = m.predict(m.make_future_dataframe(periods=90))
    fc["sku_id"] = pdf["sku_id"].iloc[0]
    return fc[["sku_id", "ds", "yhat", "yhat_lower", "yhat_upper"]]

(daily.groupBy("sku_id")
      .applyInPandas(fit, schema="sku_id string, ds timestamp, yhat double,
yhat_lower double, yhat_upper double")
      .write.mode("overwrite").parquet("s3://retail/forecasts/run_date=..."))
```

Each `sku_id` group is sent to one task; Prophet runs inside a Pandas UDF; all 5,000 models fit in parallel. Rough sizing: 8 h / 5,000 = 5.76 s per model; 5,000 / 20 executors = 250 models per executor.

(e) Serving

- Write forecasts to Hive/S3 as Parquet (P1 slide 19). Load the latest run into a small serving store (PostgreSQL or a Hive/Presto view) indexed by store and SKU.
- The inventory dashboard (Superset or Grafana) reads it: forecast line with yhat_lower/yhat_upper bands, reorder point alerts when forecast demand exceeds stock.

Primary bottleneck: model fitting (step d). Prophet fitting through Stan is CPU-heavy and is the 8-hour step; ingestion and storage are cheap by comparison. Skew can also hurt: if a few SKUs have very long histories, their tasks finish last and the whole job waits. Fix: more executors/cores, one SKU per task, and limit history to the last 2 to 3 years.

Part 2 · Discussion Questions

1. Kwak et al. found Twitter's average shortest path is 4.12 hops over 41.7M users. If a misinformation tweet is retweeted by a high-PageRank account, estimate how many users it reaches within 3 hops assuming an average out-degree of 45. What graph property makes this estimate an overestimate? [CLO4 — Apply]

ANSWER

Idea: treat spread like a tree. Each account passes the tweet to 45 new followers, and each of them to 45 more.

- Hop 1: 45 users

- Hop 2: $45^2 = 2{,}025$ users
- Hop 3: $45^3 = 91{,}125$ users

Total within 3 hops = 45 + 2,025 + 91,125 = **93,195 users** (93,196 counting the high-PageRank account itself). That is about 0.22% of the 41.7M users.

Context: since the average shortest path is only 4.12 hops (P2 slide 6), one more hop ($45^4 = 4{,}100{,}625$) would make the tree estimate reach about 4.19 million, so a tweet can reach a large part of the network in a few hops.

Why it is an overestimate: the tree model assumes every follower at each hop is a new, different person. Real social graphs have **clustering and overlapping neighbourhoods** (followers of the same account often follow each other and share followers; Twitter communities are dense inside, P1 slide 9). So many hop-2 and hop-3 users are the same people counted again. Other reasons it is high:

- Not everyone who sees a tweet retweets it. Retweet cascades are wide and shallow (P2 slide 7), so most spread stops after hop 1.
- Low reciprocity (22.1%, P2 slide 6) and the power-law degree distribution mean the average of 45 is pulled up by a few celebrities; a typical user has far fewer followers.

Other side: the high-PageRank first hop can be much bigger than 45 (celebrities have over 1 million followers), so hop 1 alone can exceed the estimate. The real number depends on who retweets.

2. A Prophet model for weekly hospital admissions has strong annual and weekly seasonality but misses a sudden COVID-19 wave onset. Identify which Prophet component should detect this changepoint and explain what prior hyperparameter controls the model's sensitivity to new changepoints. [CLO4 — Analyse]

ANSWER

Component: the **trend g(t)**. In Prophet the trend is piecewise linear, $g(t) = (k + a(t)^T\delta)t + (m + a(t)^T\gamma)$, and δ holds slope changes at automatically detected changepoints (structural breaks) (P2 slide 10). A sudden wave is a break in the trend, not a repeating seasonal cycle, so seasonality s(t) cannot capture it and h(t) only covers known dates listed in advance.

Hyperparameter: `changepoint_prior_scale`. Prophet puts a *sparse prior* on δ: most changepoints get a slope change near 0 and only a few are large (P2 slide 10). The prior scale sets how big those changes may be.

- Small (default 0.05): stiff trend, fewer real shifts allowed. Good against overfitting, but it can smooth over a sudden wave, which is what happened here.
- Larger (e.g. 0.5, as in the Live Exercise): flexible trend that bends quickly at the wave onset, but may chase noise and gives wider forecast uncertainty.

Practical fixes: raise `changepoint_prior_scale` step by step and check on held-out weeks; let changepoints cover the latest data (`changepoint_range` near 1.0, since by default Prophet only places changepoints in the first 80% of the history); or add the known wave start as a manual changepoint or an extra regressor (e.g. case counts). Weekly seasonality on weekly data does nothing (all points fall on the same weekday), so it can be turned off.

3. Design a dengue fever surveillance system for Bangladesh using BioSense-style ED data, Twitter signals, and Prophet forecasting. Specify: data sources, pipeline architecture (Kafka + Spark), model inputs, and how you would validate the system before policy use. [CLO4 — Design]

ANSWER

Model to copy: the post-GFT CDC approach: direct medical data as the main signal, Twitter as a leading extra input, Prophet as the model (P2 slides 14, 19).

Data sources

- **Primary (direct signal):** emergency department and outpatient visit records from DGHS-linked hospitals, sent as HL7 FHIR messages like BioSense (P2 slide 13). Extract a "dengue-like illness" syndrome from chief complaints (high fever, joint pain, rash, low platelets) with NLP.
- **Lab data:** NS1 antigen test counts and positives.
- **Leading indicator:** Twitter/X and Facebook posts with dengue words in English and Bangla (via REST APIs).
- **Context:** rainfall and temperature (mosquito breeding), paracetamol and saline sales.

Pipeline (Kafka + Spark)

1. Hospitals and APIs publish to Kafka topics: `ed_visits`, `lab_results`, `social_posts`.
2. Spark Structured Streaming de-identifies records, runs NLP, and aggregates counts per district per day/week.
3. Aggregates land in HDFS/S3 Parquet partitioned by week and district.
4. Nightly Spark job runs one Prophet model per district with `groupBy("district").applyInPandas(...)`, as CDC ran 60 HHS regions at once (P2 slide 14).
5. Alert engine flags districts where actual cases exceed the forecast upper band; Grafana dashboard for DGHS (P1 slide 29).

Model inputs: y = weekly dengue-like ED visits per district; yearly seasonality (dengue peaks in the monsoon); holidays such as Eid (travel and closed clinics change visits); extra regressors: lagged rainfall, Twitter dengue-post count, NS1 positivity; changepoint_prior_scale tuned so a new outbreak shows up as a trend change.

Validation before policy use

- Backtest on past seasons (e.g. the 2019 and 2023 Bangladesh dengue outbreaks): forecast each week only from data before it, compare with DGHS confirmed counts (MAE, lead time, false alarms).
- Out-of-sample test on a season not used in training, because GFT had $r^2 = 0.97$ in-sample and still over-predicted by 2× (P2 slide 14).
- Ablation: compare with and without Twitter. Twitter adds about 2 weeks of lead time but with 30% higher error (P2 slide 15), so keep it only if it improves early detection.
- Run in shadow mode for one season next to the existing reporting system, and let epidemiologists inspect the g, s, h parts before alerts drive decisions.

Two sides: social signals give speed but are noisy and biased toward urban, young users; clinical data is accurate but slower. The design uses each for what it does best.

Part 2 · Live Exercise

Task. Live Exercise — Prophet Flu Forecasting in Python. Task (25 minutes, pairs): Fit a Prophet model to synthetic weekly flu incidence data and produce a 12-week forecast. Install: pip install prophet matplotlib pandas

```
import pandas as pd
import numpy as np
from prophet import Prophet
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
dates = pd.date_range("2021-01-04",
                      periods=156, freq="W")
# Annual flu seasonality + noise
ili = (5 + 3*np.sin(
          2*np.pi*np.arange(156)/52
          - np.pi/2)
       + rng.normal(0, 0.5, 156)).clip(0)
df = pd.DataFrame({"ds": dates, "y": ili})

# Optional: add Eid holiday
holidays = pd.DataFrame({
   "holiday": "Eid",
   "ds":       pd.to_datetime([
                 "2021-05-13","2022-05-03",
                 "2023-04-22"]),
   "lower_window": 0,
   "upper_window": 3,
})

m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    changepoint_prior_scale=0.05,
    holidays=holidays)
m.fit(df)
future = m.make_future_dataframe(
    periods=12, freq="W")
forecast = m.predict(future)
m.plot(forecast)
m.plot_components(forecast)
plt.show()
```

ANSWER

What the skeleton does: builds 156 weekly points (3 years × 52 weeks) of fake influenza-like illness (ILI) data, fits Prophet with an Eid holiday list, forecasts 12 more weeks and plots the forecast and its parts.

Steps

1. `pip install prophet matplotlib pandas`
2. Run the skeleton as is and look at the two plots.
3. Add `interval_width=0.95` (Prophet's default band is 80%, so the skeleton alone does not meet Goal 3).
4. Do the Eid and changepoint experiments (Goals 4, 5) by refitting with changed settings.

Complete runnable version (covers all five goals):

```
# pip install prophet matplotlib pandas
import pandas as pd
import numpy as np
from prophet import Prophet
import matplotlib.pyplot as plt

# Goal 1: 3 years (156 weeks) of synthetic weekly ILI data
rng = np.random.default_rng(42)
dates = pd.date_range("2021-01-04", periods=156, freq="W")
ili = (5 + 3*np.sin(2*np.pi*np.arange(156)/52 - np.pi/2)
       + rng.normal(0, 0.5, 156)).clip(0)
df = pd.DataFrame({"ds": dates, "y": ili})

holidays = pd.DataFrame({
    "holiday": "Eid",
    "ds": pd.to_datetime(["2021-05-13", "2022-05-03", "2023-04-22"]),
    "lower_window": 0,
    "upper_window": 3,
})

def fit_forecast(data, cps=0.05, use_holidays=True):
    m = Prophet(
        yearly_seasonality=True,
        weekly_seasonality=False,      # weekly data: one point per week
        changepoint_prior_scale=cps,
        holidays=holidays if use_holidays else None,
        interval_width=0.95)           # Goal 3: 95% interval (default is
0.80)
    m.fit(data)
    future = m.make_future_dataframe(periods=12, freq="W")
    return m, m.predict(future)
```

```
# Goals 2 + 3: fit and plot a 12-week forecast with 95% bands
m, forecast = fit_forecast(df)
m.plot(forecast)
m.plot_components(forecast)
print(forecast[["ds", "yhat", "yhat_lower", "yhat_upper"]].tail(12))

# Goal 4: give the holiday something to find - inject an Eid spike
df_eid = df.copy()
eid_weeks = pd.to_datetime(["2021-05-16", "2023-04-23"])  # Sundays inside
the Eid windows
df_eid.loc[df_eid["ds"].isin(eid_weeks), "y"] += 3
m_no, fc_no = fit_forecast(df_eid, use_holidays=False)
m_h,  fc_h  = fit_forecast(df_eid, use_holidays=True)
print("Eid effect (holiday model):",
      fc_h.loc[fc_h["ds"].isin(eid_weeks), "Eid"].round(2).tolist())
m_h.plot_components(fc_h)

# Goal 5: compare trend flexibility
_, fc_stiff = fit_forecast(df, cps=0.05)
_, fc_flex  = fit_forecast(df, cps=0.5)
plt.figure()
plt.plot(df["ds"], df["y"], ".", color="grey", label="data")
plt.plot(fc_stiff["ds"], fc_stiff["trend"], label="trend cps=0.05")
plt.plot(fc_flex["ds"],  fc_flex["trend"],  label="trend cps=0.5")
plt.legend(); plt.title("Trend flexibility")
plt.show()
```

Facts about the data (checked with Python's datetime and math):

- `freq="W"` means weeks ending Sunday. 2021-01-04 is a Monday, so the first date is **2021-01-10** and the last is **2023-12-31**.
- The curve 5 + 3·sin(2πi/52 − π/2) runs from 2 (at i = 0, early January) to 8 (at i = 26, around 2021-07-11), mean 5, plus noise with standard deviation 0.5. So this synthetic flu peaks mid-year.
- The 12 forecast weeks run 2024-01-07 to 2024-03-24.

Expected output: the forecast plot shows black dots (data) following a smooth wave and a blue line with a shaded band. For the 12 forecast weeks the line should start near the yearly low (the true curve is 2.0 at the first forecast week) and rise to roughly 4.3 by the last one (true curve value 4.28), with a band about ±1 wide (noise 0.5 × 1.96 ≈ 0.98, plus trend uncertainty). The components plot shows a nearly flat trend near 5, a yearly wave of about ±3, and the Eid term.

1. Generate 3 years of synthetic weekly ILI data with annual seasonality + noise.

ANSWER

Done by the first block: `pd.date_range("2021-01-04", periods=156, freq="W")` gives 156 weekly dates (156 / 52 = 3 years). The sine term with period 52 weeks is the annual seasonality; `rng.normal(0, 0.5, 156)` adds noise; `.clip(0)` stops negative case counts (the lowest true value is 2, so clipping almost never acts). The seed 42 makes everyone's data identical. Prophet needs the columns named exactly `ds` (date) and `y` (value).

2. Fit Prophet with yearly and weekly seasonality.

ANSWER

`yearly_seasonality=True` fits the annual wave as a Fourier series with P = 365.25 days (P2 slide 10). The skeleton sets `weekly_seasonality=False`, and that is correct for this data: weekly seasonality models a pattern *within* a week (Monday vs Sunday), but here there is one point per week, always a Sunday, so there is nothing to learn. Turning it on would add useless terms. Weekly seasonality only makes sense for daily (or finer) data, like daily hospital admissions.

Small detail: the synthetic cycle is 52 weeks = 364 days, Prophet's year is 365.25 days, a drift of 1.25 days per year, too small to matter.

3. Plot the forecast with 95% uncertainty intervals.

ANSWER

`m.plot(forecast)` draws the data, `yhat` (the forecast line) and the shaded band from `yhat_lower` to `yhat_upper`. Prophet's default band is 80%, so pass `interval_width=0.95` to `Prophet(...)` to get 95%. Expected: the 95% band is wider than the 80% one, and it widens further into the future because Prophet simulates possible future trend changes.

4. Add a holiday for “Eid” (spike in healthcare visits) and observe the effect.

ANSWER

The `holidays` table lists three Eid dates; `upper_window: 3` extends each to the 3 days after (4 days in all). Prophet adds an h(t) term that is active only on those days.

Expected effect with the skeleton data: almost none. Two reasons:

1. The synthetic data has no Eid spike built in, so the fitted Eid effect is near 0 (just noise).
2. The data only has Sundays. 2021-05-13 (Thu) to 05-16 covers Sunday 2021-05-16; 2023-04-22 (Sat) to 04-25 covers Sunday 2023-04-23; but 2022-05-03 (Tue) to 05-06 contains no

Sunday, so that Eid matches no data point at all.

To see a real effect, add +3 to y on the Eid weeks (as in the full code). Expected: without the holiday the model treats the spikes as noise and its band widens a little; with the holiday the components plot shows an Eid bar near +3 on those weeks and the trend and seasonality stay clean. This is the h(t) idea from P1 slide 16: known one-off events are modelled, not mistaken for trend.

5. Change changepoint_prior_scale from 0.05 to 0.5 and compare trend flexibility.

ANSWER

`changepoint_prior_scale` controls how big the slope changes δ at changepoints may be (the sparse prior, P2 slide 10).

- **0.05 (default):** stiff trend. The true trend here is flat at 5, so the expected trend line is nearly flat around 5.
- **0.5:** 10× looser prior. The trend can bend at many changepoints, so it wiggles and starts to absorb noise and bits of the seasonal wave. The future band also gets wider, because Prophet projects larger possible trend changes forward.

Lesson: higher values react faster to real breaks (like a sudden outbreak, Discussion Q2) but overfit noise on stable data. Pick the value by checking forecast error on held-out weeks.

Maths for Week 10

This chapter is applied, but several numbers and formulas appear on the slides: speedup from parallel forecasting, the Prophet model and its trend and Fourier parts, network reach in hops, the power law for followers, modularity for Louvain, the Kempe greedy guarantee, spike ratios, STL anomaly thresholds, VADER cut-offs and storage sizes. Every number below was recomputed in Python.

Speedup of parallel batch forecasting

```
Speedup = T_sequential / T_parallel;   time per model =
T_sequential / number of models;   models per executor =
models / executors
```

WHAT EACH SYMBOL MEANS

- T_sequential: time to fit all models one after another on one machine
- T_parallel: time on the Spark cluster
- executor: a Spark worker process that runs tasks

WORKED EXAMPLE

P1 slide 19: 5,000 SKUs, 8 hours sequential, about 12 minutes on 20 executors.

1. 8 h = 480 min
2. Speedup = 480 / 12 = **40×** (matches the slide)
3. Time per model = 8 × 3600 s / 5,000 = **5.76 s**
4. Models per executor = 5,000 / 20 = **250**
5. If each executor ran one model at a time, the best case would be 480 / 20 = 24 min. Getting 12 min means about 40 models fit at once, e.g. 2 cores per executor.

PRACTICE

Q: The chain grows to 10,000 SKUs. Each model still takes 5.76 s and the cluster runs 40 models at once. Sequential time and parallel time?

A: Sequential = 10,000 × 5.76 = 57,600 s = **16 h**. Parallel = 57,600 / 40 = 1,440 s = **24 min**. Speedup stays 960 / 24 = 40×.

Prophet additive model

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$$

WHAT EACH SYMBOL MEANS

- y(t): the value at time t (e.g. admissions this week)
- g(t): trend, long-run growth (piecewise linear or logistic)
- s(t): seasonality, repeating cycles (Fourier series)
- h(t): holiday or event effect on listed dates
- ε(t): noise, what the model cannot explain

WORKED EXAMPLE

An Eid week: trend g = 100 admissions, seasonal effect s = +15, Eid effect h = +30, noise ε = −2.

y = 100 + 15 + 30 − 2 = **143**.

PRACTICE

Q: A forecast says y = 120 for a non-holiday week, with g = 110 and noise 0. What is s?

A: h = 0 (no holiday), so s = 120 − 110 − 0 − 0 = **+10**.

Prophet piecewise linear trend with changepoints

$$g(t) = (k + a(t)^{T}\delta)\, t + (m + a(t)^{T}\gamma)$$

WHAT EACH SYMBOL MEANS

- k: base growth rate (slope)
- m: offset (starting level)
- δ: slope changes, one per changepoint; sparse prior keeps most near 0
- a(t): 0/1 vector, 1 for every changepoint already passed at time t
- γ: offset corrections so the line stays joined at each changepoint (for one changepoint at s: γ = −sδ)

WORKED EXAMPLE

k = 0.5, m = 1, one changepoint at t = 10 with δ = 0.3, so γ = −10 × 0.3 = −3.

1. Before the changepoint, t = 10: g = 0.5 × 10 + 1 = **6**
2. After, slope = 0.5 + 0.3 = 0.8. At t = 10: 0.8 × 10 + 1 − 3 = **6** (line stays joined)
3. At t = 20: 0.8 × 20 + 1 − 3 = **14**

PRACTICE

Q: Same model but δ = −0.5 (a slowdown). Slope after t = 10 and g(20)?

A: Slope = 0.5 − 0.5 = 0. γ = −10 × (−0.5) = +5. g(20) = 0 × 20 + 1 + 5 = **6**: flat after the changepoint, still joined at 6.

Prophet Fourier seasonality

```
s(t) = Σ (n = 1 to N) [ a_n cos(2πnt / P) + b_n sin(2πnt / P) ]
```

WHAT EACH SYMBOL MEANS

- P: period in days: 365.25 for yearly, 7 for weekly (P2 slide 10)
- N: number of terms; larger N gives a wigglier, less smooth pattern
- a_n, b_n: weights the model learns; there are 2N of them
- t: time in days

WORKED EXAMPLE

Weekly seasonality (P = 7), N = 1, $a_1 = 2$, $b_1 = 0$.

- t = 0: s = 2 cos(0) = **2**
- t = 3.5 (half a week): s = 2 cos(π) = **−2**
- t = 1.75: s = 2 cos(π/2) = **0**

Number of weights: N = 10 gives 2 × 10 = 20; N = 3 gives 6.

PRACTICE

Q: Yearly seasonality (P = 365.25), N = 1, $a_1 = 0$, $b_1 = 3$. Value at t = 365.25/4 (a quarter year)?

A: s = 3 sin(2π × 0.25) = 3 sin(π/2) = **3**.

Reach within h hops (tree estimate)

```
Reach ≤ d + d² + … + d^h
```

WHAT EACH SYMBOL MEANS

- d: average out-degree (followers each user passes the post to)
- h: number of hops
- ≤: an upper bound, because overlapping friends are counted more than once

WORKED EXAMPLE

P2 slide 15: d = 45, h = 3.

$45 + 45^2 + 45^3 = 45 + 2{,}025 + 91{,}125 =$ **93,195** users, about 0.22% of 41.7M.

PRACTICE

Q: d = 30, h = 3?

A: 30 + 900 + 27,000 = **27,930** users.

Power-law follower distribution

```
P(k) ∝ k^(-α),   so P(ck) / P(k) = c^(-α)
```

WHAT EACH SYMBOL MEANS

- k: number of followers
- P(k): share of users with k followers
- α: exponent, ≈ 2.276 for Twitter followers (P2 slide 6)
- ∝: "proportional to"

WORKED EXAMPLE

How common are users with twice as many followers? 2^(−2.276) = **0.206**, about 1/5 as common.

PRACTICE

Q: Ten times as many followers?

A: 10^(−2.276) = **0.0053**, about 1 in 189 as common. That is why only the top 0.05% have more than a million followers.

Reciprocity

```
Reciprocity = mutual follow links / all follow links × 100%
```

WHAT EACH SYMBOL MEANS

- mutual link: A follows B and B follows A

WORKED EXAMPLE

Twitter: 22.1% mutual (P2 slide 6), so 100 − 22.1 = **77.9%** are one-way. Of 1.47 billion edges, 0.221 × 1.47B ≈ **324.87 million** are mutual.

PRACTICE

Q: A network has 2 million edges, 1.5 million of them mutual. Reciprocity?

A: 1.5 / 2 = **75%**: much closer to a friendship network like Facebook (≈100%) than to Twitter.

Modularity (what Louvain maximises)

```
Q = Σ_c [ L_c/m - (d_c / 2m)² ]
```

WHAT EACH SYMBOL MEANS

- c: each community
- L_c: edges inside community c
- d_c: sum of degrees of nodes in c
- m: total edges in the graph
- L_c/m is the actual fraction inside; (d_c/2m)² is the fraction expected by chance (the slide's definition, P1 slide 9). Higher Q = better split.

WORKED EXAMPLE

Two triangles joined by one edge: 6 nodes, m = 7. Each triangle is a community: L_c = 3, degrees 2 + 2 + 3 = 7.

Per community: 3/7 − (7/14)² = 0.4286 − 0.25 = 0.1786. Q = 2 × 0.1786 = **0.357**.

All nodes in one community: Q = 7/7 − (14/14)² = **0**. So the two-triangle split is better, and Louvain would keep it.

PRACTICE

Q: Two 4-node rings (each a 4-cycle) joined by one edge. Q for the two rings as communities?

A: m = 4 + 4 + 1 = 9. Each ring: $L_c = 4$, $d_c = 2+2+2+3 = 9$. Per ring: $4/9 - (9/18)^2 = 0.4444 - 0.25 = 0.1944$. Q = 2 × 0.1944 = **0.389**.

Greedy influence maximisation guarantee

```
Greedy spread ≥ (1 - 1/e) × optimal spread ≈ 0.632 × optimal
```

WHAT EACH SYMBOL MEANS

- e ≈ 2.718, the base of natural logarithms
- optimal spread: the best possible reach with k seeds (NP-hard to find exactly)

WORKED EXAMPLE

If the best k seeds would reach 10,000 users, Kempe et al.'s greedy method (P1 slide 10) is guaranteed at least 10,000 × 0.632 = **6,321** users.

PRACTICE

Q: Optimal reach 50,000. Guaranteed greedy reach?

A: 50,000 × (1 − 1/e) = **31,606** users.

Spike ratio for trend and anomaly detection

```
Spike ratio = current rate / baseline rate
```

WHAT EACH SYMBOL MEANS

- baseline rate: the normal (historical) frequency
- trend detection compares to the baseline, not absolute volume (P1 slide 12)

WORKED EXAMPLE

A phrase normally at 10/hour jumps to 10,000/hour: 10,000 / 10 = **1,000×**, so it is trending (P1 slide 12). Same ratio for the bKash agent going from 10 to 10,000 transactions per hour (P1 slide 18).

PRACTICE

Q: #QurbaniEid showed a 1,200× spike (P1 slide 12). If its baseline was 50 posts/hour, what was the peak rate?

A: 50 × 1,200 = **60,000 posts/hour**.

STL residual anomaly rule

```
Flag a point if |residual| > k × IQR(residuals), IQR = Q3 - Q1
```

WHAT EACH SYMBOL MEANS

- residual: what is left after removing trend and seasonality (STL, P1 slide 18)
- Q1, Q3: first and third quartiles (25th and 75th percentiles)
- IQR: interquartile range, the spread of the middle half
- k: a chosen multiplier (the slide leaves it open; 1.5 or 3 are common)

WORKED EXAMPLE

Residuals: 0.2, −0.1, 0.3, −0.4, 0.1, 5.0, −0.2, 0.0. Sorted: −0.4, −0.2, −0.1, 0.0, 0.1, 0.2, 0.3, 5.0.

1. Q1 = −0.125, Q3 = 0.225 (linear interpolation, Python `statistics.quantiles(method="inclusive")`)
2. IQR = 0.225 − (−0.125) = 0.35
3. k = 3: threshold = 3 × 0.35 = 1.05
4. Only |5.0| > 1.05, so the 6th point is the **anomaly**.

PRACTICE

Q: Same data, k = 1.5. Which points are flagged?

A: Threshold = 1.5 × 0.35 = 0.525. Largest normal value is |−0.4| = 0.4 < 0.525, so still **only 5.0** is flagged.

VADER sentiment classes

```
compound > 0.05 → positive; compound < -0.05 → negative;
otherwise neutral; compound ∈ [-1, +1]
```

WHAT EACH SYMBOL MEANS

- compound: VADER's single overall score for a text (P1 slide 11)

WORKED EXAMPLE

Scores 0.62, 0.03, −0.40 → **positive, neutral, negative**.

PRACTICE

Q: Classify 0.05 and −0.06.

A: 0.05 is not greater than 0.05, so **neutral**; −0.06 < −0.05, so **negative**.

Storage size for genomics

```
Total = size per genome × number of patients (× replication factor on HDFS)
```

WHAT EACH SYMBOL MEANS

- size per genome: about 200 GB (P1 slide 21)
- 1 PB = 1,000,000 GB (decimal units)
- replication factor: HDFS keeps 3 copies by default

WORKED EXAMPLE

10,000 patients × 200 GB = 2,000,000 GB = **2 PB** (matches P1 slide 21).

PRACTICE

Q: 2,500 patients stored on HDFS with replication 3. Raw disk needed?

A: 2,500 × 200 = 500,000 GB = 500 TB; × 3 = **1.5 PB**.

Correlation from r^2 (Google Flu Trends)

```
r = √(r²)
```

WHAT EACH SYMBOL MEANS

- r^2: share of variation explained by the model (0 to 1)
- r: correlation coefficient

WORKED EXAMPLE

GFT had $r^2 = 0.97$ on 2003-2008 data (P2 slide 14), so $r = \sqrt{0.97} =$ **0.985**. Still, it over-predicted by 2× in 2012-2013: in-sample fit says nothing about future seasons when behaviour changes.

PRACTICE

Q: True ILI rate is 3% and GFT over-predicts by 2×. What did GFT report?

A: 3% × 2 = **6%**.